

VirtualBow Theory Manual

v0.10.0



Contents

1	Introduction	3
2	Bow Model	4
2.1	Scope of the Model	4
2.2	Literature Review	6
2.3	Beam Theories	9
2.4	Model Definition	15
3	System Equations	18
3.1	Element Equations	18
3.2	System Equation	19
3.3	Connectivity	20
4	Element Equations	22
4.1	Mass Element	22
4.2	Beam Element	24
4.2.1	Corotational Formulation	26
4.2.2	Castigliano's Second Theorem	31
4.2.3	Stiffness Matrix of a Beam Segment	33
4.2.4	Cross-Section Properties	41
4.2.5	Lumped Masses	44
4.2.6	Damping Matrix	45
4.3	String Element	47
4.3.1	Internal Forces	47
4.3.2	Lumped Masses	51
4.3.3	Damping	52
5	Static Solution	53
5.1	The Basic Newton-Raphson Method	54
5.2	Load- and Displacement Control	55
5.3	Convergence Criterion	57
5.4	Derivative of the Equilibrium Displacements	59
5.5	Finding the Braced Equilibrium of a Bow	60

6	Dynamic Solution	62
6.1	Newmark- β Method	63
6.2	Adaptive Step Size	65
6.3	Stopping Criterion and Progress Estimation	68
6.4	Stopping Event Detection and Location	70
7	Modal Analysis	73
7.1	Generalized Eigenvalue Problem	73
7.2	Evaluating Modal Properties	74
A	Limb Geometry	75
A.1	Profile Curve	75
A.1.1	Spiral Segment	75
A.1.2	Arc Segment	76
A.1.3	Line Segment	77
A.1.4	Spline Segment	77
A.2	Width and Thickness	77
A.2.1	Cubic C^2 Spline	77
A.2.2	Monotonicity	79
B	Element Details	80
B.1	Derivatives for the Beam Element	80
B.2	Beam Stiffness Matrix by FEM	82
B.3	Derivatives for the String Element	85
B.4	Vibrations of a Viscoelastic Bar	88
C	Unfinished Work	91
C.1	Static Step Size Control	91
C.2	Backtracking Line Search	93
C.3	Generalized- α Method	96
	References	98

Chapter 1

Introduction

VirtualBow is a software tool for designing and simulating bows. This documentation explains the theory and reasoning behind the internal simulation methods used for computing the results. It is still a work in progress and continues to evolve alongside the software, so it may not always be complete and up to date. Note that reading and understanding this documentation is not required for using the VirtualBow software successfully – the user manual should be sufficient for that. Instead, it serves as a reference for developers and interested users who want to learn more about the internals of VirtualBow.

The chapters of this document roughly follow the course of a bow simulation, from the underlying model to the numerical methods used to solve it. The *Bow Model* chapter identifies the physical effects that need to be captured, reviews existing research and motivates the finite element model that VirtualBow is built on. The next two chapters develop this model in detail: *System Equations* introduces the general finite element framework and shows how individual elements combine into a single equation of motion for the whole bow. *Element Equations* develops the required element types for modeling the limbs, string and arrow. With the model in place, the remaining chapters cover its solution. *Static Solution* treats the bracing and drawing of the bow, where the system is assumed to be in static equilibrium at every step. *Dynamic Solution* then handles the shot itself as a time-dependent process, determining the motion of the limbs, string and arrow.

Chapter 2

Bow Model

A scientific model is a simplified and abstracted representation of reality, often expressed in a mathematical way. A good model reduces the complexity of a real system by capturing only its most significant aspects and neglecting less significant ones. Analyzing this simplified representation can then lead to new insights and conclusions about the real system. Which aspects of reality a model must reflect depends on the kinds of questions it intends to answer. The first step in developing a bow model is therefore to clarify its scope and intended application.

2.1 Scope of the Model

In the case of VirtualBow, the purpose of the bow model is to give users the ability to evaluate the bow designs they come up with. So what characteristics make a bow design a good one and what are the implications for the bow model? Many things could be considered, but we mainly focus on the following three categories:

1. **Viability:** Can the design be realized in practice? The most important limitation on a bow's viability is whether the chosen materials can withstand the loads that occur during use. To assess this, the model must accurately predict the distribution of stress across the limbs as well as the tensile forces in the string throughout the shot cycle. Another possible limitation on the viability of a design is stability. As an example, imagine a bow with very deep recurves and narrow limbs. If taken too far, such a design might be prone to the limbs twisting and the bow "flipping around", making it essentially useless. Other designs might "flip-flop" between two states where the tension switches between the lower and upper limb. Both might happen long before any material limit has been reached. The common reason is that one of the intermediate states of the bow is unstable, meaning that the system has no tendency to return to that state after a small disturbance. This is something that VirtualBow cannot yet predict, but it falls well within this category and is therefore planned for the future. For now we have to limit ourselves to material failure only.

2. **Performance:** Being viable is of course only the bottom line for a bow. Ideally we also want to get good performance out of it. Important performance characteristics include the arrow's velocity and the bow's efficiency, i.e. the percentage of the stored elastic energy that is transferred to the arrow during the shot. To capture these, the model has to represent all major mechanisms through which a bow loses efficiency. We will examine these mechanisms in more detail when reviewing existing research.
3. **Comfort:** Raw performance isn't everything either. A bow design is also judged by how it feels to the archer: the draw force, the shape of the draw curve (does it feel smooth or is there stacking) and the dynamic forces exerted on the archer in the form of hand shock. Acoustic vibrations technically belong in this category as well, but this is where we draw the line to keep the complexity manageable.

Any physical effects in archery that don't significantly affect one of those categories above fall outside the scope of VirtualBow. One example is *exterior ballistics*, i.e. everything that happens to the arrow and its flight path after it leaves the bow. Another example is the elastic flexing motion of the arrow, which depends on its dynamic spine, and its interaction with the string and the arrow rest. Both topics are important in their own right, but they do not significantly feed back into the fundamental design decisions of a bow and are therefore not included in the model.

Another decision that VirtualBow has to make is what kinds of bow designs it supports. There is a wide variety of design elements in bow making, and it would be impractical to support each and every one of them. This is true not only from the perspective of physical modeling but also from a UI perspective: keeping the user interface simple for the most common types of bows while ideally still allowing more complex designs. That said, some of the limitations that VirtualBow currently places on bow designs are just technical limitations, not deliberate decisions, and will hopefully be lifted in the future. An example is that cross-sections have to be rectangular and made up of horizontal layers. This is just a result of VirtualBow first focusing on the most common case of (glass-)laminated bows, since that's how most modern bows are made. In the long term, non-rectangular cross-sections and more complex internal compositions are very desirable features. Another major design limitation imposed by VirtualBow is that of symmetry: The upper and lower limbs are identical to each other and the pivot point of the handle and the nocking point on the string are both located exactly at the center of the bow. In reality, however, due to the arrow rest being located above the handle's pivot point, almost every bow is slightly asymmetric, even if we disregard extreme cases such as the typical Japanese Yumi longbow. This asymmetry is usually compensated for by one limb being slightly stronger than the other, giving the bow either a positive, neutral or negative tiller depending on the desired shooting style. Whether asymmetry will ever be included in VirtualBow is not entirely clear yet. On the one hand, it is a property that almost every bow has; on the other hand, for most bows it is more a question of fine-tuning than something that has a big impact on the fundamental properties of a bow design as defined by the three categories above.

Now that we have clarified the scope of VirtualBow's internal bow model at least somewhat, the next question becomes: What does a bow model have to look like in order to capture the things listed above? That's not easy to answer, and there is usually a certain amount of experience as well as trial and error involved in physical modeling before getting it right. Luckily, some scientific research on bow and arrow physics has already been done that we can draw inspiration from. The next section therefore examines some of that research before we go ahead and define our own model in detail.

2.2 Literature Review

Research on the mechanics of bow and arrow goes back to at least the 1930s, when C.N. Hickman, P.E. Klopsteg and others started to perform systematic experiments and developed simple mathematical models. Without modern computers available, these early models were still limited in their complexity by what could be solved analytically. Their focus was therefore on qualitative insight rather than accurate prediction. One such early model is the *Hickman Model* [15], published in 1937. It assumes the bow to consist of two rigid limbs that are connected to the stationary midsection via elastic hinges as shown in Figure 2.1. Even though this is obviously a huge simplification, this model can already be used to explain the general shape of the force-draw curve of a bow, including stacking, and to investigate a bow's efficiency, i.e. how it transfers its stored elastic energy to an arrow.

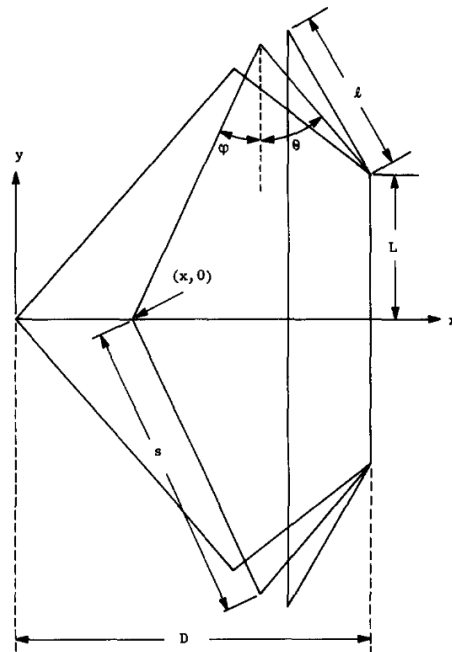


Figure 2.1: Hickman model as used by W.C. Marlow [23]

An important modification to the Hickman model was suggested by W.C. Marlow in 1980 [23]. It was already known that the original Hickman model severely overpredicted

a bow's efficiency. Marlow mentions predicted efficiencies over 90% while measurements suggest values between 70% and 85% for most bows. Popular explanations for this discrepancy were effects like hysteresis and air resistance. Marlow, however, found another reason: With the assumption of an inextensible string, as was the case in Hickman's original model, the arrow exits the bow exactly at brace height, where the limbs are at the reversal point of their motion and therefore have zero velocity. Only the kinetic energy of the string remains in the bow; the rest is transferred to the arrow. Marlow then showed that accounting for the elasticity of the string shifts the point of exit for the arrow away from the braced position, such that the limbs still have substantial kinetic energy left. In his numerical example this modification reduces the predicted efficiency to a much more realistic 78%, down from 92% previously. Of the energy remaining in the bow, 11% was the kinetic energy of the limbs, 9% the kinetic energy of the string and 2% the potential energy of limbs and string.

An actual elastic limb, as opposed to the rigid simplification used in Hickman's model, might not have such a clear reversal point with zero kinetic energy. But there is still a relevant lesson in Marlow's work: The efficiency of a bow is mainly determined by the kinetic energy that stays in the system due to the mass and elasticity of the string and the mass distribution of the limbs. The contribution of dissipative effects like air resistance and hysteresis to the overall efficiency was estimated by Marlow to be less than 2% and 3%, respectively. This is good news for us, because those dissipative effects are very tricky to model accurately. But we will still have to think about them eventually, because a model without any form of motion decay won't look very convincing either.

Moving forward in time, the widespread availability of digital computers eventually made it possible to simulate far more detailed models of bows and arrows, enabling much more accurate predictions. Significant contributions in this area were made by Dr. Bob Kooi and his co-authors, who published numerous papers on bow and arrow mechanics throughout the 1980s and 1990s, applying numerical methods to solve increasingly complex mathematical models.

The first of those papers is titled *On the static deformation of a bow* [18] and investigates the storage of deformation energy in bows with and without recurve. Figure 2.2 was taken from that paper and gives an impression of the model used there. The bow is assumed to be symmetric and to have an inextensible string. The limb is represented as an *Euler-Bernoulli* beam with varying bending stiffness along its length. We will have a closer look at the various beam theories, including Euler-Bernoulli, later. Kooi uses the general form of the Euler-Bernoulli beam's differential equation that accounts for arbitrarily large deformations. Combining this with geometric constraints for the string and suitable boundary conditions leads to the bow's equations of equilibrium, a *free-boundary value problem for a set of ordinary differential equations*. These equations of equilibrium are solved numerically for specific draw lengths of the bow.

In subsequent papers titled *On the mechanics of the bow and arrow* [16] and *On the mechanics of the modern working-recurve bow* [17] Kooi also considers the dynamic case.

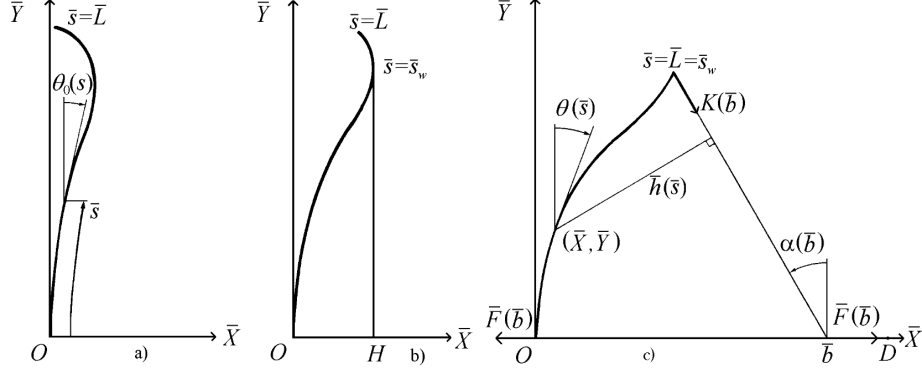


Figure 2.2: Bow model used by Kooi [18]

Like Marlow, he uses a linear-elastic string whose mass is lumped into two point masses at the string center and the limb tip. The string itself does not have any dynamic behavior on its own; in particular, it doesn't bend or flex in the transversal direction. Instead, it just acts as an ideal spring between the string center and either the limb tip or the contact point of the string on the bow limb. The arrow is modeled as a concentrated point mass sitting at the center of the string. The equations of motion for the system now form a *moving-boundary value problem for a set of partial differential equations*. They are solved numerically by using a *finite-difference method with complete discretization*. This means that both space and time are approximated by a discrete "grid" of values. The partial derivatives in the governing equations are then approximated by finite difference quotients operating on those grid points. This results in a large, but finite, number of difference equations for the unknowns that can be solved with a computer program.

In the paper *On the Mechanics of the Arrow: Archer's Paradox* [19], Kooi refines his model even more by including an elastic arrow in sliding contact with the bow's grip. This makes it possible to analyze the *Archer's Paradox*: The phenomenon of arrows going around the grip of a bow by performing a bending motion in the horizontal plane. The dynamics of the arrow are, however, treated as partly decoupled from the bow's dynamics: Only the influence of the bow on the arrow's motion is considered; the other way around is neglected. This is in line with our assumption from Section 2.1 that the bending motion of the arrow doesn't contribute much to a bow's overall performance and is therefore out of scope for our model.

2.3 Beam Theories

At this point it makes sense to take a brief look at different beam theories. A beam, in this context, is a slender structural element whose length is much greater than its cross-sectional dimensions, and which can resist loads applied perpendicular to its longitudinal axis through bending, shear and sometimes torsion. All beam theories share the same basic approach: they reduce the mechanical properties of each cross-section to an idealized, infinitely thin centerline, which then serves as the basis for analyzing the beam's static or dynamic behavior.

Linear vs. Nonlinear

In many engineering applications, a beam can be assumed to bend only slightly. These small deflections do not change the shape of the beam enough to significantly affect its internal balance of forces. If the materials are also linear-elastic – meaning stresses increase proportionally with strains – then the entire beam responds in a linear way: increasing the externally applied loads proportionally increases the deformation. Problems in this linear regime can usually be solved analytically, with many solutions to standard cases available in textbooks. According to the preconditions listed above, linear beam theory can break down for two main reasons:

1. **Geometric nonlinearity:** The assumption of small deformations does not hold. The beam deflects enough that the balance of forces in the final, deflected state is no longer similar to that in the original, undeflected state.
2. **Material nonlinearity:** The material does not follow a simple proportional relationship between stress and strain.

Most common materials – including those used for making bows – behave linearly up to a certain level of strain, called the *proportional limit* (see Figure 2.3). Beyond this limit, the stress-strain curve begins to deviate from the linear projection. Plastic (irreversible) deformation and eventually failure of the material follow.

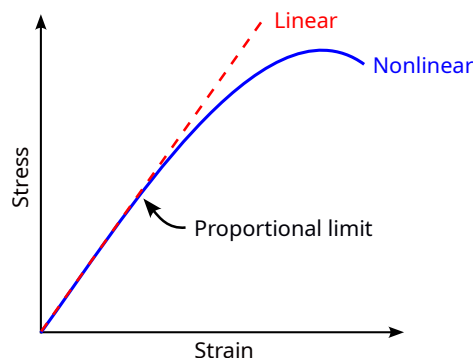


Figure 2.3: Linear vs. nonlinear material behavior

If we assume the materials in a bow to operate safely below their elastic limit, which is

the point where plastic deformation sets in, they will usually also stay within their proportional limit. Assuming linear material behavior in a bow therefore seems reasonable; we just have to keep in mind that the simulation results become invalid if stresses beyond the proportional limit occur. But at that point the bow design usually has to be rejected anyway. The deflections of a bow limb, however, are far too large to treat in a geometrically linear way, making the problem geometrically nonlinear even if the material itself remains linear-elastic. Geometrically nonlinear bending problems can't generally be solved analytically and therefore require numerical methods for computing an approximate solution.

Cross-Section Kinematics

Now that we have established that we need the nonlinear form of any beam theory we might consider, the main point of distinction between the various beam theories becomes how they assume the cross-sections to deform in relation to the deformation of the beam's centerline.

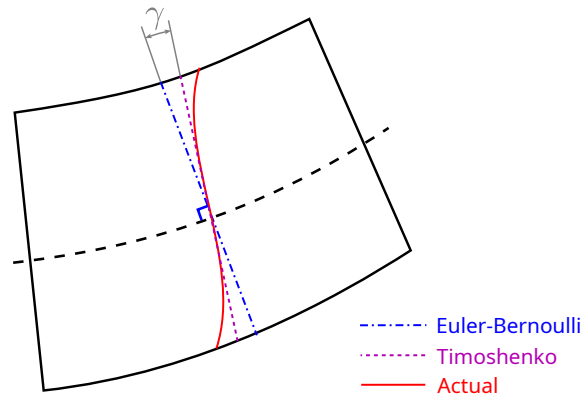


Figure 2.4: Deformation of a beam segment according to different beam theories (exaggerated)

The oldest and most widely used beam theory is the Euler-Bernoulli theory, which we already came across in Kooi's bow models. In this theory, all cross-sections of a beam are subject to the following assumptions:

1. Initially planar cross-sections stay planar during the deformation of the beam
2. Cross-sections perpendicular to the beam's centerline stay perpendicular during deformation

In Figure 2.4, which schematically shows a deformed segment of a beam with its centerline, the Euler-Bernoulli case is shown as the blue, planar section perpendicular to the beam's centerline. A more accurate but also more complex beam theory is the *Timoshenko theory*. It relaxes the assumption of perpendicular cross-sections. This is shown in Figure 2.4 as the purple section, allowed to rotate by the so-called *shear angle* γ with respect to the

perpendicular orientation. But even this theory is not perfect, because cross-sections of a deforming beam don't stay exactly planar: they actually warp in a complicated way, which more advanced beam theories attempt to capture in varying amounts of detail.

So how do those theories affect the overall behavior of a beam and in which situations does the improved accuracy of more advanced theories make a difference? Consider a simple cantilever beam as shown in Figure 2.5, clamped on the left side and subjected to a transverse tip force F at the right.

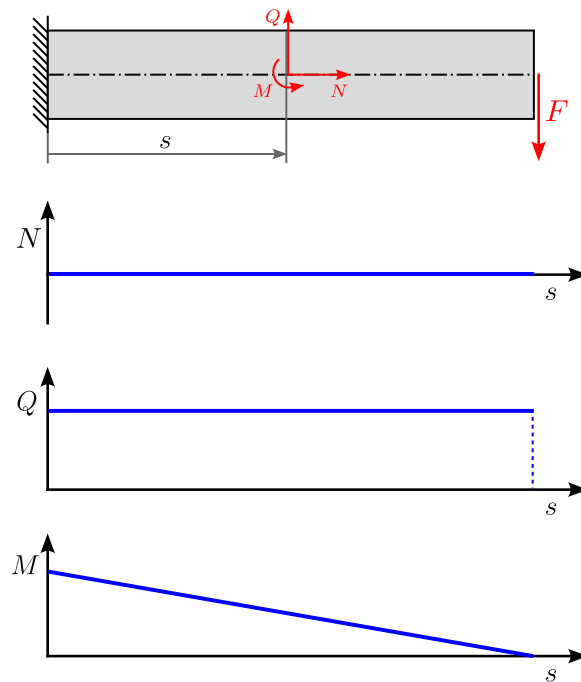


Figure 2.5: Cross-section forces of a cantilever beam with tip load

Let's call the length along the beam axis s . At each length s there are three internal forces that can potentially act on the cross-section: a normal force N perpendicular to the section, a shear force Q parallel to the section and a bending moment M around the section's rotation axis. In the static case, those are determined by the condition that each section has to be in balance with the externally applied load F . For this cantilever beam this means that the normal force N is zero over the length s , the shear force Q is constant and opposed to F and the bending moment M has a linear distribution, since it is proportional to the "lever arm" between the section in question and the external force. Each of these section forces has a different effect on the deformation of the beam:

- **The bending moment M** , as its name already suggests, causes a bending deformation of the beam as shown in Figure 2.6 (1). The cross-sections develop a linear stress and strain distribution in the normal direction. This bending stress is greatest at the top and bottom surfaces of the beam and zero at a single point in between, which defines the so-called *neutral fiber*. The resultant of those bending stresses is the bending moment M .

- **The shear force Q** also makes a distinct contribution to the beam's deformation, as shown in Figure 2.6 (2). The shear deformation corresponds to stresses and strains in parallel to the section and distorts the beam into a parallelogram-like shape. Note how the shear deformation causes the beam's cross-sections to rotate relative to the centerline.
- **The normal force N** , if present, would extend or contract the beam in the longitudinal direction.

As Figure 2.6 also shows, the **total deformation (3)** of the beam is a combination of the **bending deformation (1)** and the **shear deformation (2)** and also includes the warping of the cross-sections.

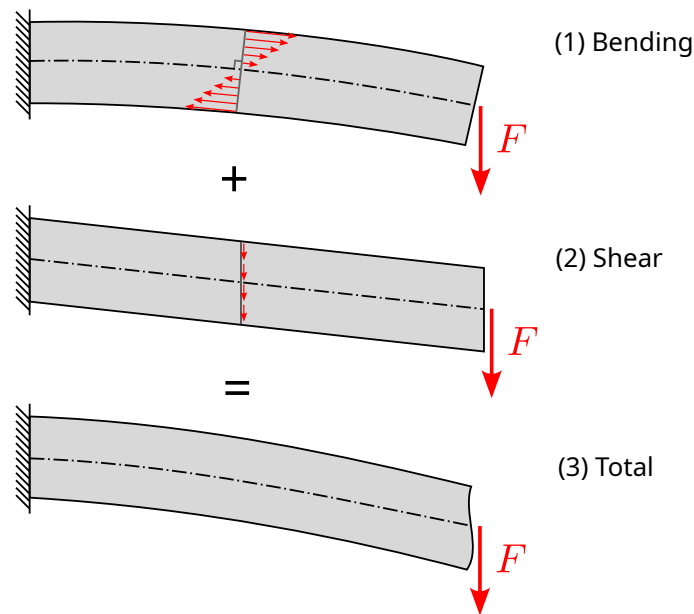


Figure 2.6: Deflection of a beam as a sum of bending, shear and warping

The Euler-Bernoulli theory only captures the bending aspect of the deformation. This is because shear deformation, as we saw, requires the cross-sections to rotate relative to the beam's centerline, which the Euler-Bernoulli theory does not permit. The Timoshenko theory, on the other hand, does capture the additional shear deformation for exactly this reason. So how relevant is this limitation of the Euler-Bernoulli theory? Does shear deformation actually meaningfully contribute to the overall deflection of a beam? This depends in large part on the "slenderness" of the beam. The more slender a beam is, i.e. the higher the *slenderness ratio* l/h of length to cross-section height, the more dominant the bending deformation becomes over the shear deformation and the more accurate the Euler-Bernoulli theory becomes. Short, stubby beams, on the other hand, can show significant shear deformation, requiring more advanced theories like Timoshenko's. This becomes plausible if we look at Figure 2.5 again: The shear force Q is constant; it only depends on the external force F and does not increase when the beam is made longer.

The peak of the bending moment M , however, increases with the length of the beam, due to the increasing leverage, leading to the bending effect becoming more dominant relative to the shearing.

Selection of a Beam Theory

Ideally we would like to choose the simplest theory that is accurate enough for our use case. Therefore the question we need to answer is whether or not a bow limb is slender enough to justify the application of the Euler-Bernoulli theory. A common rule of thumb states that beams with a slenderness ratio of $l/h > 10$ are a good match for the Euler-Bernoulli theory. It only takes a few back-of-the-envelope calculations to realize that basically all reasonable bow limb designs far exceed this minimum slenderness requirement, which seems to justify the use of the Euler-Bernoulli theory. But there is one more complication that we have to consider. Shear deformation can become important not only in short and stubby beams, but also in composite beams where layers of very soft and very stiff materials are combined with each other.

As an example, let's look at a classical sandwich beam as shown in Figure 2.7. It consists of a soft and light core of thickness t_c and shear modulus G_c , covered by two comparatively much stiffer face sheets of thickness t_f and with a Young's modulus of E_f . The figure shows how the beam would deform under pure bending (green/dashed line) and under the additional influence of the core's shear deformation (red/solid line).

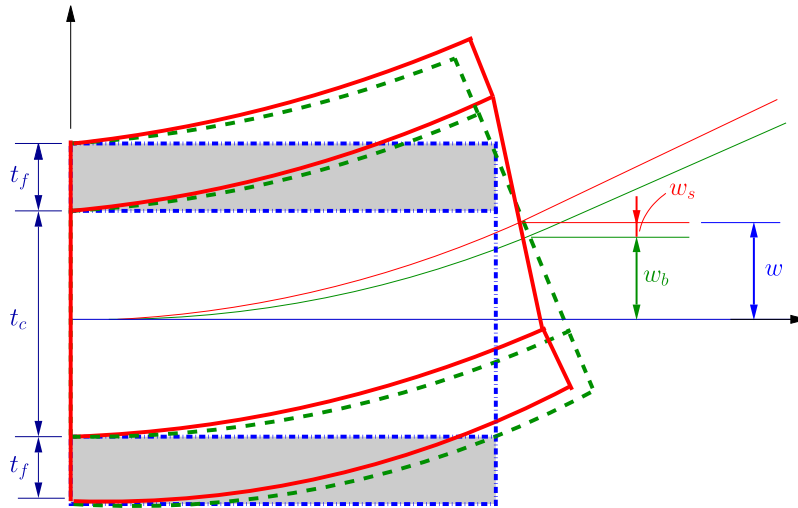


Figure 2.7: Deformation of a sandwich beam

According to [30], the deflection of such a beam, assuming that the stiffness of the face sheets is much higher than that of the core, is calculated as the sum of bending deflection w_b and shear deflection w_s as

$$w = w_b + w_s, \quad w_s = \frac{FL}{S}, \quad w_b = \frac{FL^3}{3D} \quad (2.1)$$

with the shear stiffness $S = \frac{G_c d^2}{t_c}$ and the bending stiffness $D = \frac{E_f t_f d^2}{2}$ and $d = t_c + t_f$ being the distance between the centers of the faces. This assumes small deflections, i.e. linear beam theory, but that's okay for this estimate. Since we are interested in the relevance of the shear deflection compared to the total deflection, we calculate the ratio of the two as

$$\eta = \frac{w_s}{w} = \frac{w_s}{w_b + w_s} = \frac{1}{1 + \frac{SL^2}{3D}} \quad (2.2)$$

Table 2.1 shows this value for a beam of length $l = 0.6$ m that is meant to be somewhat representative of a bow limb, and for different combinations of materials.

Face	Glass	Carbon	Carbon	Carbon
E_f	39000 MPa	180000 MPa	180000 MPa	180000 MPa
t_f	1.0 mm	1.0 mm	1.0 mm	1.0 mm
Core	Wood	Wood	Foam (stiff)	Foam (soft)
G_c	3600 MPa	3600 MPa	150 MPa	25 MPa
t_c	4 mm	4 mm	4 mm	4 mm
η	0.02 %	0.08 %	1.96 %	10.71 %

Table 2.1: Ratio of shear deflection for different material combinations. Material properties: Hypothetical soft wood with $\nu = 0.4$, others from [30].

The combination of a wooden core with fiberglass sheets, which is probably the most common pairing for modern bows, produces a negligible shear deflection of just 0.02% in this example. Switching the sheets from glass to carbon, which has a higher tensile stiffness, increases the shear deflection to 0.08%, which is still fairly small. Relevant shear deflection only starts to happen in this example with more exotic core materials, like synthetic foams. The material properties of foams that are actually used in modern bow limbs are not known to the author, so two hypothetical foams, stiff and soft, have been selected to cover a plausible range. It is probably safe to say, though, that at least the soft foam, which leads to a significant shear deflection of 10.71%, is not strong enough to actually work as a bow material in this combination.

This rough calculation suggests that shear deformation in a bow is negligible in most practical situations, which makes Euler-Bernoulli beam theory a good general modeling choice. Nevertheless, extreme material combinations where shear effects become significant might be possible, so it's worth keeping this in mind as a potential limitation of the model and an area for future improvement.

2.4 Model Definition

We have now learned all we need to know in order to define our own bow model. First of all, a few words on the general modeling approach. As we saw in the previous sections, any decently complex bow model cannot be solved analytically but has to be combined with some sort of numerical method for calculating an approximate solution. Models based on continuum mechanics, such as the Euler-Bernoulli beam theory, possess infinitely many degrees of freedom. This means that their state cannot be represented by a finite number of variables, only by continuous functions such as the deflection curve in the case of the Euler-Bernoulli theory. In order to be solvable by a computer program, such a model will eventually have to be approximated by a discrete model with finitely many degrees of freedom. This step is called *discretization*. Kooi, for example, like a true mathematician, formulates the complete equations of motion for his continuous bow model and only then starts to employ a discretization technique, in his case a finite difference method, to compute a solution for those equations. This results in a clear separation between the mathematical model and the numerical solution method.

In contrast, we're going to use a more engineering-like approach, starting with a discrete system in the first place. We do this by constructing the continuous parts of the bow from a large number of so-called *finite elements*. A finite element can be interpreted as an approximation of a small part of a continuous structure, but with only finitely many degrees of freedom. Those finite degrees of freedom are the positions and rotation angles of the nodes at which the elements are connected to each other. The more elements are used, the better the approximation gets and the closer the results will be to the exact solution. The finite element method leads to a system of ordinary differential equations, one for each degree of freedom, which can be solved by standard numerical methods as we will see later.

Figure 2.8 shows the finite element model of our bow. It is symmetric with respect to the horizontal plane and assumes the limbs to satisfy the Euler-Bernoulli assumptions, constructing them from geometrically nonlinear Euler-Bernoulli beam elements. A concentrated point mass at the limb tip accounts for any additional weight that might be placed there, such as tip overlays.

The string model already has some history behind it. Up to VirtualBow version 0.9, the string was modeled as a chain of point masses and linear springs. This model had a higher fidelity than Kooi's, since it also captured the transversal dynamics of the string. The differences were especially apparent in the motion of the string after the arrow left the bow, and for very light arrows. But this model also caused a number of issues. The chain-like setup sometimes behaved in strange and unexpected ways and tended to cause numerical problems for the simulation. The additional degrees of freedom made the model more complicated without actually improving the accuracy of key simulation results. Therefore, starting with version 0.10, a string model much more like Kooi's has been adopted. As shown in Figure 2.8, the string now consists of a single spring-like element, with some special treatment for possible contact with the limb for recurve bows, and whose mass is

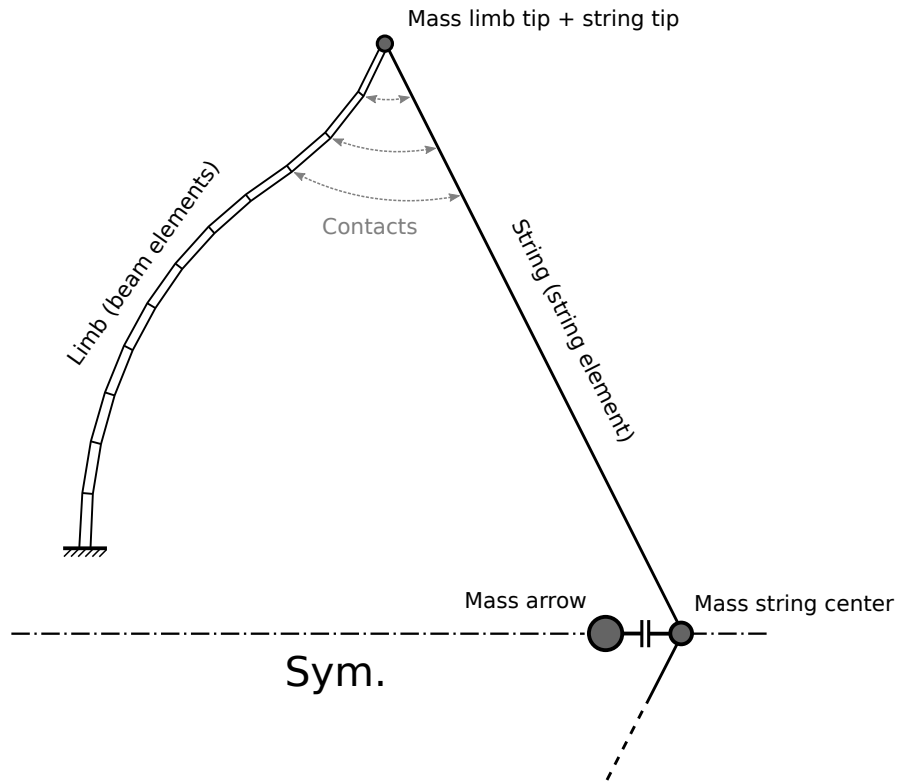


Figure 2.8: Finite element model of the bow

distributed to two point masses placed at the string center and the string tip. We limit ourselves to linear-elastic string materials for pragmatic reasons, since obtaining the exact nonlinear stress-strain curve of a particular string material is pretty difficult. Instead, any possible material nonlinearity will have to be accounted for by using an estimated average stiffness.

The arrow is another simple point mass, without any additional flexing motion, as justified earlier. It is connected to the string by an ideal, frictionless contact that releases the arrow cleanly as soon as the contact force becomes negative and exceeds a specified clamping force.

As we have seen, modeling dissipative effects is not particularly important for simulating the performance of a bow. However, a simulation without any energy dissipation doesn't really "look right", especially after the arrow has left the bow. So VirtualBow uses a crude way to include some configurable damping at the system level by assigning a desired damping ratio to the limbs and to the string. How this works in detail will be explained later.

All in all, there are three different types of elements present in this model:

- Beam Elements (Limbs)
- String Element (String)
- Mass Elements (Arrow + additional masses)

All of them have to account for arbitrarily large movements and are therefore geometrically nonlinear.

In the next chapters we will first show how the equations of motion for a finite element system are assembled from the equations of motion for the individual elements. We will then derive the equations of motion for the element types listed above. Finally we're going to discuss the numerical solution of those equations for the static and dynamic cases and how this is implemented in VirtualBow.

Chapter 3

System Equations

3.1 Element Equations

Consider a single, unconstrained finite element with $n \in \mathbb{N}$ degrees of freedom. Their number is the sum of the translational and rotational degrees of freedom of each of the element's nodes. The configuration of an element k at time $t \in \mathbb{R}$ will be described by the vector $\mathbf{u}_k(t) \in \mathbb{R}^n$ of nodal coordinates, which contains a position or angle for each degree of freedom of the element. For the kinds of elements we are interested in, the equation of motion, which governs the static and dynamic behavior of the element, takes the form

$$\mathbf{M}_k \ddot{\mathbf{u}}_k(t) + \mathbf{Q}_k(\mathbf{u}_k(t), \dot{\mathbf{u}}_k(t)) = \mathbf{P}_k(t). \quad (3.1)$$

This is a nonlinear ordinary differential equation of second order for the displacements $\mathbf{u}_k(t)$. Second order because it involves the nodal velocities $\dot{\mathbf{u}}_k(t)$ and accelerations $\ddot{\mathbf{u}}_k(t)$, which are the first and second derivatives of the nodal coordinates with respect to time. The matrix $\mathbf{M}_k \in \mathbb{R}^{n \times n}$ is called the element's *mass matrix*. It defines the element's inertia and must be constant and positive definite. The only nonlinear term in the equation is the vector $\mathbf{Q}_k \in \mathbb{R}^n$ of internal forces, which depends on the nodal positions and velocities in a possibly nonlinear way and captures effects such as elastic forces due to the element's deformation or damping forces due to the element's motion. Finally, the vector $\mathbf{P}_k(t) \in \mathbb{R}^n$ of external forces defines externally applied forces/moments at time t , one for each of the element's nodal coordinates.

In later chapters we are going to derive the actual components of this element equation for the various element types used in VirtualBow, but first let's consider the overall system that results from combining a number of such elements.

3.2 System Equation

If the overall finite element system has a total of N degrees of freedom, its configuration can be described by a global coordinate vector $\mathbf{u}(t) \in \mathbb{R}^N$. The local coordinates $\mathbf{u}_k$ of each element k are now no longer independent but instead directly related to the global coordinates of the system they are part of. There are many conceivable ways in which the local coordinates could be related to the global coordinates, but in our case we select the transformation

$$\mathbf{u}_k(t) = \mathbf{A}_k \mathbf{u}(t) + \mathbf{b}_k, \quad (3.2)$$

where $\mathbf{A}_k \in \mathbb{R}^{n \times N}$ is a linear transformation matrix for the element and $\mathbf{b}_k \in \mathbb{R}^n$ is a constant offset. Applying this kinematic relationship to the local equations of motion (3.1) leads to the element equations expressed in global coordinates,

$$\mathbf{A}_k^\top \mathbf{M}_k \mathbf{A}_k \ddot{\mathbf{u}}(t) + \mathbf{A}_k^\top \mathbf{Q}_k(\mathbf{A}_k \mathbf{u}(t) + \mathbf{b}_k, \mathbf{A}_k \dot{\mathbf{u}}(t)) = \mathbf{A}_k^\top \mathbf{P}_k. \quad (3.3)$$

And finally, by summing up the element contributions, the equation of motion of the overall system turns out to have the same structure as the element equations,

$$\mathbf{M} \ddot{\mathbf{u}} + \mathbf{Q}(\mathbf{u}, \dot{\mathbf{u}}) = \mathbf{P}(t) \quad (3.4)$$

where the global mass matrix $\mathbf{M} \in \mathbb{R}^{N \times N}$, the internal forces $\mathbf{Q} \in \mathbb{R}^N$ and the external forces $\mathbf{P} \in \mathbb{R}^N$ are calculated from the local element matrices as

$$\mathbf{M} = \sum_k \mathbf{A}_k^\top \mathbf{M}_k \mathbf{A}_k, \quad \mathbf{Q} = \sum_k \mathbf{A}_k^\top \mathbf{Q}_k(\mathbf{A}_k \mathbf{u} + \mathbf{b}_k, \mathbf{A}_k \dot{\mathbf{u}}), \quad \mathbf{P} = \sum_k \mathbf{A}_k^\top \mathbf{P}_k. \quad (3.5)$$

In order to solve this equation of motion for equilibrium, we're later going to need the derivative of the internal forces with respect to the nodal coordinates, which is called the *tangent stiffness matrix* $\mathbf{K} \in \mathbb{R}^{N \times N}$ of the system and can be calculated from the elements' local tangent stiffness matrices as

$$\mathbf{K} = \frac{\partial \mathbf{Q}}{\partial \mathbf{u}} = \frac{\partial}{\partial \mathbf{u}} \sum_k \mathbf{A}_k^\top \mathbf{Q}_k = \sum_k \mathbf{A}_k^\top \frac{\partial \mathbf{Q}_k}{\partial \mathbf{u}_k} \frac{\partial \mathbf{u}_k}{\partial \mathbf{u}} = \sum_k \mathbf{A}_k^\top \mathbf{K}_k \mathbf{A}_k. \quad (3.6)$$

Similarly, the *tangent damping matrix* $\mathbf{D} \in \mathbb{R}^{N \times N}$ is the derivative of the internal forces with respect to the nodal velocities and is calculated from the elements' tangent damping matrices as

$$\mathbf{D} = \frac{\partial \mathbf{Q}}{\partial \dot{\mathbf{u}}} = \frac{\partial}{\partial \dot{\mathbf{u}}} \sum_k \mathbf{A}_k^\top \mathbf{Q}_k = \sum_k \mathbf{A}_k^\top \frac{\partial \mathbf{Q}_k}{\partial \dot{\mathbf{u}}_k} \frac{\partial \dot{\mathbf{u}}_k}{\partial \dot{\mathbf{u}}} = \sum_k \mathbf{A}_k^\top \mathbf{D}_k \mathbf{A}_k. \quad (3.7)$$

3.3 Connectivity

The transformation matrices $\mathbf{A}$ and $\mathbf{b}$ for the connectivity of the elements are not totally arbitrary. In our case it is sufficient that any element coordinate u_i , where i is the index in the local coordinate vector, only depends on the corresponding global coordinate $\bar{u}_{m(i)}$, where $m(i)$ is a mapping from the local to the corresponding global coordinate index. The relationship between the two that we want to model is

$$u_i = a_i \bar{u}_{m(i)} + b_i \quad (3.8)$$

This includes the special case of $a_i = 0$, where the element coordinate is fixed at the value b_i and does not depend on a global coordinate. This can be used to implement fixed boundary conditions. Another special case is $a_i = 1$, where the local coordinate scales exactly like the global coordinate. In order to understand how this relates to the calculation of the transformation matrices $\mathbf{A}$ and $\mathbf{b}$, consider two local coordinates u_i and u_j ,

$$\begin{aligned} u_i &= a_i \bar{u}_{m(i)} + b_i \\ u_j &= a_j \bar{u}_{m(j)} + b_j \end{aligned}$$

The transformation with respect to those two displacements takes the following form, where only two rows in $\mathbf{A}$ contain a nonzero element a that relates two displacements:

$$\underbrace{\begin{bmatrix} \vdots \\ u_i \\ \vdots \\ u_j \\ \vdots \end{bmatrix}}_{\mathbf{u}_k} = \underbrace{\begin{bmatrix} & \vdots & \vdots \\ 0 \dots 0 & a_i & 0 & 0 \dots 0 \\ & \vdots & \vdots \\ 0 \dots 0 & 0 & a_j & 0 \dots 0 \\ & \vdots & \vdots \end{bmatrix}}_{\mathbf{A}_k} \underbrace{\begin{bmatrix} \vdots \\ u_{m(i)} \\ \vdots \\ u_{m(j)} \\ \vdots \end{bmatrix}}_{\mathbf{u}} + \underbrace{\begin{bmatrix} \vdots \\ b_i \\ \vdots \\ b_j \\ \vdots \end{bmatrix}}_{\mathbf{b}_k}$$

By transposing this matrix, we can write the dependence of the global internal forces on the local internal forces as:

$$\underbrace{\begin{bmatrix} \vdots \\ \bar{Q}_{m(i)} \\ \vdots \\ \bar{Q}_{m(j)} \\ \vdots \end{bmatrix}}_{\mathbf{Q}} = \underbrace{\begin{bmatrix} & 0 & 0 \\ \vdots & \vdots & \vdots \\ \dots & a_i & \dots & 0 & \dots \\ \dots & 0 & \dots & a_j & \dots \\ \vdots & \vdots & \vdots \\ 0 & 0 \end{bmatrix}}_{\mathbf{A}_k^T} \underbrace{\begin{bmatrix} \vdots \\ Q_i \\ \vdots \\ Q_j \\ \vdots \end{bmatrix}}_{\mathbf{Q}_k}$$

which can simply be written as the scalar relationship $\overline{Q}_{m(i)} = a_i Q_i$. Similarly, the dependence of the global mass matrix on the local mass matrix is

$$\underbrace{\begin{bmatrix} \vdots \\ \dots \overline{M}_{m(i),m(j)} \dots \\ \vdots \end{bmatrix}}_M = \underbrace{\begin{bmatrix} 0 & 0 \\ \vdots & \vdots \\ \dots a_i \dots 0 \dots \\ \dots 0 \dots a_j \dots \\ \vdots & \vdots \\ 0 & 0 \end{bmatrix}}_{A_k^T} \underbrace{\begin{bmatrix} \vdots \\ \dots M_{i,j} \dots \\ \vdots \end{bmatrix}}_{M_k} \underbrace{\begin{bmatrix} \vdots & \vdots \\ 0 \dots 0 & a_i & 0 & 0 \dots 0 \\ \vdots & \vdots \\ 0 \dots 0 & 0 & a_j & 0 \dots 0 \\ \vdots & \vdots \end{bmatrix}}_{A_k}$$

which can be written as the scalar relationship $\overline{M}_{m(i),m(j)} = a_i a_j M_{i,j}$. Analogous results are obtained for the transformation of $\mathbf{P}$, $\mathbf{D}$ and $\mathbf{K}$. This shows that it is actually not necessary to perform any matrix operations with $\mathbf{A}$ when converting between local and global quantities; in fact, we don't even need to construct them. Instead, the mapping between element coordinates and global coordinates is enough. This mapping and the scalar relationships found can be used to transform the element vectors/matrices one entry at a time and add their contributions into the corresponding global vectors/matrices in one go.

Chapter 4

Element Equations

4.1 Mass Element

The point mass is the simplest element type used in the bow model. Figure 4.1 shows its definition. It only consists of a single node, denoted by an index of zero, whose position is described by the Cartesian coordinates $x_0(t)$ and $y_0(t)$. A concentrated mass m is located at that node and subjected to the external forces $F_x(t)$ and $F_y(t)$ that are acting in the direction of the respective nodal coordinates.

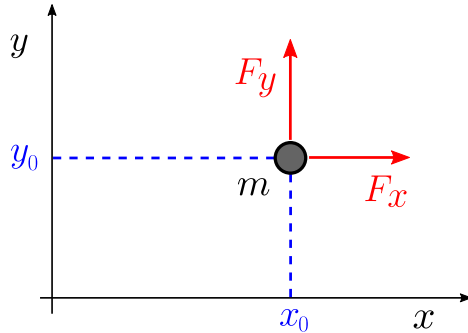


Figure 4.1: Point mass element

The equation of motion for this system can be written down directly by using Newton's second law of motion:

$$m \cdot \ddot{x}_0 = F_x \quad (4.1)$$

$$m \cdot \ddot{y}_0 = F_y \quad (4.2)$$

Rearranging this into a matrix equation, using the nodal coordinates $\mathbf{u}(t) = [x_0, y_0]^\top$, results in

$$\underbrace{\begin{bmatrix} m & 0 \\ 0 & m \end{bmatrix}}_{\mathbf{M}} \underbrace{\begin{bmatrix} \ddot{x}_0 \\ \ddot{y}_0 \end{bmatrix}}_{\ddot{\mathbf{u}}} + \underbrace{\begin{bmatrix} 0 \\ 0 \end{bmatrix}}_{\mathbf{Q}(\mathbf{u}, \dot{\mathbf{u}})} = \underbrace{\begin{bmatrix} F_x \\ F_y \end{bmatrix}}_{\mathbf{P}(t)}. \quad (4.3)$$

In this form we can immediately identify the element's mass matrix, internal forces (which are zero) and external forces. Because the internal forces are zero, the tangent stiffness and damping matrices are zero as well:

$$\mathbf{K} = \frac{\partial \mathbf{Q}}{\partial \mathbf{u}} = \begin{bmatrix} 0 & 0 \\ 0 & 0 \end{bmatrix}, \quad \mathbf{D} = \frac{\partial \mathbf{Q}}{\partial \dot{\mathbf{u}}} = \begin{bmatrix} 0 & 0 \\ 0 & 0 \end{bmatrix}. \quad (4.4)$$

So this element only contributes to the mass matrix of the overall system. Note also that the element's mass matrix is diagonal, because there is no coupling between the two directions of motion.

4.2 Beam Element

The beam elements are the core of the bow simulation, since they are used for modeling the static deflection and dynamic motion of the limbs. There are various approaches to constructing geometrically nonlinear beam elements that can handle arbitrarily large displacements and rotations. One such approach is the *corotational formulation*, which is also what VirtualBow uses. As a visual motivation for this method, consider the bow limb shown in Figure 4.2. In this figure, a short segment of the limb is tracked from the initial unbraced state of the bow to full draw. All shapes shown were exported from actual simulation results and are to scale.

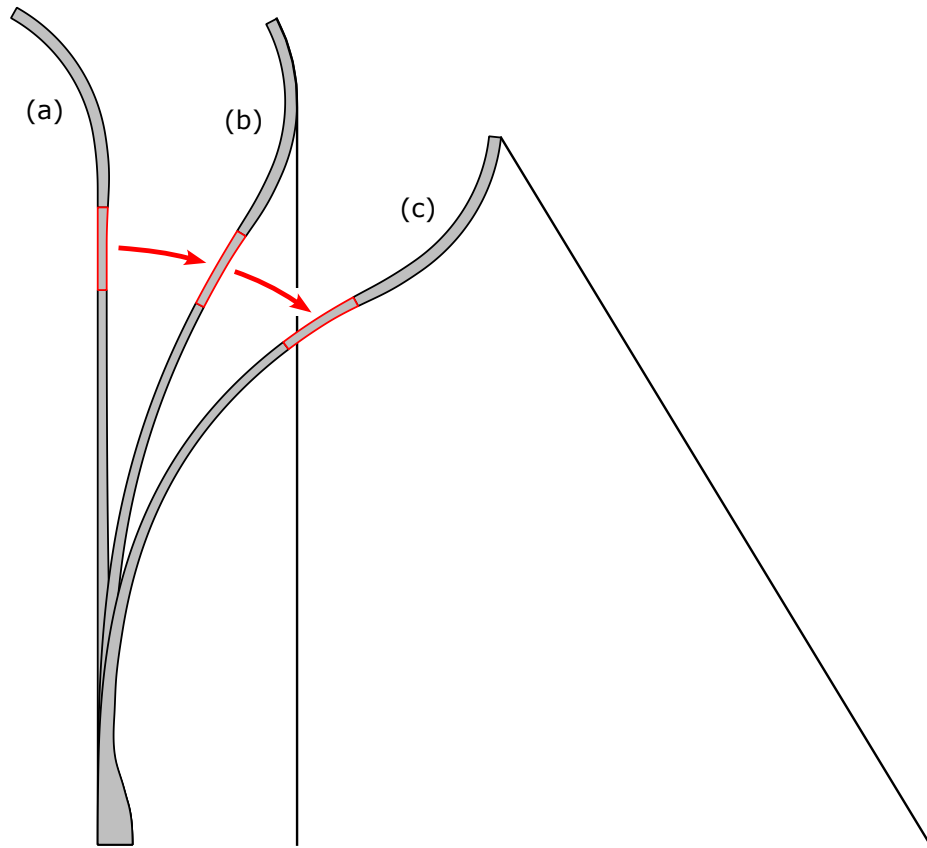


Figure 4.2: Motion of a single limb segment from unbraced state (a) over the braced state (b) to full draw (c)

The first observation is that the overall deflection of the limb as well as the motion of the tracked segment are indeed fairly large. So there really is no hope for geometrically linear methods that treat the overall deflection as small. Another observation, though, is that the tracked segment, while translating and rotating quite a bit, doesn't actually bend very much. To confirm this, Figure 4.3 shows the three segment shapes inside a common reference frame where the $\bar{x}$ -axis passes through the center of the first and the last cross-section.

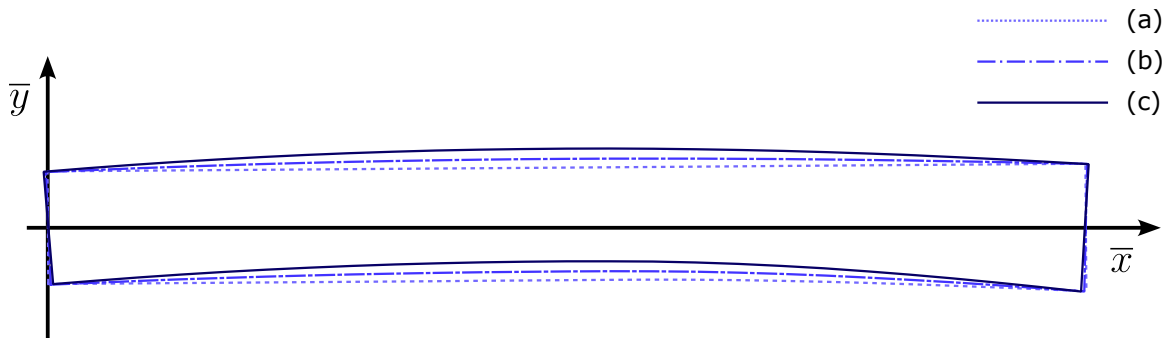


Figure 4.3: Limb segment in the states (a), (b) and (c) within a common reference frame

And indeed, when viewed within this reference frame, the segment bends only very slightly. It is easy to imagine that this elastic deformation would decrease even further if the length of the tracked segment were made shorter. The large overall deflection of the limb seems to be the result of the compounding effect of all the small, local deflections. The corotational formulation makes use of this basic principle. Within this formulation, each beam element is given a local frame of reference, similar to that shown in Figure 4.3, that translates and rotates along with the element as it moves. The actual motion of the element is then interpreted as the superposition of two parts:

1. A large, geometrically nonlinear, **rigid-body motion** of the element, as defined by the translation and rotation of its local reference frame.
2. A small, geometrically linear, **elastic deformation** of the element relative to its local reference frame.

The big advantage of this approach is that the local, elastic deformations can be treated as linear, which is both conceptually simpler and computationally more efficient. The resulting elements are valid for geometrically nonlinear problems where the overall deflection and rotation of a beam can be large as long as the local deformation within each element stays small. We have already seen that this is a reasonable assumption for bows, and that it can always be made true by making the elements shorter and using more of them.

4.2.1 Corotational Formulation

Let's now get a bit more precise and formulate this idea of a corotational reference frame in mathematical terms. First we are going to look at a beam segment within its local frame of reference according to Figure 4.4. The element is given two nodes, one at the left and one at the right. The reference frame $\{\bar{x}, \bar{y}\}$ is positioned with its origin at the first node and oriented so that the $\bar{x}$ -axis passes through the second node. This is not the only way that a corotational reference frame can be defined, but it seems to be the most common one [22, 29] and compares favorably with the alternatives [4].

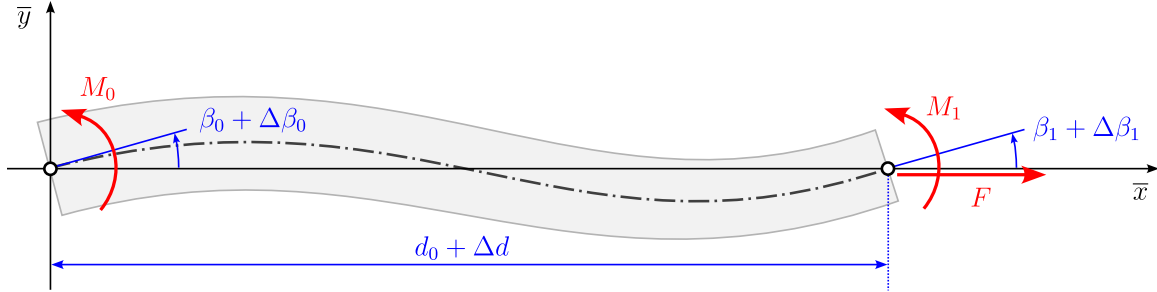


Figure 4.4: Element in its local reference frame

The element is not assumed to be initially straight, or to have a constant cross-section for that matter. Due to the possible initial curvature, its centerline forms an initial angle with the $\bar{x}$ -axis of β_0 at the first node and of β_1 at the second node. The initial distance between the two nodes is denoted by d_0 .

There are three degrees of freedom in which the nodes can move relative to the reference frame, complemented by three external forces:

1. Rotation of the first node, described by the angular change $\Delta\beta_0$ and corresponding to the external moment M_0 .
2. Rotation of the second node, described by the angular change $\Delta\beta_1$ and corresponding to the external moment M_1 .
3. Translation of the second node, described by the change in distance Δd and corresponding to the external force F .

We combine the local elastic displacements into the vector $\bar{\mathbf{u}} = [\Delta d, \Delta\beta_0, \Delta\beta_1]^\top$ and the corresponding external forces into the vector $\bar{\mathbf{P}} = [F, M_0, M_1]^\top$. According to the premise of the corotational formulation, the relationship between those forces and displacements within the local reference frame is linear. We express this relationship as

$$\bar{\mathbf{P}} = \bar{\mathbf{Q}}(\mathbf{u}_e, \dot{\mathbf{u}}_e) = \bar{\mathbf{K}} \cdot \bar{\mathbf{u}} + \bar{\mathbf{D}} \cdot \dot{\bar{\mathbf{u}}}, \quad (4.5)$$

with the local internal forces $\bar{\mathbf{Q}}$, defined by the stiffness matrix $\bar{\mathbf{K}} \in \mathbb{R}^{3 \times 3}$ and the damping matrix $\bar{\mathbf{D}} \in \mathbb{R}^{3 \times 3}$, both of which are constant. The derivation of those matrices from

the beam's geometry and material properties will be shown later; for now we just take them as given. Instead we will continue by developing the kinematic relations required for a corotational formulation. Consider now the element in a state as shown in Figure 4.5, where its local reference frame has translated and rotated within the global reference frame.

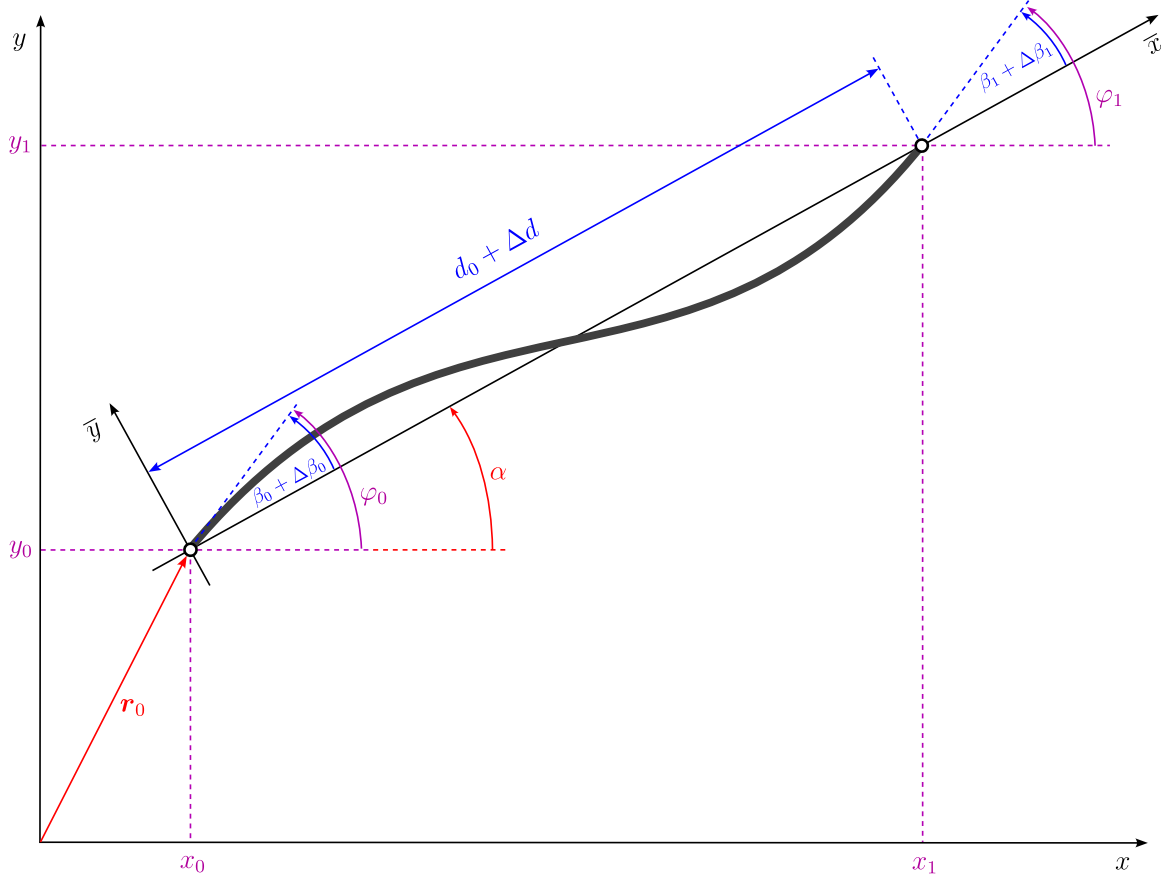


Figure 4.5: Element in the global reference frame

Three different kinds of coordinates are shown in this figure: The **elastic displacements** of the element relative to its local reference frame that were defined earlier. The **rigid-body motion** of the element's local reference frame, parameterized by the translation $\mathbf{r}_0$ of the reference frame's origin and its rotation angle α . And finally the **nodal positions and rotations** of the element in global coordinates. These global positions and rotations are the coordinates in which we actually want to formulate the equation of motion of the element:

$$\mathbf{u} = [x_0, \ y_0, \ \varphi_0 \ x_1, \ y_1, \ \varphi_1]^\top \quad (4.6)$$

These nodal coordinates fully characterize the motion of the element, including its rigid-body motion and its elastic deformation. This is consistent with counting the degrees of

freedom: three elastic displacement coordinates together with three rigid-body coordinates yield six degrees of freedom, matching the number of nodal coordinates. This means that, given the nodal coordinates, we can calculate the elastic and rigid-body coordinates.

The rigid-body translation $\mathbf{r}_0$ is given by the coordinates x_0 and y_0 of the first node. The rigid-body rotation α can be calculated via $\arctan 2$ as the angle of the $\bar{x}$ -axis passing through the two nodes. Let's first define some common terms, namely the differences Δx and Δy between the two nodes in x - and y -direction and their current distance d :

$$\begin{aligned}\Delta x &= x_1 - x_0, & d &= \sqrt{\Delta x^2 + \Delta y^2} \\ \Delta y &= y_1 - y_0,\end{aligned}\tag{4.7}$$

The rigid-body coordinates are then expressed as

$$\mathbf{r}_0 = \begin{bmatrix} x_0 \\ y_0 \end{bmatrix}, \quad \alpha = \arctan 2 \left(\frac{\Delta y}{\Delta x} \right).\tag{4.8}$$

The elastic elongation Δd is the difference between the current distance d of the nodes and the initial distance d_0 . The elastic angular displacements $\Delta\beta_0$ and $\Delta\beta_1$ follow from considering the sum of angles at each node. The relation between the elastic displacements $\bar{\mathbf{u}}$ and the overall coordinates $\mathbf{u}$ therefore turns out to be

$$\bar{\mathbf{u}}(\mathbf{u}) = \begin{bmatrix} \Delta d \\ \Delta\beta_0 \\ \Delta\beta_1 \end{bmatrix} = \begin{bmatrix} d - d_0 \\ \text{normalize}(\varphi_0 - \beta_0 - \alpha) \\ \text{normalize}(\varphi_1 - \beta_1 - \alpha) \end{bmatrix}.\tag{4.9}$$

One critical subtlety must be addressed here: the resulting angular displacements need to be *normalized*, i.e. wrapped to an interval of $]-\pi, \pi]$. That's because the nodal angles φ_0, φ_1 may be arbitrarily large, while the angle α is clamped to an interval of $]-\pi, \pi]$ due to the return value of the $\arctan 2$ function. So the difference of those angles might be off by a multiple of 2π , which is removed by the normalization. The velocities of the elastic displacements are obtained by differentiating with the chain rule,

$$\dot{\bar{\mathbf{u}}}(\mathbf{u}) = \frac{\partial \bar{\mathbf{u}}}{\partial \mathbf{u}} \frac{d\mathbf{u}}{dt} = \mathbf{J}(\mathbf{u}) \dot{\mathbf{u}},\tag{4.10}$$

with the Jacobian $\mathbf{J}(\mathbf{u}) \in \mathbb{R}^{3 \times 6}$ that depends on the nodal coordinates and can be worked out as

$$\mathbf{J}(\mathbf{u}) = \frac{\partial}{\partial \mathbf{u}} \begin{bmatrix} \Delta d \\ \Delta\beta_0 \\ \Delta\beta_1 \end{bmatrix} = \begin{bmatrix} -\frac{\Delta x}{d} & -\frac{\Delta y}{d} & 0 & \frac{\Delta x}{d} & \frac{\Delta y}{d} & 0 \\ -\frac{\Delta y}{d^2} & \frac{\Delta x}{d^2} & 1 & \frac{\Delta y}{d^2} & -\frac{\Delta x}{d^2} & 0 \\ -\frac{\Delta y}{d^2} & \frac{\Delta x}{d^2} & 0 & \frac{\Delta y}{d^2} & -\frac{\Delta x}{d^2} & 1 \end{bmatrix}.\tag{4.11}$$

This kinematic relationship between nodal coordinates and elastic displacements is the key to formulating the element's internal forces in terms of its nodal coordinates. Since the virtual work of the global internal forces $\mathbf{Q}$ must be equivalent to the virtual work of the local internal forces $\bar{\mathbf{Q}}$, we get the relation

$$\begin{aligned}\delta \mathbf{u}^\top \mathbf{Q} &= \delta \bar{\mathbf{u}}^\top \bar{\mathbf{Q}} = (\mathbf{J} \delta \mathbf{u})^\top \bar{\mathbf{Q}} = \delta \mathbf{u}^\top \mathbf{J}^\top \bar{\mathbf{Q}} \\ \Rightarrow \mathbf{Q} &= \mathbf{J}^\top \bar{\mathbf{Q}}\end{aligned}\tag{4.12}$$

Stated in words: the transpose of the same Jacobian $\mathbf{J}$ that converts between local and global velocities also transforms between local and global forces. By combining (4.12) with the linear forces (4.5) and rewriting $\mathbf{u}_e$ and $\dot{\mathbf{u}}_e$ in terms of global coordinates, we obtain the element's internal forces as

$$\begin{aligned}\mathbf{Q}(\mathbf{u}, \dot{\mathbf{u}}) &= \mathbf{J}^\top \bar{\mathbf{Q}} \\ &= \mathbf{J}^\top (\bar{\mathbf{K}} \bar{\mathbf{u}} + \bar{\mathbf{D}} \dot{\bar{\mathbf{u}}}) \\ &= \mathbf{J}^\top \bar{\mathbf{K}} \bar{\mathbf{u}}(\mathbf{u}) + \mathbf{J}^\top \bar{\mathbf{D}} \dot{\mathbf{u}}.\end{aligned}\tag{4.13}$$

Note how the corotational formulation yields relatively simple, fully analytical expressions for the internal forces. This stands in contrast to formulations in which the internal forces have to be evaluated numerically, often through *Gaussian quadrature* or related integration schemes. The tangent stiffness and damping matrices are obtained by differentiation of the internal forces as

$$\mathbf{K} = \frac{\partial \mathbf{Q}}{\partial \mathbf{u}}, \quad \mathbf{D} = \frac{\partial \mathbf{Q}}{\partial \dot{\mathbf{u}}}.\tag{4.14}$$

Carrying out these derivatives is somewhat laborious but fairly straightforward, and it is therefore presented in full detail in Appendix B.1.

Now only one final component of the element's equation of motion is still missing: the inertia forces, i.e. the forces associated with the element's moving mass. As already noted, the corotational formulation yields comparatively simple expressions for the internal forces. The other side of that tradeoff, however, is that the inertia forces, when derived in a consistent way and without simplifications, become highly nonlinear and much more complex; see for example [22]. In particular, they cannot be captured by a constant mass matrix, as has been the premise up to this point. So instead of using a consistent mass matrix, we are going to construct a *lumped mass matrix*, which is a common approximation where an element's mass and inertia are distributed, i.e. *lumped*, to its nodes. Suppose that, after this lumping process, each node carries a mass m_i and a moment of inertia J_i , while the rest of the element is massless. This leads to the following lumped mass matrix for our element:

$$\mathbf{M} = \begin{bmatrix} m_0 & & & \cdots & 0 \\ & m_0 & & \ddots & \vdots \\ & & J_0 & & \\ & & & m_1 & \\ \vdots & \ddots & & & m_1 \\ 0 & \cdots & & & J_1 \end{bmatrix}, \quad (4.15)$$

Like all lumped mass matrices, it is not only constant but also diagonal, which provides a very nice computational advantage. If all element types used in a system have a diagonal mass matrix, then the overall system's mass matrix will be diagonal as well. This greatly simplifies the computation of accelerations in dynamic analysis, for which the mass matrix has to be inverted. Unlike more general matrices, a diagonal matrix can be trivially inverted just by independently inverting its diagonal entries. Of course, a lumped mass matrix remains an approximation and comes with certain drawbacks. Its diagonal structure implies that any coupling between inertia forces in different directions is neglected. Moreover, as we will see later, distributing a beam segment's mass and especially its rotational inertia to its nodes is not as straightforward as it may appear. Several lumping strategies exist, each with its own limitations. For VirtualBow, the tradeoffs that come with a lumped mass matrix make sense though. While the dynamic bow simulation needs to be sufficiently accurate, its primary focus is the overall motion of the bow. Capturing every higher harmonic of the limb's oscillation in detail is not essential for this purpose. So a consistent, and therefore more accurate, mass matrix would likely offer little to no practical benefit while making the simulation more complicated than it needs to be.

With this we conclude the derivation of the corotational formulation for the beam element. The next sections will fill in the missing details that have been handwaved here. Specifically, the computation of the linear stiffness and damping matrices that were taken as given in (4.5) as well as the determination of the masses and inertias appearing in (4.15).

4.2.2 Castigliano's Second Theorem

As a preparation for deriving the linear stiffness matrix of the beam element, we first revisit a very useful result from linear elasticity: *Castigliano's second theorem*. This theorem provides a general method for computing the displacements of arbitrary linear-elastic structures under external loads. Wikipedia [26] states it as follows:

If the strain energy of a linearly elastic structure can be expressed as a function of generalized force P_i then the partial derivative of the strain energy with respect to generalized force gives the generalized displacement u_i in the direction of P_i . (Notation adapted)

To illustrate this relationship between forces, displacements and potential energy, let's consider a bar under tension as shown in Figure 4.6. It has an initial length of l and a variable longitudinal stiffness $EA(s)$ that depends on the position $s \in [0, l]$ along the bar. As a first exercise, let's compute the displacement u at the end of the bar that results from the force F being applied there.

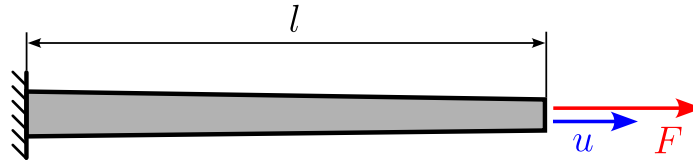


Figure 4.6: Bar with a tensile force applied at the free end

First of all, the strain energy of a bar is given by

$$\Pi = \frac{1}{2} \int_0^l \frac{N^2}{EA} ds \quad (4.16)$$

where $N(s)$ is the normal force at each cross-section along the length of the bar. In this case it is easy to see that we have a constant normal force of $N(s) = F$. According to Castigliano's theorem, the displacement u in the direction of the force is

$$u = \frac{\partial \Pi}{\partial F} = \frac{\partial}{\partial F} \left(\frac{1}{2} \int_0^l \frac{F^2}{EA} ds \right) = \underbrace{\left(\int_0^l \frac{1}{EA} ds \right)}_{K^{-1}} F. \quad (4.17)$$

With this we have identified the inverse K^{-1} of the bar's stiffness in this loading case. It is an integral over the initial length of the bar that takes the exact distribution of longitudinal stiffness into account. If we assume EA to be constant for a quick sanity check, we get a stiffness of $K = EA/l$, which is the expected result for the stiffness of a uniform bar. It is especially worth mentioning that we arrived at this result without any kinematic considerations, i.e. we didn't have to think about the distribution of strain in the bar due to the normal force and how that relates to the displacement u , which makes Castigliano's method very simple and elegant. One apparent shortcoming is that this only

gives us the displacement at the end of the bar, where the external force is applied, and nowhere else. This limitation can be worked around by introducing additional “imaginary” forces at other points of interest and later setting them to zero after having worked out the associated displacements. So as a second exercise, let’s calculate the displacement u^* at an arbitrary point s^* within the bar as shown in Figure 4.7 by introducing the imaginary auxiliary force $F^* = 0$.

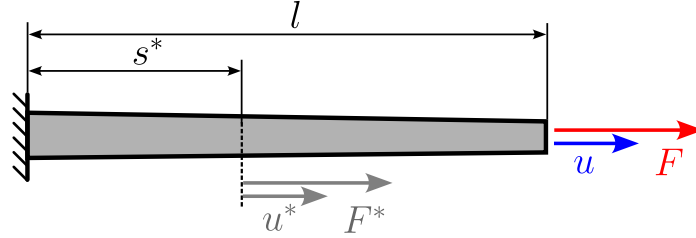


Figure 4.7: Bar with a tensile force and an auxiliary force

The normal force over the length of the bar is now given by the piecewise function

$$N(s) = \begin{cases} F + F^*, & \text{if } 0 \leq s < s^* \\ F, & \text{if } s^* \leq s \leq l \end{cases} \quad (4.18)$$

Therefore the potential elastic energy becomes

$$\Pi = \frac{1}{2} \int_0^l \frac{N^2}{EA} ds = \frac{1}{2} \left(\int_0^{s^*} \frac{(F + F^*)^2}{EA} ds + \int_{s^*}^l \frac{F^2}{EA} ds \right) \quad (4.19)$$

Application of Castigliano’s theorem gives the displacement u^* depending on the forces F and F^* as

$$u^* = \frac{\partial \Pi}{\partial F^*} = \frac{\partial}{\partial F^*} \left(\frac{1}{2} \int_0^{s^*} \frac{(F + F^*)^2}{EA} ds \right) = \left(\int_0^{s^*} \frac{1}{EA} ds \right) (F + F^*) \quad (4.20)$$

The actual displacement is then obtained by setting the auxiliary force F^* to zero, resulting in the displacement distribution within the bar,

$$u^*(s^*) = \left(\int_0^{s^*} \frac{1}{EA} ds \right) F. \quad (4.21)$$

As a quick check, setting $s^* = l$ gives the same result for the end displacement as before. Since Castigliano’s method works not only for bars, but for any linear-elastic structure, it will be an important tool for deriving the stiffness matrices required for the corotational beam element.

4.2.3 Stiffness Matrix of a Beam Segment

We can now use Castigliano's theorem to calculate the stiffness matrix and the deformation of a general, linear-elastic beam segment. The segment can be initially curved and have varying cross-section properties along its length. The only assumption we will make is that all displacements are small, so that we can apply linear-elastic theory to the problem. Our approach will be similar to the one presented in [24].

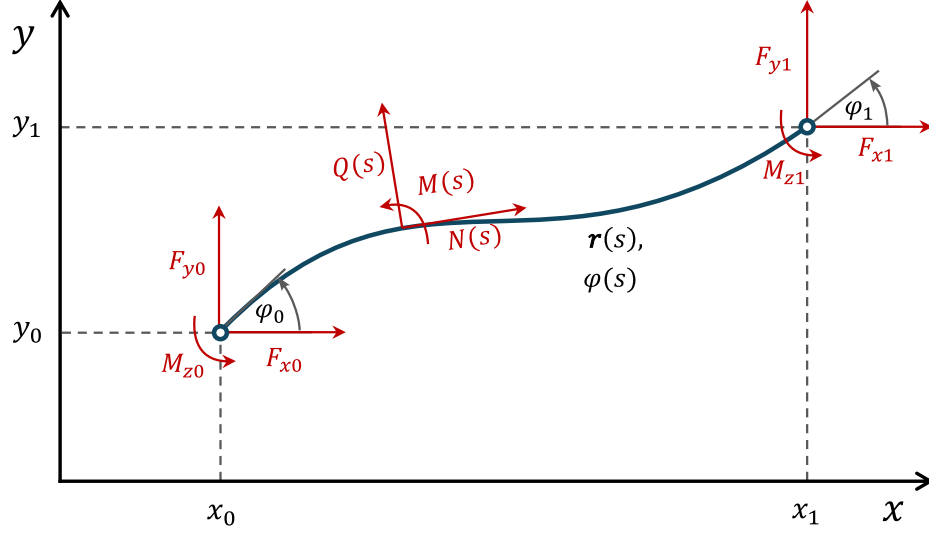


Figure 4.8: Arbitrary beam segment with two nodes

Consider a beam segment as shown in Figure 4.8. It has two nodes and six nodal degrees of freedom, for which we are going to derive the complete 6×6 stiffness matrix. The reduction to the 3×3 matrix required by our corotational formulation will be done later. Its undeformed shape is described by the reference curve $\mathbf{r}(s) = [x(s), y(s)]^T$ and the associated orientation angle $\varphi(s)$, where the parameter $s \in [s_0, s_1]$ is the arc length along the curve. We combine the reference position and angle into a single function $\mathbf{p}(s)$ and call the associated small displacements $\Delta \mathbf{p}(s)$,

$$\mathbf{p}(s) = \begin{bmatrix} x(s) \\ y(s) \\ \varphi(s) \end{bmatrix}, \quad \Delta \mathbf{p}(s) = \begin{bmatrix} \Delta x(s) \\ \Delta y(s) \\ \Delta \varphi(s) \end{bmatrix}. \quad (4.22)$$

The stiffness properties of the cross-sections are defined by their stiffness matrix $\mathbf{C}(s) \in \mathbb{R}^{3 \times 3}$, which is assumed to be known and establishes a linear constitutive relation of the form

$$\underbrace{\begin{bmatrix} N \\ Q \\ M \end{bmatrix}}_{\mathbf{f}(s)} = \underbrace{\begin{bmatrix} C_{\varepsilon\varepsilon} & C_{\varepsilon\gamma} & C_{\varepsilon\kappa} \\ C_{\varepsilon\gamma} & C_{\gamma\gamma} & C_{\gamma\kappa} \\ C_{\varepsilon\kappa} & C_{\gamma\kappa} & C_{\kappa\kappa} \end{bmatrix}}_{\mathbf{C}(s)} \underbrace{\begin{bmatrix} \varepsilon \\ \gamma \\ \kappa \end{bmatrix}}_{\mathbf{e}(s)} \quad (4.23)$$

between the cross-section forces $\mathbf{f}$ and the generalized strains $\mathbf{e}$ at each length s . The cross-section forces consist of the normal force N , the shear force Q and the bending moment M as shown in Figure 4.8 while the strains consist of the elongation ε of the beam axis, the shear angle γ between the cross-section and the beam axis and the curvature κ of the beam axis. You may wonder why we introduced shear deformation here, even though the bow model was said to be based on Euler-Bernoulli theory earlier. This is simply to keep the formulation more general and to prepare for future improvements. Right now, VirtualBow does very little with this additional deformation.

The potential elastic energy (strain energy) of the beam segment, which we are going to need later, can be evaluated by integrating the product $\mathbf{f}^\top \mathbf{e}$ of section forces and strains over the beam's length and using the stiffness relation (4.23) to eliminate the unknown strains:

$$\Pi = \frac{1}{2} \int_{s_0}^{s_1} \mathbf{f}^\top \mathbf{e} ds = \frac{1}{2} \int_{s_0}^{s_1} \mathbf{f}^\top \mathbf{C}^{-1} \mathbf{f} ds. \quad (4.24)$$

We assign two nodes to this segment, one at the starting point $s = s_0$ and one on the endpoint $s = s_1$ of the curve. Each node is given two translational degrees of freedom in x - and y -direction and an angular degree of freedom around the z -axis. At each node, the respective forces and moments

$$\mathbf{F}_0 = \begin{bmatrix} F_{x,0} \\ F_{y,0} \\ M_{z,0} \end{bmatrix}, \quad \mathbf{F}_1 = \begin{bmatrix} F_{x,1} \\ F_{y,1} \\ M_{z,1} \end{bmatrix} \quad (4.25)$$

are applied, to which the beam segment reacts with the (small) nodal displacements

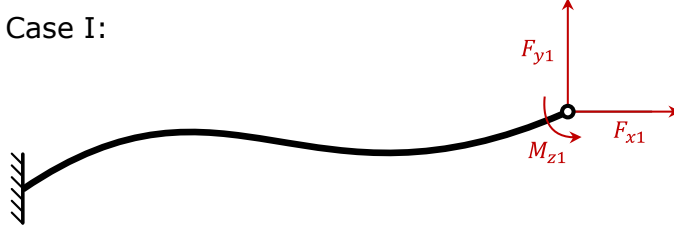
$$\Delta \mathbf{p}_0 = \begin{bmatrix} \Delta x_0 \\ \Delta y_0 \\ \Delta \varphi_0 \end{bmatrix}, \quad \Delta \mathbf{p}_1 = \begin{bmatrix} \Delta x_1 \\ \Delta y_1 \\ \Delta \varphi_1 \end{bmatrix} \quad (4.26)$$

in the respective directions of the forces/moments and relative to the initial state. The goal is now to find the stiffness matrix of the segment, which describes the linear relationship between the combined nodal forces and displacements. This can be formally written as

$$\underbrace{\begin{bmatrix} \mathbf{F}_0 \\ \mathbf{F}_1 \end{bmatrix}}_{\mathbf{F}} = \underbrace{\begin{bmatrix} \mathbf{K}_{0,0} & \mathbf{K}_{0,1} \\ \mathbf{K}_{1,0} & \mathbf{K}_{1,1} \end{bmatrix}}_{\mathbf{K}} \underbrace{\begin{bmatrix} \Delta \mathbf{p}_0 \\ \Delta \mathbf{p}_1 \end{bmatrix}}_{\mathbf{u}} \quad (4.27)$$

with the overall stiffness matrix $\mathbf{K} \in \mathbb{R}^{6 \times 6}$ and four submatrices $\mathbf{K}_{ij} \in \mathbb{R}^{3 \times 3}$. The overall forces $\mathbf{F}$ and displacements $\mathbf{u}$ have been chosen as the combination of the forces and displacements at nodes 0 and 1. We can get a physical interpretation of the submatrices $\mathbf{K}_{ij}$ by fixing one node at a time as shown in Figure 4.9.

Case I:



Case II:



Figure 4.9: Beam segment with constrained nodes

In **Case I**, where the first node is fixed ($\Delta \mathbf{p}_0 = \mathbf{0}$), the resulting forces are

$$\mathbf{F}_1 = \mathbf{K}_{1,1} \Delta \mathbf{p}_1 \quad (4.28)$$

$$\mathbf{F}_0 = \mathbf{K}_{0,1} \Delta \mathbf{p}_1 \quad (4.29)$$

In **Case II**, where the second node is fixed ($\Delta \mathbf{p}_1 = \mathbf{0}$), the resulting forces are

$$\mathbf{F}_0 = \mathbf{K}_{0,0} \Delta \mathbf{p}_0 \quad (4.30)$$

$$\mathbf{F}_1 = \mathbf{K}_{1,0} \Delta \mathbf{p}_0 \quad (4.31)$$

Put into words, the diagonal matrix blocks $\mathbf{K}_{0,0}$ and $\mathbf{K}_{1,1}$ relate the forces and displacements at the respective free node to each other if the other node is restrained. The off-diagonal blocks $\mathbf{K}_{1,0}$ and $\mathbf{K}_{0,1}$ relate the displacements at a free node to the forces at a restrained node. This means that we can obtain the full stiffness matrix of the segment by analyzing those two simpler load cases and assembling the partial results in the end.

As a first step we note that the nodal forces $\mathbf{F}_0$ and $\mathbf{F}_1$ are not completely independent of each other, since they have to be in static equilibrium as a necessary condition for the element to be in equilibrium as a whole. Balancing the forces and moments around node 1 gives us the three conditions

$$x : F_{x,0} + F_{x,1} = 0 \quad (4.32)$$

$$y : F_{y,0} + F_{y,1} = 0 \quad (4.33)$$

$$z : M_{z,0} + M_{z,1} + F_{x,0}(y_1 - y_0) - F_{y,0}(x_1 - x_0) = 0 \quad (4.34)$$

Solving for either $\mathbf{F}_0$ or $\mathbf{F}_1$ results in the following linear relationship between the nodal forces,

$$\begin{aligned} \mathbf{F}_0 &= \mathbf{B}_{0,1} \cdot \mathbf{F}_1, \\ \mathbf{F}_1 &= \mathbf{B}_{1,0} \cdot \mathbf{F}_0, \end{aligned} \quad \mathbf{B}_{i,j} = \begin{bmatrix} -1 & 0 & 0 \\ 0 & -1 & 0 \\ y_j - y_i & x_i - x_j & -1 \end{bmatrix}. \quad (4.35)$$

with the static equilibrium matrix $\mathbf{B}_{i,j} = \mathbf{B}_{j,i}^{-1}$. We can use this relationship to calculate the constraint forces at the clamped nodes in cases I and II in order to obtain the off-diagonal stiffnesses $\mathbf{K}_{1,0}$ and $\mathbf{K}_{0,1}$ as follows,

$$\mathbf{F}_0 = \mathbf{B}_{0,1} \mathbf{F}_1 = \underbrace{\mathbf{B}_{0,1} \mathbf{K}_{1,1}}_{\mathbf{K}_{0,1}} \Delta \mathbf{p}_1, \quad (4.36)$$

$$\mathbf{F}_1 = \mathbf{B}_{1,0} \mathbf{F}_0 = \underbrace{\mathbf{B}_{1,0} \mathbf{K}_{0,0}}_{\mathbf{K}_{1,0}} \Delta \mathbf{p}_0. \quad (4.37)$$

This means that we only have to determine the diagonal blocks $\mathbf{K}_{0,0}$ and $\mathbf{K}_{1,1}$ from load cases I and II and then use the results above in order to complete the stiffness matrix.

Stiffness and Displacements for Case I

In the first case, the beam is clamped at node 0 and a force is applied at node 1. Due to static equilibrium, the distribution of cross-section forces in the beam for an external force $\mathbf{F}_\eta$ that is applied at the position s_η with $\eta \in [0, 1]$ is found to be

$$\underbrace{\begin{bmatrix} N \\ Q \\ M \end{bmatrix}}_{\mathbf{f}_\eta^I(s)} = \underbrace{\begin{bmatrix} \cos(\varphi(s)) & \sin(\varphi(s)) & 0 \\ -\sin(\varphi(s)) & \cos(\varphi(s)) & 0 \\ y(s) - y(s_\eta) & x(s_\eta) - x(s) & 1 \end{bmatrix}}_{\mathbf{H}_\eta(s)} \underbrace{\begin{bmatrix} F_{x,\eta} \\ F_{y,\eta} \\ M_{z,\eta} \end{bmatrix}}_{\mathbf{F}_\eta} \quad (4.38)$$

Combining the nodal force $\mathbf{F}_1$ at s_1 and the auxiliary force $\mathbf{F}_\eta$ at s_η (used for Castigliano's theorem and later set to zero), we get the total cross-section force

$$\mathbf{f}^I(s) = \begin{cases} \mathbf{H}_1(s) \mathbf{F}_1 + \mathbf{H}_\eta(s) \mathbf{F}_\eta, & \text{if } s_0 \leq s < s_\eta \\ \mathbf{H}_1(s) \mathbf{F}_1, & \text{if } s_\eta \leq s \leq s_1 \end{cases} \quad (4.39)$$

The strain energy for this load case according to (4.24) is therefore

$$\Pi^I = \frac{1}{2} \left(\int_{s_0}^{s_\eta} (\mathbf{H}_1 \mathbf{F}_1 + \mathbf{H}_\eta \mathbf{F}_\eta)^\top \mathbf{C}^{-1} (\mathbf{H}_1 \mathbf{F}_1 + \mathbf{H}_\eta \mathbf{F}_\eta) ds + \int_{s_\eta}^{s_1} (\mathbf{H}_1 \mathbf{F}_1)^\top \mathbf{C}^{-1} (\mathbf{H}_1 \mathbf{F}_1) ds \right) \quad (4.40)$$

and the displacements $\Delta \mathbf{p}_\eta$ at position s_η are obtained by Castigliano's theorem as

$$\begin{aligned}\Delta \mathbf{p}_\eta^I &= \left. \frac{\partial \Pi^I}{\partial \mathbf{F}_\eta} \right|_{\mathbf{F}_\eta=0} = \frac{\partial}{\partial \mathbf{F}_\eta} \left(\frac{1}{2} \int_{s_0}^{s_\eta} (\mathbf{H}_1 \mathbf{F}_1 + \mathbf{H}_\eta \mathbf{F}_\eta)^\top \mathbf{C}^{-1} (\mathbf{H}_1 \mathbf{F}_1 + \mathbf{H}_\eta \mathbf{F}_\eta) ds \right) \Big|_{\mathbf{F}_\eta=0} \\ &= \underbrace{\left(\int_{s_0}^{s_\eta} \mathbf{H}_\eta^\top \mathbf{C}^{-1} \mathbf{H}_1 ds \right)}_{\mathbf{K}_{1,\eta}^{-1}} \mathbf{F}_1.\end{aligned}\quad (4.41)$$

The inverse stiffness matrix $\mathbf{K}_{1,1}^{-1}$ for the right node follows for $\eta = 1$.

Stiffness and Displacements for Case II

In the second case, where the beam is clamped at node 1 and the force is applied at node 0 instead, the section forces turn out to be

$$\underbrace{\begin{bmatrix} N \\ Q \\ M \end{bmatrix}}_{\mathbf{f}_\eta^{\text{II}}(s)} = \underbrace{\begin{bmatrix} -\cos(\varphi(s)) & -\sin(\varphi(s)) & 0 \\ \sin(\varphi(s)) & -\cos(\varphi(s)) & 0 \\ y(s_\eta) - y(s) & x(s) - x(s_\eta) & -1 \end{bmatrix}}_{-\mathbf{H}_\eta(s)} \underbrace{\begin{bmatrix} F_{x,\eta} \\ F_{y,\eta} \\ M_{z,\eta} \end{bmatrix}}_{\mathbf{F}_\eta}, \quad (4.42)$$

$$\mathbf{f}^{\text{II}}(s) = \begin{cases} -\mathbf{H}_0(s) \mathbf{F}_0, & \text{if } s_0 \leq s \leq s_\eta \\ -\mathbf{H}_0(s) \mathbf{F}_0 - \mathbf{H}_\eta(s) \mathbf{F}_\eta, & \text{if } s_\eta \leq s < s_1 \end{cases}. \quad (4.43)$$

The strain energy follows as

$$\Pi^{\text{II}} = \frac{1}{2} \left(\int_{s_0}^{s_\eta} (\mathbf{H}_0 \mathbf{F}_0)^\top \mathbf{C}^{-1} (\mathbf{H}_0 \mathbf{F}_0) ds + \int_{s_\eta}^{s_1} (\mathbf{H}_0 \mathbf{F}_0 + \mathbf{H}_\eta \mathbf{F}_\eta)^\top \mathbf{C}^{-1} (\mathbf{H}_0 \mathbf{F}_0 + \mathbf{H}_\eta \mathbf{F}_\eta) ds \right) \quad (4.44)$$

and finally, the displacements according to Castigliano's theorem are

$$\begin{aligned}\Delta \mathbf{p}_\eta^{\text{II}} &= \left. \frac{\partial \Pi^{\text{II}}}{\partial \mathbf{F}_\eta} \right|_{\mathbf{F}_\eta=0} = \frac{\partial}{\partial \mathbf{F}_\eta} \left(\frac{1}{2} \int_{s_\eta}^{s_1} (\mathbf{H}_0 \mathbf{F}_0 + \mathbf{H}_\eta \mathbf{F}_\eta)^\top \mathbf{C}^{-1} (\mathbf{H}_0 \mathbf{F}_0 + \mathbf{H}_\eta \mathbf{F}_\eta) ds \right) \Big|_{\mathbf{F}_\eta=0} \\ &= \underbrace{\left(\int_{s_\eta}^{s_1} \mathbf{H}_\eta^\top \mathbf{C}^{-1} \mathbf{H}_0 ds \right)}_{\mathbf{K}_{0,\eta}^{-1}} \mathbf{F}_0\end{aligned}\quad (4.45)$$

from which the stiffness matrix $\mathbf{K}_{0,0}^{-1}$ for the left node follows for $\eta = 0$.

Combined Stiffness and Displacements

Combining all previous results, the four components of the full stiffness matrix are computed as

$$\mathbf{K}_{0,0}^{-1} = \int_{s_0}^{s_1} \mathbf{H}_0^\top \mathbf{C}^{-1} \mathbf{H}_0 ds \quad \mathbf{K}_{1,1}^{-1} = \int_{s_0}^{s_1} \mathbf{H}_1^\top \mathbf{C}^{-1} \mathbf{H}_1 ds \quad (4.46)$$

$$\mathbf{K}_{1,0} = \mathbf{B}_{1,0} \mathbf{K}_{0,0} \quad \mathbf{K}_{0,1} = \mathbf{B}_{0,1} \mathbf{K}_{1,1} \quad (4.47)$$

Alternatively, since the stiffness matrix must be symmetric, the relation $\mathbf{K}_{0,1} = \mathbf{K}_{1,0}^\top$ can be used for calculating one of the off-diagonal blocks in terms of the other. Verifying that this is true is left as an exercise for the reader ;) The full displacements of the beam segment under forces at both nodes are obtained by adding together the displacements from both load cases due to linear superposition,

$$\Delta \mathbf{p}_\eta = \mathbf{K}_{0,\eta}^{-1} \mathbf{F}_0 + \mathbf{K}_{1,\eta}^{-1} \mathbf{F}_1. \quad (4.48)$$

Expressed in terms of the nodal displacements $\mathbf{u}$, this turns into

$$\begin{aligned} \Delta \mathbf{p}_\eta &= \mathbf{K}_{0,\eta}^{-1} \mathbf{K}_{0,0} \mathbf{u}_0 + \mathbf{K}_{1,\eta}^{-1} \mathbf{K}_{1,1} \mathbf{u}_1 \\ &= \underbrace{\left[\mathbf{K}_{0,\eta}^{-1} \mathbf{K}_{0,0} \mid \mathbf{K}_{1,\eta}^{-1} \mathbf{K}_{1,1} \right]}_{\mathbf{E}_{u,\eta}} \underbrace{\begin{bmatrix} \mathbf{u}_0 \\ \mathbf{u}_1 \end{bmatrix}}_{\mathbf{u}} = \mathbf{E}_{u,\eta} \cdot \mathbf{u} \end{aligned} \quad (4.49)$$

with the displacement evaluation matrix $\mathbf{E}_{u,\eta}$ at the point η . We can simplify the computation of the matrices $\mathbf{K}_{0,\eta}^{-1}$ and $\mathbf{K}_{1,\eta}^{-1}$ by realizing that $\mathbf{H}_0$ and $\mathbf{H}_1$ are related by the static equilibrium matrix $\mathbf{B}_{i,j}$ as

$$\mathbf{H}_\eta(s) = -\mathbf{H}_0(s) \mathbf{B}_{\eta,0} \quad (4.50)$$

Using this, the flexibility matrices can be written by using the same expression inside the integral, but integrated over different bounds:

$$\mathbf{K}_{0,\eta}^{-1} = -\mathbf{B}_{0,\eta}^\top \left(\int_{s_\eta}^{s_1} \mathbf{H}_0^\top \mathbf{C}^{-1} \mathbf{H}_0 ds \right) \quad (4.51)$$

$$\mathbf{K}_{1,\eta}^{-1} = \mathbf{B}_{0,\eta}^\top \left(\int_{s_0}^{s_\eta} \mathbf{H}_0^\top \mathbf{C}^{-1} \mathbf{H}_0 ds \right) \mathbf{B}_{0,1}. \quad (4.52)$$

Note that up to this point all results have been exact, in the sense that no further simplifications within the linear-elastic beam theory have been made. The integrals over

the segment's length account for arbitrary variations of shape and cross-section properties. In the actual implementation, though, the integration itself is done numerically, because accounting for all kinds of shapes and cross-sections analytically would be too complicated. The integration interval $[s_0, s_1]$ is divided into n subintervals, and the integral for each subinterval is approximated using *Simpson's rule*. To obtain the required integrals from above, these partial results are summed up as needed. Since the stiffness matrix and the evaluation matrices are constant, they have to be computed only once prior to the simulation. Therefore the integration is not performance-critical and can be done fairly accurately with small numerical tolerances.

Corotational Reference Frame

We now have a general 6×6 stiffness matrix of an arbitrary beam segment, but according to Figure 4.5 we need the stiffness matrix relative to a coordinate system that is rotated by the initial angle of $\alpha(t = 0)$ and reduced to the three local coordinates $\bar{\mathbf{u}} = [\Delta d, \Delta\beta_0, \Delta\beta_1]^T$. The relation between the segment's full displacements $\mathbf{u}$ and the local coordinates is found to be

$$\mathbf{u} = \begin{bmatrix} \Delta x_0 \\ \Delta y_0 \\ \Delta\varphi_0 \\ \Delta x_1 \\ \Delta y_1 \\ \Delta\varphi_1 \end{bmatrix} = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 1 & 0 \\ \cos(\alpha) & 0 & 0 \\ \sin(\alpha) & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} \Delta d \\ \Delta\beta_0 \\ \Delta\beta_1 \end{bmatrix} = \mathbf{T}\bar{\mathbf{u}} \quad (4.53)$$

with the transformation matrix $\mathbf{T}$. The transformed stiffness matrix is then $\bar{\mathbf{K}} = \mathbf{T}^T \mathbf{K} \mathbf{T}$. Similarly, the evaluation matrices in the corotated frame are $\bar{\mathbf{E}} = \mathbf{E} \mathbf{T}$.

But we don't only want to transform things into the corotational frame; we also want to transform some results out of it and into global coordinates, namely the elastic displacements of the beam segment and their velocities. As shown in Figure 4.10, the position and orientation of a point on the beam in local coordinates is the sum of its initial position $\bar{\mathbf{p}}$ and the associated displacements $\Delta\bar{\mathbf{p}}$ at that point according to (4.49).

The position and orientation $\mathbf{p}$ in global coordinates is then obtained by accounting for the rotation α and the translation (x_0, y_0) of the local reference frame:

$$\begin{aligned} \mathbf{p}(s, t) &= \underbrace{\begin{bmatrix} \cos(\alpha(t)) & -\sin(\alpha(t)) & 0 \\ \sin(\alpha(t)) & \cos(\alpha(t)) & 0 \\ 0 & 0 & 1 \end{bmatrix}}_{\mathbf{R}(\alpha(t))} \left(\bar{\mathbf{p}}(s) + \Delta\bar{\mathbf{p}}(s) \right) + \underbrace{\begin{bmatrix} x_0(t) \\ y_0(t) \\ \alpha(t) \end{bmatrix}}_{\mathbf{b}(t)} \\ &= \mathbf{R}(\bar{\mathbf{p}} + \mathbf{E}_u \bar{\mathbf{u}}) + \mathbf{b} \end{aligned} \quad (4.54)$$

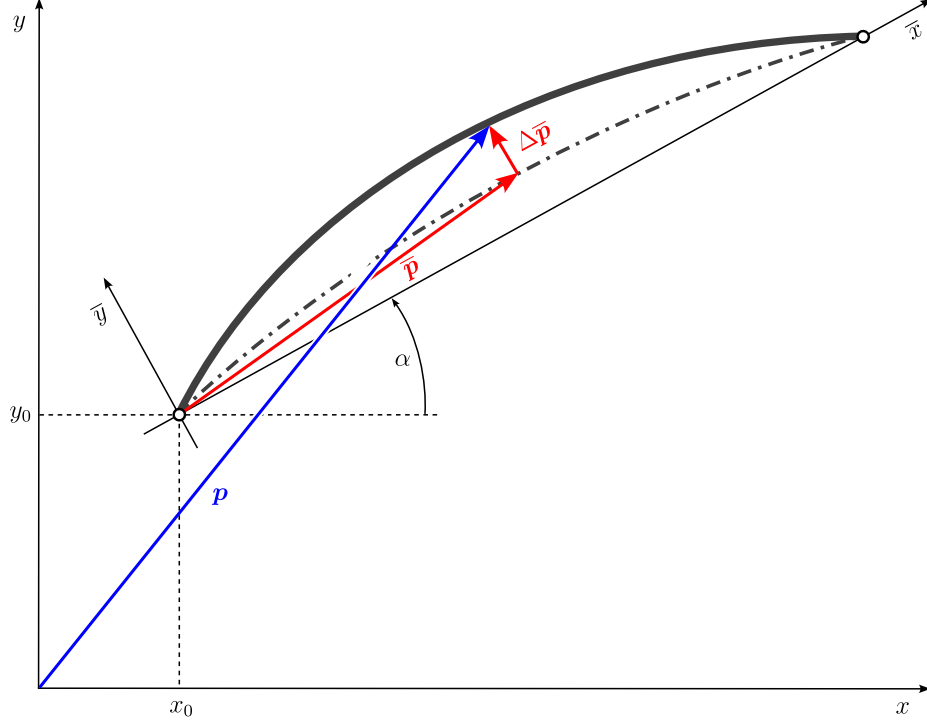


Figure 4.10: Position of the beam axis in local and global coordinates

By differentiation, the global velocity $\dot{\mathbf{p}}$ follows as

$$\begin{aligned}\dot{\mathbf{p}}(s, t) &= \dot{\mathbf{R}}(\bar{\mathbf{p}} + \Delta\bar{\mathbf{p}}) + \mathbf{R}\Delta\dot{\bar{\mathbf{p}}} + \dot{\mathbf{b}}(t) \\ &= \dot{\alpha}\mathbf{R}'(\bar{\mathbf{p}} + \Delta\bar{\mathbf{p}}) + \mathbf{R}\Delta\dot{\bar{\mathbf{p}}} + \dot{\mathbf{b}},\end{aligned}\quad (4.55)$$

with the following derivative terms:

$$\dot{\bar{\mathbf{p}}} = \mathbf{E}_u \mathbf{J} \dot{\mathbf{u}}, \quad \dot{\alpha} = \frac{\partial \alpha}{\partial \mathbf{u}} \dot{\mathbf{u}}, \quad (4.56)$$

$$\mathbf{R}' = \begin{bmatrix} -\sin(\alpha) & -\cos(\alpha) & 0 \\ \cos(\alpha) & -\sin(\alpha) & 0 \\ 0 & 0 & 0 \end{bmatrix}, \quad \frac{\partial \alpha}{\partial \mathbf{u}} = \frac{1}{\Delta x^2 + \Delta y^2} \begin{bmatrix} \Delta y \\ -\Delta x \\ 0 \\ -\Delta y \\ \Delta x \\ 0 \end{bmatrix}. \quad (4.57)$$

These transformations enable the evaluation of the beam's deflection curve between the nodes, leading to finer-grained simulation output irrespective of the number of elements being used.

4.2.4 Cross-Section Properties

When we derived the elastic properties of the beam segment, we did it in a fairly general way. The only assumption about the beam's cross-sections was that they exhibit a linear relationship between forces and strains, as formalized by equation (4.23), where this relationship is described by the stiffness matrix $\mathbf{C} \in \mathbb{R}^{3 \times 3}$ of the section. But in practice we have to calculate these matrices for a subset of cross-sections that is relevant to VirtualBow, which we are going to do here. The following derivations have been adapted from [13] and [28].

Consider a laminated cross-section as shown in Figure 4.11. It is symmetric with respect to the y -axis, bends/rotates around the z -axis and consists of a number of rectangular layers. Each layer has its own density ρ_i , Young's modulus E_i and shear modulus G_i . The geometry of the layers is given by their width w_i , height h_i and center position y_i .

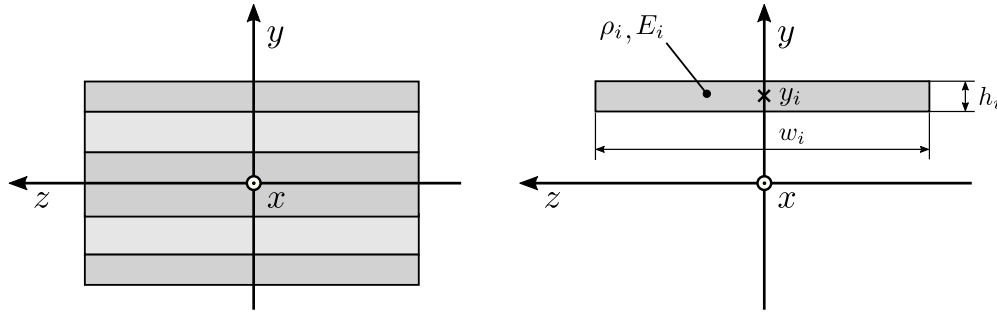


Figure 4.11: Composite cross-section and single layer

In Euler-Bernoulli beam theory the cross-sections are assumed to stay flat and perpendicular to the beam axis during deformation. We assume this to be true for our laminated cross-section as well. The distribution of normal strain over the section is therefore given by the continuous function

$$\bar{\varepsilon}(y) = \varepsilon - \kappa y, \quad (4.58)$$

where ε is the strain at the beam's centerline in x -direction and κ its curvature around the z -axis. The stress distribution, however, is not necessarily continuous, because each layer can have a different elastic modulus. According to Hooke's law, the stress within a layer is given by

$$\sigma_i(y) = E_i \cdot \bar{\varepsilon}(y), \quad y \in [y_i - \frac{h_i}{2}, y_i + \frac{h_i}{2}]. \quad (4.59)$$

In order to determine the elastic constants C_{ij} of the section's stiffness matrix, we calculate the total normal force N and bending moment M acting on the cross-section by integrating the stresses (4.59) over the section's area. For the normal force this amounts to

$$\begin{aligned}
N &= \int_A \sigma \, dA = \sum_i \int_{A_i} \sigma_i \, dA_i = \sum_i \int_{A_i} E_i \bar{\varepsilon}(y) \, dA_i = \sum_i E_i \int_{A_i} (\varepsilon - \kappa y) \, dA_i, \\
&= \left(\sum_i E_i \int_{A_i} dA_i \right) \varepsilon - \left(\sum_i E_i \int_{A_i} y \, dA_i \right) \kappa, \\
&= \underbrace{\left(\sum_i E_i A_i \right)}_{C_{\varepsilon\varepsilon}} \varepsilon - \underbrace{\left(\sum_i E_i A_i y_i \right)}_{C_{\varepsilon\kappa}} \kappa.
\end{aligned} \tag{4.60}$$

This gives us the term $C_{\varepsilon\varepsilon}$ for the stiffness of the section against tension and the coupling term $C_{\varepsilon\kappa}$ between tension and bending. The same can be done for the bending moment,

$$\begin{aligned}
M &= - \int_A y \sigma \, dA = - \sum_i \int_{A_i} y \sigma_i \, dA_i, \\
&= - \sum_i \int_{A_i} y E_i (\varepsilon - \kappa y) \, dA_i = - \sum_i \int_{A_i} (E_i \varepsilon y - E_i \kappa y^2) \, dA_i, \\
&= - \left(\sum_i E_i \int_{A_i} y \, dA_i \right) \varepsilon + \left(\sum_i E_i \int_{A_i} y^2 \, dA_i \right) \kappa, \\
&= - \underbrace{\left(\sum_i E_i A_i y_i \right)}_{C_{\varepsilon\kappa}} \varepsilon + \underbrace{\left(\sum_i E_i I_i \right)}_{C_{\kappa\kappa}} \kappa.
\end{aligned} \tag{4.61}$$

This gives us the same coupling term $C_{\varepsilon\kappa}$ as well as the stiffness $C_{\kappa\kappa}$ against bending. Note that the coupling between tension and bending would vanish if the beam's reference line passed through the centroid of each section. This is often assumed for simplicity, but in the case of VirtualBow the beam's reference line is the user-defined profile curve, which does not generally have this property.

One section force is still missing: the shear force Q . Unfortunately, the distribution of shear stresses over a cross-section like this is pretty complicated. But as already mentioned, VirtualBow doesn't make use of the shear component right now; it merely acts as a placeholder. So we are going to make a crude approximation for the shear stiffness, based on a constant shear stress within each layer:

$$Q = \int_A \tau \, dA = \sum_i \int_{A_i} \tau_i \, dA_i = \sum_i \int_{A_i} G_i \gamma \, dA_i = \underbrace{\left(\sum_i G_i A_i \right)}_{C_{\gamma\gamma}} \gamma. \tag{4.62}$$

The plan is to make this approximation obsolete in the future by switching to a fully numerical cross-section analysis instead, which accurately computes all stiffnesses and stress distributions of the section, including those for shear and torsion, and doesn't make any kinematic simplifications as in the Euler-Bernoulli or Timoshenko theories. But for now we end up with the following analytical stiffnesses:

$$C_{\varepsilon\varepsilon} = \sum_i E_i A_i, \quad C_{\kappa\kappa} = \sum_i E_i I_i, \quad (4.63)$$

$$C_{\gamma\gamma} = \sum_i G_i A_i, \quad C_{\varepsilon\kappa} = - \sum_i E_i A_i y_i. \quad (4.64)$$

Combining them into the section's stiffness matrix results in

$$\mathbf{C} = \begin{bmatrix} C_{\varepsilon\varepsilon} & C_{\varepsilon\gamma} & 0 \\ C_{\varepsilon\gamma} & C_{\gamma\gamma} & 0 \\ 0 & 0 & C_{\kappa\kappa} \end{bmatrix} \quad (4.65)$$

as defined in (4.23). In the case of the rectangular layers shown above, the area A_i and the second moment of area I_i evaluate to

$$A_i = w_i h_i, \quad I_i = A_i \left(\frac{h_i^2}{12} + y_i^2 \right). \quad (4.66)$$

To complete the element's mass matrix, we're also going to need the linear density (mass per unit length) of the section. This can be calculated in a straightforward way by summing up the densities of the individual layers as

$$\mu = \sum_i \rho_i A_i. \quad (4.67)$$

4.2.5 Lumped Masses

When trying to come up with the diagonal entries of our lumped mass matrix (4.15), we are basically asking how to pick concentrated masses and moments of inertia m_0 , J_0 and m_1 , J_1 at the nodes of the element such that it behaves like the actual beam segment, whose mass is continuously distributed along the curved and moving beam axis. It is not surprising that this isn't possible in an exact sense. There is a reason, after all, why the consistent mass matrix for a corotational beam element is as complex as it is. So we will have to make reasonable approximations.

First of all, we can compute the total mass of the beam segment by integrating the section's linear density (4.67) over its length, which yields

$$m = \int_0^l \mu(s) ds. \quad (4.68)$$

We distribute it between the nodes by assigning each one the mass of its half of the segment,

$$m_0 = \int_0^{\frac{l}{2}} \mu(s) ds, \quad m_1 = \int_{\frac{l}{2}}^l \mu(s) ds. \quad (4.69)$$

This conserves the total mass of the element, which is an important requirement. It does not, however, conserve the rigid-body inertia of the element. That's because concentrating the mass at the two nodes has turned the element into something like a barbell, with all the mass pushed to the ends, thereby increasing the inertia.

Determining the rotary inertias J_0 and J_1 is much less straightforward. One reason is that there is no comparable "total inertia" to divide up. The other reason is that the total rigid-body inertia of the element is already too large, as explained above, even before adding J_0 and J_1 . So in a certain sense, the most accurate values for these would be zero or even negative, which is obviously not permissible. This is the fundamental dilemma when lumping the masses of a beam, which is also discussed in [9]. According to that reference, a common choice for straight, uniform Euler-Bernoulli beams is

$$J_0 = J_1 = \alpha \cdot \rho A l^3 \quad (4.70)$$

with a fudge factor α that should be small due to the dilemma described above and is typically chosen between 0 and $1/50$. The expression $\rho A l^3$ is proportional to the beam's rigid rotary inertia, but the exact point of reference is baked into α .

We attempt to generalize this for our case of a non-uniform and curved beam by defining the nodal moments of inertia as

$$J_0 = \alpha \int_0^l \mu(s) \|\mathbf{r}(s) - \mathbf{r}(0)\|^2 ds, \quad (4.71)$$

$$J_1 = \alpha \int_0^l \mu(s) \|\mathbf{r}(s) - \mathbf{r}(l)\|^2 ds, \quad (4.72)$$

where α is a different fudge factor and the integrals are the rigid-body inertia of the segment with respect to each of the nodes.

4.2.6 Damping Matrix

The last missing piece for the beam element is the damping matrix $\overline{\mathbf{D}}$ of the beam segment, analogous to its stiffness matrix, as required for the calculation of the local forces (4.5). Since the stiffness matrix has been derived from the segment's material properties and geometry, it would seem logical to do the same for the damping matrix as well. But characterizing the dissipative properties of materials is much trickier than characterizing their elastic properties. Most common materials do not follow a simple, linear damping law that would lead to a linear damping matrix in the first place. Therefore the damping matrix can only ever be a crude approximation, such that at least some energy dissipation is included in the bow model.

So instead of deriving the damping matrices from actual material properties, we are going to identify them based on the damping properties of the overall system, i.e. the bow limb. The damping model that is utilized for this is called *Rayleigh damping* or *proportional damping*, which is a simple approach that is very popular when little is known about the underlying physical details of the damping. The idea is to write the damping matrix of a linear(ized) system as a weighted sum of its stiffness and mass matrices,

$$\mathbf{D} = \alpha \mathbf{K} + \beta \mathbf{M} \quad (4.73)$$

with the factor α for *stiffness-proportional* damping and the factor β for *mass-proportional* damping. The stiffness-proportional part dampens the system as if the material behaved in a viscous way while the mass-proportional part dampens the system as if the system was moving in a viscous medium. It can be shown [12] that the natural frequencies ω_i and the corresponding damping ratios ζ_i in such a system are related by

$$\zeta_i = \frac{1}{2} \left(\alpha \omega_i + \frac{\beta}{\omega_i} \right). \quad (4.74)$$

This relationship can be used to identify α and β from prescribed values of the damping ratio at specific modes. In our case we use stiffness-proportional damping only, so β is

set to zero. If we assign a damping ratio of ζ_0 to the first mode of the system, for which we know the undamped natural frequency ω_0 , we can determine the required Rayleigh parameter α as

$$\zeta_0 = \frac{1}{2}\alpha\omega_0 \quad \Rightarrow \quad \alpha = \frac{2\zeta_0}{\omega_0}. \quad (4.75)$$

In practice this works as follows for the bow limb. The value ζ_0 is set by users as the damping ratio of the first eigenmode of the limb. This is supposed to be an empirical value that can be experimentally determined on real bows and then chosen from experience for simulations. When the bow simulation is set up, the limb is initialized without damping at first. With this undamped limb model, before the string is attached, a modal analysis is performed to determine its first undamped natural frequency. Using this frequency and the prescribed damping ratio, the required damping coefficient α is determined from equation (4.75). The damping matrices of each beam element are then initialized as $\bar{\mathbf{D}} = \alpha\bar{\mathbf{K}}$. When the system is linearized, as is the case here, the factor α carries over from the corotational elements to the system level as in (4.73).

4.3 String Element

4.3.1 Internal Forces

An idealized string only transfers tensile forces and has no resistance to shear and bending. Figure 4.12 shows such a string with an initial length of l_0 . When subjected to a normal force N it elongates by Δl .

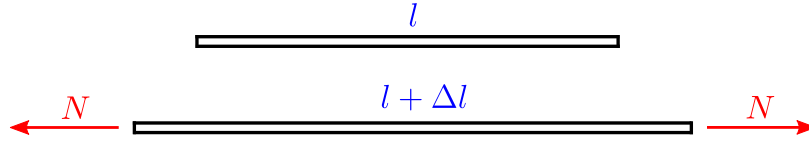


Figure 4.12: Elongation of the string under load

We assume the strain $\varepsilon = \frac{\Delta l}{l}$ to be constant over the length of the string. In order to simulate the elastic as well as the damping properties of the string, a viscoelastic material model of the form

$$\sigma = E\varepsilon + \eta\dot{\varepsilon} \quad (4.76)$$

is chosen. Here E is the elastic modulus and η the viscosity of the material. This model is called the *Kelvin-Voigt model* and combines linear elastic and linear viscous behavior. We will later identify η based on the prescribed damping ratio of the string, similar to how we identified the damping properties of the beam elements. The normal force in the string, given a constant cross-section area A , is therefore

$$N = \sigma A = EA\varepsilon + \eta A\dot{\varepsilon}. \quad (4.77)$$

The product EA is also called the *longitudinal stiffness*. To use this as the string model in our bow simulation, we have to pick a set of nodes and formulate the internal forces in terms of the nodal coordinates. It seems reasonable to place the string between two nodes, one start point and one endpoint, but there is an additional complication: the string model must also handle recurve bows, so we have to consider its possible contact with the bow limb. This means that the current shape of the string, as well as its internal forces, don't only depend on its own start point and endpoint, but also on the nodes of the limb. Figure 4.13 shows the resulting setup for our element. The first node, with the position (x_0, y_0) , is the starting point of the string. This is the same node where the arrow is attached in the final model. This is followed by the nodes (x_i, y_i, φ_i) , $i = 1 \dots n - 1$ of the limb's reference line. Since the string doesn't contact the limb at its reference line, but at its outer surface, each of those also carries a distance d_i , which defines a possible contact point perpendicular to the reference line. The contact point of the last

limb node $(x_{n-1}, y_{n-1}, \varphi_{n-1})$ is the endpoint of the string; this would be the limb tip in the bow model.

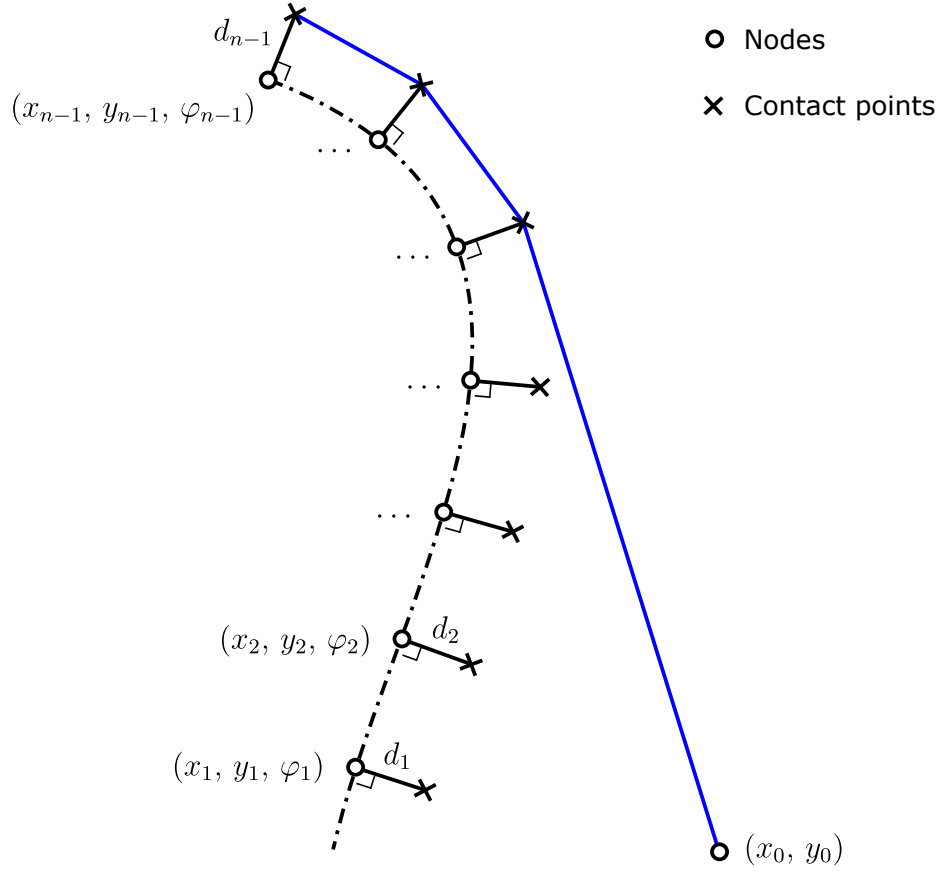


Figure 4.13: Contact interface between limb and string

One shortcoming of this approach is already apparent: the contact with the string is determined only by the positions of the limb nodes, not by the actual deformed geometry of the limb's beam elements. This is a possible future improvement that would allow us to accurately resolve the string contact with a much smaller number of limb nodes/elements.

We define the coordinate vector $\mathbf{u}$ of the element, describing the string's current configuration, as

$$\mathbf{u} = [x_0, y_0, x_1, y_1, \varphi_1, \dots, x_{n-1}, y_{n-1}, \varphi_{n-1}]^T \quad (4.78)$$

The first step in evaluating the internal forces of this element is to determine which of the limb nodes make contact with the string. The contact point candidates at the belly surface of the limb are given by

$$\bar{x}_0 = x_0, \quad \bar{x}_i = x_i - d_i \sin(\varphi_i) \quad (4.79)$$

$$\bar{y}_0 = y_0, \quad \bar{y}_i = y_i + d_i \cos(\varphi_i) \quad (4.80)$$

with the perpendicular distances d_i as obtained from the limb's cross-section geometry. The task of finding the contact points is then equivalent to finding the convex hull of the points $\{x_0, y_0, \bar{x}_1, \bar{y}_1, \dots, \bar{x}_{n-1}, \bar{y}_{n-1}\}$. This is currently implemented by a method similar to the *Graham scan* [27], but adapted for 1D lines. The scan is performed on each evaluation of the element. A possible improvement over this would be an algorithm that keeps track of the contact points and only considers what has changed since the last evaluation.

Having found the contact points, we now know the exact geometrical configuration of the string and can begin formulating the internal forces for the element. Let's denote all points that are part of the convex hull by the index k . The distance between two surface points i and j is given by

$$l_{ij} = \sqrt{(\bar{x}_j - \bar{x}_i)^2 + (\bar{y}_j - \bar{y}_i)^2} \quad (4.81)$$

The total length of the string is the sum of all line segments of the convex hull,

$$l(\mathbf{u}) = \sum_k l_{k-1,k}. \quad (4.82)$$

The strain and strain rate are expressed in terms of the overall length l as

$$\varepsilon = \frac{l(\mathbf{u}) - l_0}{l_0}, \quad \dot{\varepsilon} = \frac{1}{l_0} \frac{\partial l}{\partial \mathbf{u}} \dot{\mathbf{u}} \quad (4.83)$$

The virtual work of the internal forces is obtained by integrating the product of stress σ and the strain variation $\delta\varepsilon$ over the element's volume, leading to

$$\delta W_I = \int_V \sigma \delta\varepsilon dV = \int_0^{l_0} \int_A \sigma \delta\varepsilon dA ds = (EA\varepsilon + \eta A\dot{\varepsilon}) l_0 \frac{\partial \varepsilon}{\partial \mathbf{u}} \delta \mathbf{u}. \quad (4.84)$$

The internal forces themselves are therefore

$$\mathbf{Q}(\mathbf{u}, \dot{\mathbf{u}}) = (EA\varepsilon + \eta A\dot{\varepsilon}) l_0 \frac{\partial \varepsilon}{\partial \mathbf{u}} = \underbrace{\left(k(l - l_0) + d\dot{l} \right)}_{N(\mathbf{u}, \dot{\mathbf{u}})} \frac{\partial l}{\partial \mathbf{u}} \quad (4.85)$$

where $k = EA/l_0$ and $d = \eta A/l_0$ are the stiffness and damping, respectively, corresponding to a linear spring-damper, and N is the normal force within the string. The tangent stiffness matrix is derived from the internal forces as

$$\begin{aligned} \mathbf{K} = \frac{\partial \mathbf{Q}}{\partial \mathbf{u}} &= \left(k \frac{\partial l}{\partial \mathbf{u}}^\top + d \frac{\partial \dot{l}}{\partial \mathbf{u}}^\top \right) \frac{\partial l}{\partial \mathbf{u}} + \left(k(l - l_0) + d\dot{l} \right) \frac{\partial^2 l}{\partial \mathbf{u}^2}, \\ &= \left(k \frac{\partial l}{\partial \mathbf{u}}^\top + d \dot{\mathbf{u}}^\top \frac{\partial^2 l}{\partial \mathbf{u}^2} \right) \frac{\partial l}{\partial \mathbf{u}} + \left(k(l - l_0) + d\dot{l} \right) \frac{\partial^2 l}{\partial \mathbf{u}^2}. \end{aligned} \quad (4.86)$$

Similarly, the tangent damping matrix is obtained as

$$\mathbf{D} = \frac{\partial \mathbf{Q}}{\partial \dot{\mathbf{u}}} = d \frac{\partial \dot{l}}{\partial \dot{\mathbf{u}}} \frac{\partial l}{\partial \mathbf{u}} = d \frac{\partial l}{\partial \mathbf{u}}^\top \frac{\partial l}{\partial \mathbf{u}}. \quad (4.87)$$

The derivatives $\partial l / \partial \mathbf{u}$ and $\partial^2 l / \partial \mathbf{u}^2$ of the string length, which are required for evaluating these expressions, are carried out in detail in Appendix B.3.

One final tweak to the string model is required for dynamic analysis. Right now the model resists tension as well as compression, even though one of the defining properties of a string or cable is that it only resists tension, but not compression. For static analysis this is not an actual problem, since the string should be under tension in any equilibrium state of the bow anyway. In dynamic analysis, however, especially in the slightly chaotic phase after the arrow leaves the string, it can happen that the string becomes shorter than its initial length, which means that it is under compression. This must not generate forces that counteract that compression; otherwise the string would behave like a bar and not like a string. In other words, the stiffness of the string in compression should be zero. Since a stiffness of exactly zero, however, leads to a singular tangent stiffness matrix for the system, which creates problems, we set it to a very small value instead by multiplying it with a small factor $c \ll 1$. Doing the same for the damping, the longitudinal stiffness EA^* and damping ηA^* used for dynamic analysis become

$$EA^* = \begin{cases} EA, & \text{if } \Delta l \geq 0 \\ c \cdot EA, & \text{otherwise} \end{cases}, \quad \eta A^* = \begin{cases} \eta A, & \text{if } \Delta l \geq 0 \\ c \cdot \eta A, & \text{otherwise} \end{cases} \quad (4.88)$$

depending on the sign of the elongation Δl . This introduces a non-smooth transition at $\Delta l = 0$, which is not ideal, but hasn't caused any obvious problems in practice either.

4.3.2 Lumped Masses

We make no attempt to derive a consistent mass matrix for this element, or any mass matrix at all. There is no way it would have any nice properties, like being constant or diagonal. Instead we keep the element massless and move the responsibility of accounting for the string's mass to the bow model, where this is done by dividing it between two point masses placed at the center and the end of the string.

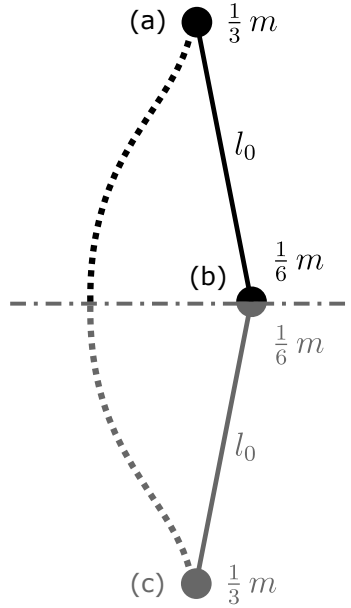


Figure 4.14: Lumping of the string's total mass to three points

When determining those point masses, we have to keep in mind that the string element we derived only models and simulates one half of the string, while the actual bow is mirrored as shown in Figure 4.14. Since the length l_0 of the element is only half the total length of the string, the total string mass is $m = 2\rho A l_0$. According to Figure 4.14 there are three obvious points for distributing this mass: the upper end (a) of the string, the string center (b) and the lower end (c). If the total mass is evenly divided, each of those points is given a mass of $\frac{1}{3}m$. But only the mass (a) and half of (b) are actually part of the upper half of the bow that is being simulated. So the actual masses for the model are $\frac{1}{3}m$ for the upper string end (a) and $\frac{1}{6}m$ as the upper half of the center mass (b). The same distribution of the string's mass was also used by Kooi [16], which seems to confirm this reasoning.

4.3.3 Damping

The question of identifying the viscosity η of the string material is still open. The idea is, as with the limb, to prescribe a certain damping ratio for the string and identify the viscosity based on that. But unlike for the limbs, we don't need a numerical modal analysis of the string to do this, since we can carry it out analytically. As shown in Appendix B.4, the damping ratio of the first longitudinal eigenmode of a continuous string of initial length l_0 is

$$\zeta = \frac{\pi}{4l_0} \frac{\eta A}{\sqrt{\rho A \cdot EA}} \quad (4.89)$$

which, when solved for ηA , gives us the effective viscosity of the string as

$$\eta A = \frac{4l_0}{\pi} \sqrt{\rho A \cdot EA} \cdot \zeta, \quad (4.90)$$

where ζ is the desired damping ratio. It is worth noting that these longitudinal eigenmodes only apply to a continuum model of a string and are not actually present in the string element as we modeled it. But that doesn't matter for our purpose here.

Chapter 5

Static Solution

Having assembled the various element equations into an overall equation of motion for the bow-and-arrow system, the actual simulation consists of solving these equations of motion for a given initial state and boundary conditions. Different numerical methods are needed for this, depending on whether a static or dynamic analysis is to be performed.

The first part of the bow simulation, which encompasses bracing the bow and drawing it from brace height to full draw, is static in nature. That's because the process is fairly slow and therefore the influence of the moving masses of the bow can be neglected. A system such as ours, with an equation of motion of the form

$$M\ddot{\mathbf{u}} + \mathbf{Q}(\mathbf{u}, \dot{\mathbf{u}}) = \mathbf{P}, \quad (5.1)$$

is in static equilibrium if the nodal positions $\mathbf{u}(t)$ stay constant over time. This requires the velocities and accelerations to be zero, i.e. $\dot{\mathbf{u}}(t), \ddot{\mathbf{u}}(t) \equiv \mathbf{0}$, leading to the static equilibrium condition

$$\underbrace{\mathbf{Q}(\mathbf{u}, \mathbf{0}) - \mathbf{P}}_{\mathbf{R}(\mathbf{u})} = \mathbf{0}. \quad (5.2)$$

In words, all components of the internal forces $\mathbf{Q}$ have to be in balance with the respective external loads $\mathbf{P}$. We introduce the residual forces $\mathbf{R}(\mathbf{u})$ as a shorthand for the difference between the two. The equilibrium condition $\mathbf{R}(\mathbf{u}) = \mathbf{0}$ is a nonlinear system of equations for the unknown equilibrium displacements $\mathbf{u}$ and/or external forces $\mathbf{P}$, depending on the exact problem statement.

5.1 The Basic Newton-Raphson Method

In the most basic case, the external loads $\mathbf{P}$ are given and the task is to calculate the corresponding equilibrium displacements. This case is called *load-controlled*. Equation (5.2) becomes a nonlinear system of equations in $\mathbf{u}$ exclusively. Except for very simple systems, this can only be solved numerically. A commonly used method for this kind of problem is the *Newton-Raphson method*. The idea behind it is to linearize the equilibrium equation (5.2) around a known displacement vector $\mathbf{u}_i$ and solve the resulting linear equation in order to obtain an improved approximation $\mathbf{u}_{i+1}$. Starting with an initial value $\mathbf{u}_0$, the procedure is repeated until the solution is accurate enough, as determined by some convergence criterion. Linearization of (5.2) yields

$$\mathbf{R}(\mathbf{u}_i) + \underbrace{\left(\frac{\partial \mathbf{R}}{\partial \mathbf{u}} \bigg|_{\mathbf{u}_i} \right)}_{\mathbf{K}(\mathbf{u}_i)} \underbrace{(\mathbf{u}_{i+1} - \mathbf{u}_i)}_{\Delta \mathbf{u}_i} = \mathbf{0}. \quad (5.3)$$

Solving this for the solution increment $\Delta \mathbf{u}_i$ leads to the iterative procedure

$$\Delta \mathbf{u}_i = \mathbf{K}^{-1}(\mathbf{u}_i) (\mathbf{P} - \mathbf{Q}(\mathbf{u}_i)), \quad (5.4)$$

$$\mathbf{u}_{i+1} = \mathbf{u}_i + \Delta \mathbf{u}_i. \quad (5.5)$$

which is carried out until the solution is accurate enough, as determined by a convergence criterion based on either the increment $\Delta \mathbf{u}_i$, the residual $\mathbf{R}(\mathbf{u}_i)$ or a combination of both. We will have a more thorough look at the convergence criterion later. In the implementation, the tangent stiffness matrix in (5.4) is not actually inverted, because that would be numerically inefficient. Instead, $\Delta \mathbf{u}_i$ is obtained by solving the equivalent linear system of equations using an appropriate matrix decomposition. Since the tangent stiffness matrix is symmetric, an algorithm that exploits this can be selected. It is not generally positive definite, though, unlike the stiffness matrices of many linear mechanical systems.

This basic Newton-Raphson scheme can be improved and extended in many ways, for example by combining it with a line search or by avoiding the calculation of the tangent stiffness matrix at every iteration. Relevant keywords in this context are *simplified Newton methods* and *quasi-Newton methods* such as the *BFGS method*. The first extension we are going to look at, which is essential for our bow simulation, is turning our load-controlled method into a *displacement controlled* one.

5.2 Load- and Displacement Control

When simulating the drawing of the bow, the set of known and unknown variables is not quite the same as assumed in the previous section. The vector of external forces consists of the draw force, which is applied at the center node of the string. But while the direction of this draw force is known, its magnitude is not. We do know, however, the position of the string node, since that corresponds to the current draw length at which the bow is to be evaluated. A solution to this is *displacement control*, where we prescribe a value for one of the displacement components and instead treat the magnitude of the external forces as a variable that is determined during solution. Displacement control also has the benefit that it still works when there is a decrease in load in the force-draw curve, which can theoretically happen with some extreme recurve designs.

Before the bow is drawn, there is actually another static simulation step: bracing. Starting from the bow's initial, unbraced state, the braced equilibrium state of the bow with an applied string is determined. The bracing simulation is described in more detail later; it too makes use of displacement control.

The following derivations have been adapted from [20]. Displacement control is a special case of a wider class of methods where the equilibrium equations are modified as

$$\mathbf{Q}(\mathbf{u}) - \lambda \mathbf{P} = 0, \quad (5.6)$$

$$c(\mathbf{u}, \lambda) = 0. \quad (5.7)$$

Here the external forces are scaled by the unknown load factor λ . To compensate for this additional unknown, a scalar constraint $c(\mathbf{u}, \lambda) = 0$ on the displacements and the load factor is introduced. Depending on the choice of constraint, the method can be turned into *load control*, *displacement control* or other more sophisticated schemes like the *arc-length method*. We will carry out the derivation in a generic way and specify the constraints only as the last step. Let us first restate equations (5.6) and (5.7) in more general terms as

$$\mathbf{f}(\mathbf{x}, \lambda) = 0, \quad (5.8)$$

$$c(\mathbf{x}, \lambda) = 0, \quad (5.9)$$

with the functions $\mathbf{f} : \mathbb{R}^n, \mathbb{R} \rightarrow \mathbb{R}^n$ and $c : \mathbb{R}^n, \mathbb{R} \rightarrow \mathbb{R}$. This system of nonlinear equations has to be solved for the unknown vector $\mathbf{x}$ and the scalar parameter λ . As before, the Newton-Raphson method is used: The equations are linearized around a given approximation $\{\mathbf{x}_i, \lambda_i\}$ and the resulting linear system is solved to obtain an improved approximation $\{\mathbf{x}_{i+1}, \lambda_{i+1}\}$. This iterative process is repeated until $\mathbf{x}$ and λ satisfy both the vector equation and the scalar constraint reasonably well as determined by some convergence criterion. Carrying out the linearization leads to the two equations

$$\mathbf{f}(\mathbf{x}_i + \Delta\mathbf{x}_i, \lambda_i + \Delta\lambda_i) = \mathbf{f}(\mathbf{x}_i, \lambda_i) + \frac{\partial \mathbf{f}}{\partial \mathbf{x}} \Delta\mathbf{x}_i + \frac{\partial \mathbf{f}}{\partial \lambda} \Delta\lambda_i \quad (5.10)$$

$$c(\mathbf{x}_i + \Delta\mathbf{x}_i, \lambda_i + \Delta\lambda_i) = c(\mathbf{x}_i, \lambda_i) + \frac{\partial c}{\partial \mathbf{x}} \Delta\mathbf{x}_i + \frac{\partial c}{\partial \lambda} \Delta\lambda_i \quad (5.11)$$

In order to solve this for the solution increments $\Delta\mathbf{x}_i$ and $\Delta\lambda_i$, we could rearrange the equations (5.10) and (5.11) into a single linear system

$$\begin{bmatrix} \frac{\partial \mathbf{f}}{\partial \mathbf{x}} & \frac{\partial \mathbf{f}}{\partial \lambda} \\ \frac{\partial c}{\partial \mathbf{x}} & \frac{\partial c}{\partial \lambda} \end{bmatrix} \begin{bmatrix} \Delta\mathbf{x}_i \\ \Delta\lambda_i \end{bmatrix} = - \begin{bmatrix} \mathbf{f}(\mathbf{x}_i, \lambda_i) \\ c(\mathbf{x}_i, \lambda_i) \end{bmatrix}. \quad (5.12)$$

and solve this with standard methods. But it's much more numerically efficient to solve it by block elimination, i.e. by considering the equations separately. From equation (5.10), the unknowns $\Delta\mathbf{x}_i$ can be expressed as

$$\begin{aligned} \Delta\mathbf{x}_i &= - \underbrace{\left(\frac{\partial \mathbf{f}}{\partial \mathbf{x}} \right)^{-1} \mathbf{f}(\mathbf{x}_i, \lambda_i)}_{\boldsymbol{\alpha}} - \underbrace{\left(\frac{\partial \mathbf{f}}{\partial \mathbf{x}} \right)^{-1} \frac{\partial \mathbf{f}}{\partial \lambda} \Delta\lambda_i}_{\boldsymbol{\beta}} \\ &= \boldsymbol{\alpha} + \boldsymbol{\beta} \Delta\lambda_i \end{aligned} \quad (5.13)$$

with the auxiliary vectors $\boldsymbol{\alpha}$ and $\boldsymbol{\beta}$ being computed by factorizing the Jacobian matrix $\partial \mathbf{f} / \partial \mathbf{x}$ and computing two solutions with different right-hand sides. Substituting (5.13) into the second equation (5.11) gives us the parameter increment as

$$\Delta\lambda_i = - \frac{c(\mathbf{x}_i, \lambda_i) + \frac{\partial c}{\partial \mathbf{x}} \boldsymbol{\alpha}}{\frac{\partial c}{\partial \lambda} + \frac{\partial c}{\partial \mathbf{x}} \boldsymbol{\beta}}. \quad (5.14)$$

This is then used to compute $\Delta\mathbf{x}_i$ according to (5.13). From equation (5.14) we can see that the parameter increment $\Delta\lambda_i$ is invalid if the denominator of the expression vanishes, i.e. $\frac{\partial c}{\partial \lambda} + \frac{\partial c}{\partial \mathbf{x}} \boldsymbol{\beta} = 0$. This needs to be checked in each iteration and usually points to an ill-posed problem with regard to the imposed constraint function.

Finally, the constraint function has to be chosen. To get a displacement-controlled method as initially stated, we choose the constraint function as

$$c(\mathbf{u}, \lambda) = u^{(k)} - \bar{u} \quad (5.15)$$

where $u^{(k)}$ is the k -th component of the displacement vector that we want to control and $\bar{u}$ is the prescribed value for it. The partial derivatives are

$$\frac{\partial c}{\partial \mathbf{u}} = \mathbf{e}_k, \quad \frac{\partial c}{\partial \lambda} = 0,$$

with e_k being the k -th unit vector. If a load-controlled method is needed instead, the constraint becomes

$$c(\mathbf{u}, \lambda) = \lambda - \bar{\lambda}, \quad (5.16)$$

with the desired load factor $\bar{\lambda}$. The partial derivatives are then

$$\frac{\partial c}{\partial \mathbf{u}} = \mathbf{0}, \quad \frac{\partial c}{\partial \lambda} = 1.$$

5.3 Convergence Criterion

In order to stop the iteration when an acceptable level of accuracy is reached, we need a stopping or convergence criterion. Formulating a convergence criterion that works for all edge cases and whose numerical tolerances scale well with different systems can be surprisingly tricky. Convergence criteria are commonly formulated in terms of either the displacement step $\Delta \mathbf{x}$, the residuals $\mathbf{f}$ or their product $\Delta \mathbf{x}^\top \mathbf{f}$, which has the dimension of an energy in our case. They can furthermore be based either on the absolute error of the respective quantity or on a relative error with respect to the start of the iteration. An overview of some of the possibilities is shown in Table 5.1.

Quantity	Absolute Criterion	Relative Criterion
$\mathbf{f}$	$\ \mathbf{f}_i\ < \varepsilon_{\text{abs}}$	$\ \mathbf{f}_i\ / \ \mathbf{f}_0\ < \varepsilon_{\text{rel}}$
$\Delta \mathbf{x}$	$\ \Delta \mathbf{x}_i\ < \varepsilon_{\text{abs}}$	$\ \Delta \mathbf{x}_i\ / \ \Delta \mathbf{x}_0\ < \varepsilon_{\text{rel}}$
$\Delta \mathbf{x}^\top \mathbf{f}$	$\ \Delta \mathbf{x}_i^\top \mathbf{f}_i\ < \varepsilon_{\text{abs}}$	$\ \Delta \mathbf{x}_i^\top \mathbf{f}_i\ / \ \Delta \mathbf{x}_0^\top \mathbf{f}_0\ < \varepsilon_{\text{rel}}$

Table 5.1: Different absolute and relative convergence criteria for the Newton-Raphson method. The vector norm $\|\cdot\|$ can also be chosen freely.

Each choice has advantages and disadvantages; therefore it is common to combine some of those criteria. Relative criteria are more appealing because they are dimensionless and independent of the overall scale of the system. This means that we can find a generic value for the tolerance ε_{rel} that will work most of the time and for very different systems. The absolute tolerances ε_{abs} , on the other hand, represent a different accuracy depending on what typical forces, displacements or energies arise in the system. Unfortunately, though, relative criteria are not defined for the very first iteration and also don't work when the system is already in (or close to) static equilibrium at the start of the iterations. That's why they have to be combined with an absolute criterion anyway.

Since what we want to compute are the equilibrium displacements, it would make sense to use a criterion on $\Delta \mathbf{x}$ and stop when the displacements do not change meaningfully anymore. It is, however, possible that the iterative displacement change is very small while the out-of-balance loads $\mathbf{f}$ are still large, which is why Crisfield [6] advises against using a displacement-based criterion on its own without supplementing it with a force-based one.

Criteria based on the residual forces or displacement increments also suffer from an additional scaling problem. Both of these vectors contain translational as well as rotational entries, so the application of a simple vector norm $\|\cdot\|$ does not make much sense since entries with different units (length vs. angle or force vs. torque) are added together. This scaling issue between translational and rotational degrees of freedom is absent in the energy-based criterion: In the expression $\Delta \mathbf{x}^\top \mathbf{f}$, lengths are multiplied by forces and angles are multiplied by torques, resulting in a vector where each entry has the dimension of an energy. This makes the energy criterion very appealing. However, as you might have already guessed, it also comes with serious drawbacks. The condition $\Delta \mathbf{x}^\top \mathbf{f} = 0$ is not only true when $\Delta \mathbf{x}$ and $\mathbf{f}$ are both zero, which is the case we want, but also when they are nonzero and orthogonal to each other. Or, as Crisfield [6] puts it, the condition merely implies that a stationary energy position has been reached along the current search direction $\Delta \mathbf{x}$, which frequently occurs when the solution is still a very long way from equilibrium. He therefore warns that such criteria should be used with great caution.

VirtualBow currently uses only a single, absolute criterion based on the displacement increments $\Delta \mathbf{x}$. The reason for that is that we can actually derive reasonable absolute tolerances from the dimensions of the bow model. We first define a generic numerical tolerance ε , which is called “static iteration tolerance” or “dynamic iteration tolerance” in the UI. From this we derive absolute tolerances for lengths, angles, accelerations and the load factor (which will be needed later) based on a reference length l_{ref} , reference angle φ_{ref} and reference acceleration a_{ref} that represent the overall scale of the model,

$$\varepsilon_{\text{abs},l} = l_{\text{ref}} \cdot \varepsilon \quad (5.17)$$

$$\varepsilon_{\text{abs},\varphi} = \varphi_{\text{ref}} \cdot \varepsilon \quad (5.18)$$

$$\varepsilon_{\text{abs},a} = a_{\text{ref}} \cdot \varepsilon \quad (5.19)$$

$$\varepsilon_{\text{abs},\lambda} = \varepsilon \quad (5.20)$$

The reference length is chosen as the length of the limb and the reference angle is set to $\pi/2$. The reference acceleration for dynamic iterations is the initial acceleration of the arrow, i.e. the draw force divided by the arrow mass. The dimensionless load factor $\lambda \in [0, 1]$ does not require adjusting of the dimension and can just use the generic tolerance ε . The actual convergence criterion for the displacements is then

$$\|\mathbf{S}^{-1} \Delta \mathbf{x}\|_\infty < 1 \quad (5.21)$$

where $\|\cdot\|_\infty$ is the maximum norm of the displacement increments, scaled by a diagonal scaling matrix $\mathbf{S} = \text{diag}(\varepsilon_{\text{abs},0}, \dots, \varepsilon_{\text{abs},n-1})$ that contains the absolute tolerances for each displacement component depending on its type. This solves the problem of mixing dimensions because we can use different tolerances for lengths and angles and the end result will be dimensionless. If the method is constrained, the analogous convergence condition

$$|\Delta\lambda/\varepsilon_{\text{abs},\lambda}| < 1 \quad (5.22)$$

for the load factor has to be fulfilled too, in addition to the condition on the displacements.

5.4 Derivative of the Equilibrium Displacements

The derivative $\partial\mathbf{x}/\partial\lambda$ of the equilibrium displacements with respect to the load factor is used by VirtualBow to obtain the stiffness of the force-draw curve, i.e. the increase in force (load factor) per increase in equilibrium displacement. In static equilibrium, $\mathbf{x}$ and λ are, according to (5.8), a solution to the equation

$$\mathbf{f}(\mathbf{x}, \lambda) = \mathbf{0}. \quad (5.23)$$

Applying the total differential to this equation turns it into the relationship

$$\frac{\partial\mathbf{f}}{\partial\mathbf{x}} \frac{\partial\mathbf{x}}{\partial\lambda} + \frac{\partial\mathbf{f}}{\partial\lambda} = \mathbf{0} \quad (5.24)$$

between the various derivatives. Solving for $\partial\mathbf{x}/\partial\lambda$ we find that the derivative we are looking for is actually equal to the auxiliary vector $\boldsymbol{\beta}$ as defined in (5.13),

$$\frac{\partial\mathbf{x}}{\partial\lambda} = - \left(\frac{\partial\mathbf{f}}{\partial\mathbf{x}} \right)^{-1} \frac{\partial\mathbf{f}}{\partial\lambda} = \boldsymbol{\beta}. \quad (5.25)$$

From this, the stiffness of any displacement $u^{(k)}$ with respect to the load factor is obtained as the inverse

$$K^{(k)} = \frac{\partial\lambda}{\partial u^{(k)}} = \frac{1}{\boldsymbol{\beta}^{(k)}}. \quad (5.26)$$

For VirtualBow, this gives the exact stiffness of the force-draw curve without having to compute a numerical derivative and irrespective of how fine or coarse the resolution of the curve is.

5.5 Finding the Braced Equilibrium of a Bow

The first part of the static simulation also tends to be the trickiest one: Finding the first equilibrium state of the bow at the given brace height. Older versions of VirtualBow in particular would frequently struggle with this step and give users cryptic error messages from failed solution attempts. The bracing method described below is the result of a lot of trial and error and seems to be the most robust so far, even though it is still not perfect.

One difficulty of bracing is that the braced state of the bow tends to be “far away” from the initial, unbraced state. Newton-Raphson iterations, however, are much more efficient and robust when they start from a good initial guess, i.e. an initial state that is not very far from the desired equilibrium. So ideally we would like to go from the unbraced to the braced state in a series of intermediate equilibrium states. The other difficulty is that the length of the string is not known in advance, only the brace height that the bow should have when the string is applied.

The current bracing method tries to solve both of these problems by applying an initially overlong string to the bow and iteratively shortening it until the desired brace height is reached. How much the string is shortened in each step is controlled/limited in a way that increases the chances for convergence.

As shown in Figure 5.1, a string with initial (half-)length l_0 is applied to the unbraced configuration of the bow such that it passes through the desired brace height on the y -axis. This initial length is chosen such that there is no tension yet in the string.

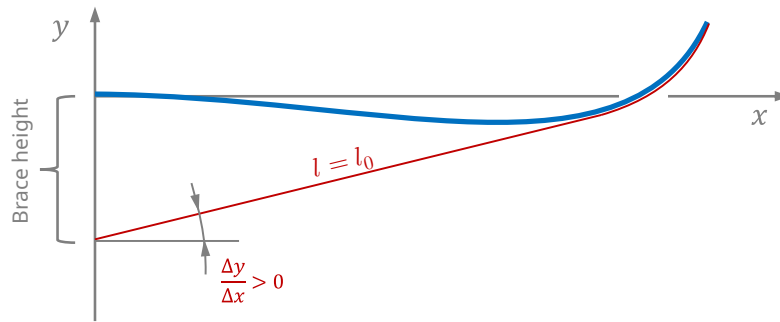


Figure 5.1: Unbraced state

At the point where the string intersects the y axis, it initially has a positive slope of $\Delta y / \Delta x$ as measured relative to the x -direction. If this initial slope were negative, the bow model would have to be rejected as invalid because the specified brace height would be too low for the given limb geometry. The actual bracing process now consists of successively shortening the string and iterating for equilibrium each time, until the slope reaches zero, while also keeping the string center pinned at the desired brace height via displacement control. This pinning of the string ensures a favorable angle (not too shallow) between string and limb.

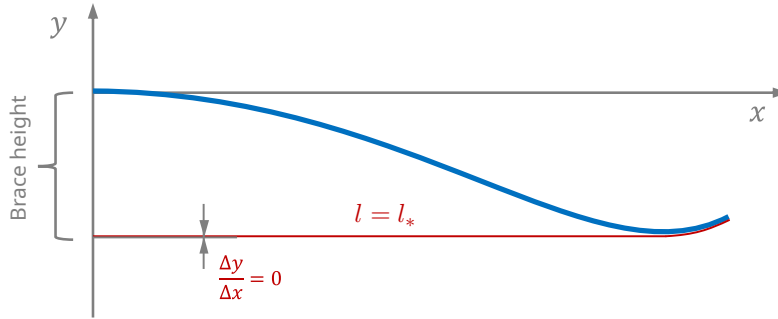


Figure 5.2: Braced state

As the string becomes shorter, the slope decreases until it eventually crosses into the negative. The final string length l_* is the length for which the slope is approximately zero. This is shown in Figure 5.2. So it all reduces to the problem of finding the root $\frac{\Delta y}{\Delta x}(l) = 0$ of the slope as a function of the string length. This can be made dimensionless by introducing the length reduction factor $p = \frac{l}{l_0}$ and writing the slope as $\frac{\Delta y}{\Delta x}(p)$ instead. Figure 5.3 shows that relationship for a real example, where the final string length is found to be approximately $0.96 l_0$.

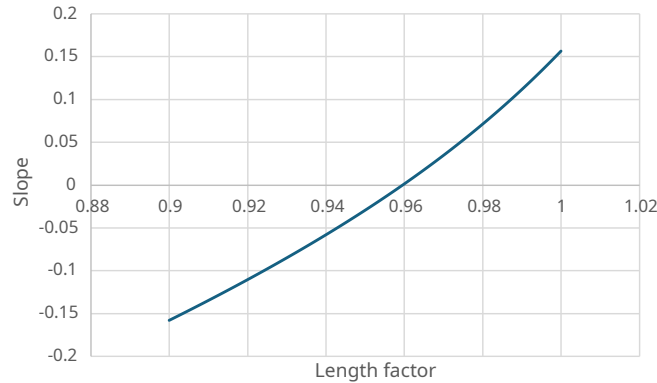


Figure 5.3: Slope versus string length for a real example

However, if we just use one of the many standard root-finding algorithms for this, it might make big steps in string length and cause the underlying static equilibrium iteration to fail. Instead, the method starts at $p = 1$ and treats the problem as one of finding an equilibrium path with the path parameter p . This parameter is controlled by taking into account maximum and minimum bounds on the step size and the performance of the static iterations. A dedicated root-finding algorithm is used only after the first negative slope is encountered in order to home in on a precise final result. Currently this is the *regula falsi method*, which is simple and reasonably effective when derivatives are not available and the interval in which the root lies is known.

Chapter 6

Dynamic Solution

The equation of motion of the system takes the form of a second-order ordinary differential equation (ODE). Together with the initial positions $\mathbf{u}_0$ and velocities $\mathbf{v}_0$ at time t_0 , they form the second-order initial value problem

$$M\ddot{\mathbf{u}} + \mathbf{Q}(\mathbf{u}, \dot{\mathbf{u}}) = \mathbf{P}(t) \quad (6.1)$$

$$\mathbf{u}(t_0) = \mathbf{u}_0 \quad (6.2)$$

$$\dot{\mathbf{u}}(t_0) = \mathbf{v}_0 \quad (6.3)$$

whose solution is the time history $\mathbf{u}(t)$, $t > t_0$ of the displacements. Since the ODE (6.1) is nonlinear, there is no general analytical solution to this problem, and we once again have to employ numerical methods.

Numerical solution of ODEs is an extensive topic, and a large number of algorithms with different advantages and disadvantages is available. Most general algorithms require equation (6.1) to be transformed into an equivalent first-order system of the form $\dot{\mathbf{x}} = \mathbf{f}(\mathbf{x}, t)$, with a single state vector $\mathbf{x}(t)$ that describes the whole system state, i.e. the displacements and the velocities in our case. The common choice of $\mathbf{x}(t) = [\mathbf{u}(t), \dot{\mathbf{u}}(t)]^T$ leads to the first-order ODE

$$\underbrace{\frac{d}{dt} \begin{bmatrix} \mathbf{u}(t) \\ \dot{\mathbf{u}}(t) \end{bmatrix}}_{\mathbf{x}(t)} = \underbrace{\begin{bmatrix} \dot{\mathbf{u}}(t) \\ M^{-1}(\mathbf{P}(t) - \mathbf{Q}(\mathbf{u}, \dot{\mathbf{u}})) \end{bmatrix}}_{\mathbf{f}(\mathbf{x}, t)}. \quad (6.4)$$

Early prototypes of VirtualBow followed this approach and solved the resulting first-order problem with methods provided by the C++ library *Odeint* [1]. The error-controlled *Runge-Kutta methods* in particular worked reasonably well. However, in addition to such general methods for first-order systems, there are also numerous methods that have been developed with finite elements/structural dynamics in mind. They usually operate directly on the second-order ODE and tend to be more effective for this type of problem.

Besides the order of the equation, numerical methods for solving ODEs can also be divided into explicit and implicit ones. Explicit methods use the current state of the system at time t to calculate the next state at $t + \Delta t$. Implicit methods instead find the next state by iteratively solving an equation that involves the current as well as the next state. In the context of finite element analysis, implicit methods require equilibrium iterations at every timestep, involving the system's tangent stiffness and damping matrices, while explicit methods only require simple vector operations to compute the next solution state. Implicit methods therefore have a higher computational cost per timestep, but can make up for it by requiring fewer steps overall, since their better stability properties usually allow them to take much larger steps than explicit methods.

Explicit methods are easier to implement and are a good choice for short time scales, such as impact and wave propagation problems. They are most effective when the computation of the timesteps is fast, so they work well with constant, diagonal (lumped) mass matrices and simple elements with low-order shape functions. Implicit methods, in contrast, are well suited for larger time scales, where explicit algorithms would need an excessive number of steps. Examples are structural response problems and numerically stiff systems, i.e. systems whose natural frequencies range from very small to very large.

Based on these criteria, implicit methods should be the better fit for simulating the dynamics of a bow. However, from version 0.1 to 0.9, VirtualBow used the explicit *central difference method* [2], which is one of the standard explicit methods used in finite element analysis. The main reason for this choice was the relatively simple implementation. Starting with version 0.10, VirtualBow switched to the *Newmark- β method*, which is a very common implicit method in structural dynamics. The original plan was to use the *Generalized- α method*, which is closely related to the Newmark method and improves some of its properties. It was decided, however, to release VirtualBow with the slightly simpler Newmark method first to gather some experience and feedback and only switch to the Generalized- α method later if a clear advantage for the bow simulation can be shown.

6.1 Newmark- β Method

The continuous solution $\mathbf{u}(t)$ of the ODE is approximated by the discrete values $\mathbf{u}_i = \mathbf{u}(t_i)$, with the timesteps $\Delta t = t_{i+1} - t_i$ between the solution points. The core of the Newmark method [2] is a set of approximations that express the displacements $\mathbf{u}_{i+1}$ and velocities $\mathbf{v}_{i+1}$ at the next timestep in terms of the known state $\mathbf{u}_i$, $\mathbf{v}_i$, $\mathbf{a}_i$ at the previous timestep as well as the unknown acceleration $\mathbf{a}_{i+1}$:

$$\mathbf{u}_{i+1} = \mathbf{u}_i + \Delta t \mathbf{v}_i + \Delta t^2 ((1/2 - \beta) \mathbf{a}_i + \beta \mathbf{a}_{i+1}) \quad (6.5)$$

$$\mathbf{v}_{i+1} = \mathbf{v}_i + (1 - \gamma) \Delta t \mathbf{a}_i + \gamma \Delta t \mathbf{a}_{i+1} \quad (6.6)$$

The choice of the parameters β and γ determines how the acceleration is assumed to vary within the interval and leads to different variants of the scheme, e.g. $\beta = \frac{1}{4}$, $\gamma = \frac{1}{2}$ for the

constant average acceleration method or $\beta = \frac{1}{6}$, $\gamma = \frac{1}{2}$ for the *linear acceleration method*. With (6.5) and (6.6) there are already two equations for the three unknowns $\mathbf{u}_{i+1}$, $\mathbf{v}_{i+1}$ and $\mathbf{a}_{i+1}$ at the next timestep. The third equation is obtained by requiring that the system's equation of motion (6.1) is satisfied at the end of the interval,

$$\mathbf{M}\mathbf{a}_{i+1} + \mathbf{Q}(\mathbf{u}_{i+1}, \mathbf{v}_{i+1}) = \mathbf{P}(t_{i+1}). \quad (6.7)$$

This is what makes the method implicit, since an equation involving the current state as well as the yet unknown next state of the system has to be solved. Finding the accelerations $\mathbf{a}_{i+1}$ is then equivalent to finding the root of the residual forces

$$\mathbf{R}(\mathbf{a}_{i+1}) = \mathbf{M}\mathbf{a}_{i+1} + \mathbf{Q}(\mathbf{u}_{i+1}(\mathbf{a}_{i+1}), \mathbf{v}_{i+1}(\mathbf{a}_{i+1})) - \mathbf{P}(t_{i+1}) \quad (6.8)$$

which can be done with Newton-Raphson iterations very similar to those in the case of static equilibrium. The Jacobian matrix required for this can be calculated as

$$\frac{\partial \mathbf{R}}{\partial \mathbf{a}_{i+1}} = \mathbf{M} + \gamma \Delta t \mathbf{D}(\mathbf{u}_{i+1}, \mathbf{v}_{i+1}) + \beta \Delta t^2 \mathbf{K}(\mathbf{u}_{i+1}, \mathbf{v}_{i+1}) \quad (6.9)$$

and involves the tangent stiffness and damping matrices $\mathbf{K}$ and $\mathbf{D}$ of the system, together with other constant terms.

The Newmark method is at least second-order accurate for $\gamma = \frac{1}{2}$, which includes both the linear and constant acceleration methods. This means that the integration error decreases at least quadratically with the step size Δt . For linear systems it can be further shown that the constant acceleration method, i.e. $\beta = \frac{1}{4}$, is unconditionally stable. The same is not true for the linear acceleration method, i.e. $\beta = \frac{1}{6}$, where a critical timestep exists above which the solution becomes unstable, i.e. blows up exponentially over time. This critical timestep is, however, much larger than the critical timesteps typically encountered in explicit methods, so it is not a big practical concern. For nonlinear systems, like ours, there are no such mathematical stability guarantees either way. VirtualBow currently uses the constant average acceleration method, but there are no strong reasons behind this decision.

6.2 Adaptive Step Size

Selecting the step size Δt for time integration is a tradeoff between accuracy and computational effort. Smaller timesteps, up to a certain limit, lead to a more detailed solution with less numerical error, while larger timesteps require fewer steps overall and therefore reduce the computational cost. Ideally we would like to choose a timestep that is as large as possible while still maintaining a prescribed minimum degree of accuracy. However, this ideal timestep will generally vary over time. Sections where the solution is relatively smooth and uneventful allow for larger timesteps, while sections with frequent changes require smaller timesteps in order to accurately resolve all details. We are therefore looking for an algorithm that estimates the current accuracy of the solution and increases or decreases the timestep accordingly.

Many such automatic time-stepping algorithms have been proposed in the literature, both for general problems and in the context of structural dynamics. We can't possibly assess all of them here, but [25] gives a good overview of various methods for structural dynamics and also compares three of them in more detail by solving several example problems. Of the three methods investigated, the one by Bergan and Mollestad [3] seems to be the best established, is conceptually simple and didn't perform worse overall than the newer methods it was compared against. It is based on a so-called *current characteristic frequency* ω and an associated *current characteristic period* T , which are defined as

$$\omega^2 = \left| \frac{\Delta \mathbf{u}^\top \mathbf{K} \Delta \mathbf{u}}{\Delta \mathbf{u}^\top \mathbf{M} \Delta \mathbf{u}} \right|, \quad T = \frac{2\pi}{\omega} \quad (6.10)$$

with $\Delta \mathbf{u} = \mathbf{u}_i - \mathbf{u}_{i-1}$ being the displacement increment during the last timestep. The current characteristic frequency is based on *Rayleigh's quotient* and provides an approximation of the frequency that currently dominates the system's dynamic response. It is in general not equal to any of the natural frequencies of the system.

We now introduce a number N of desired timesteps per characteristic period. This value can be chosen to control how accurately the solution is resolved. A value of $N = 20$ is mentioned as typical in [25] with $N = 1000$ representing high accuracy and $N = 10$ low accuracy. From the current characteristic period and the desired timesteps per period we can calculate the estimated optimal timestep as

$$\Delta t^* = \frac{T}{N}. \quad (6.11)$$

In Figure 6.1, the current characteristic frequency and the resulting optimal timestep have been plotted for the dynamic simulation of a simple example bow. The actual timestep was kept constant for this simulation.

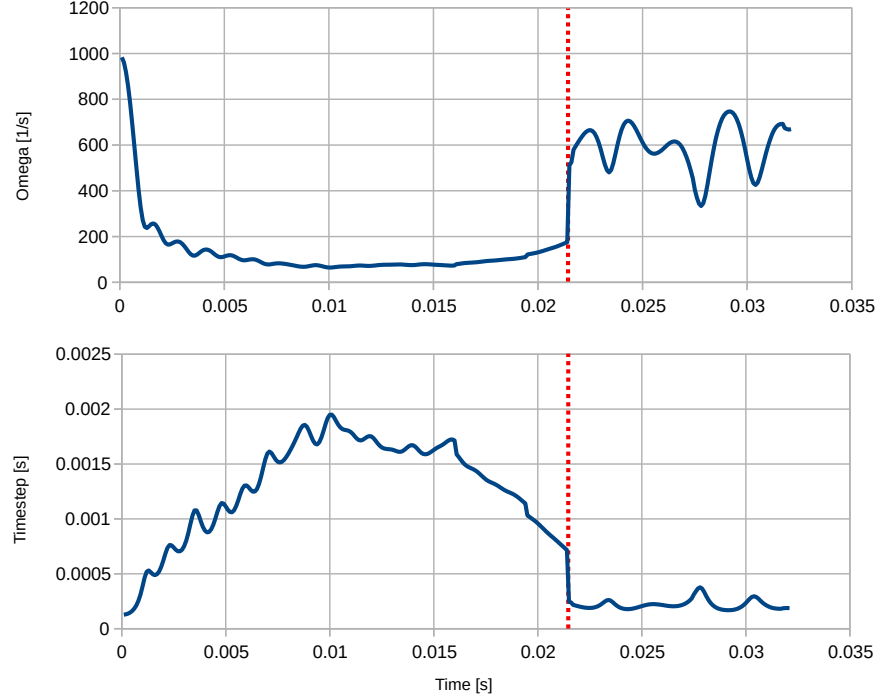


Figure 6.1: Example of the characteristic frequency ω and the associated timestep Δt^* for $N = 50$. Marked in red is the departure of the arrow.

It can be seen that the characteristic frequency varies greatly over the duration of the simulation, which confirms the usefulness of adaptive time stepping for dynamic bow simulations. The characteristic frequency is high at the start of the simulation, presumably because the sudden release of the string sets off a lot of initial oscillations in the string and the bow limbs. After this initial disturbance settles down, the motion of the bow turns into a more steady and continuous acceleration of the limbs and the arrow, represented by a relatively long and smooth section with a low characteristic frequency and larger optimal timesteps. The second spike in characteristic frequency occurs when the string reaches brace height and the arrow separates from the bow, which is not surprising.

Having verified the applicability of the Bergan and Mollestad method to our use case of simulating bows, we still have to define the actual algorithm by which the step size is to be adjusted. The easiest solution would be to always choose the estimated optimal step size Δt^* as the next step size,

$$\Delta t_{i+1} = \Delta t_{i+1}^*. \quad (6.12)$$

In the original method, however, the authors introduce a tuning function f as

$$\Delta t_{i+1} = f(\xi_i) \Delta t_i, \quad \xi_i = \frac{\Delta t_{i+1}^*}{\Delta t_i} \quad (6.13)$$

in order to moderate the increase or decrease in step size based on the ratio ξ_i between the optimal step size and the previously selected step size. Their proposed tuning function is defined as

$$f(\xi_i) = \begin{cases} 1, & \text{if } 1/\xi_p < \xi_i < \xi_p \\ 2, & \text{if } \xi_i > \xi_m \\ \xi_i, & \text{otherwise} \end{cases}. \quad (6.14)$$

Figure 6.2 shows the graph of this function. It evaluates to $f(\xi_i) = \xi_i$ over most of its domain, which means that the suggested timestep is applied, except for the following two modifications:

- A plateau around $\xi_i = 1$, controlled by the parameter ξ_p with a typical value given as 1.3 or 1.4. This plateau prevents the timestep from changing too frequently when the relative changes are small.
- A limitation of the increase in step size to a factor of ξ_m , typically chosen as 2. This prevents the timestep from prematurely increasing too much. The decrease in step size remains unlimited.

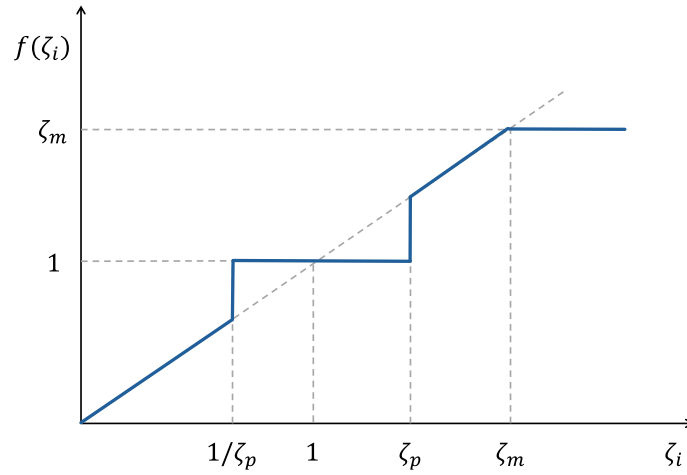


Figure 6.2: Tuning function $f(\xi_i)$ suggested by Bergan and Mollestad

The reasoning behind the plateau in the middle comes from linear dynamic analysis, where changing the timestep incurs an additional cost, because the system matrices have to be factorized once again. In nonlinear dynamic analysis, this is much less important, since the matrices have to be factorized at each timestep anyway due to the equilibrium iterations. Our implementation therefore does not include this plateau. The limitation of the relative step size increase has turned out to be essential, though. As another custom modification, we clamp the timestep resulting from the tuning function between a minimum value $\Delta t_{\min}$ and a maximum value $\Delta t_{\max}$. The minimum value ensures that the algorithm terminates in a reasonable amount of time and doesn't get stuck in a loop of ever-decreasing timesteps.

The maximum value ensures a certain minimum time resolution of the solution, even when the integration could have been done with larger timesteps. With these modifications, the actual update logic of the timestep used by VirtualBow is given by

$$f(\xi_i) = \min \left\{ \xi_i, 2 \right\}, \quad (6.15)$$

$$\Delta t_{i+1} = \text{clamp} \left\{ f(\xi_i) \cdot \Delta t_i, \Delta t_{\min}, \Delta t_{\max} \right\}. \quad (6.16)$$

The minimum timestep $\Delta t_{\min}$ also serves as the first timestep Δt_0 of the simulation, which is needed to start the adaptive scheme. The value chosen for the first timestep has to be much lower than the expected optimal timestep to ensure that it is too small rather than too large. An initial timestep that is too small can be increased quickly by the adaptive method in subsequent steps, while a timestep that is too large might introduce a permanent error. The method only ever adapts the next timestep on the basis of the estimated characteristic frequency from the previous timestep. It does not retroactively reject timesteps that were too large, as some more advanced methods do.

6.3 Stopping Criterion and Progress Estimation

The dynamic simulation in VirtualBow is carried out until some stopping criterion is satisfied. This criterion has to be chosen such that the simulated time interval contains all the information needed by users while also keeping it as short as possible. It has to include the separation of the arrow from the string plus some additional time, because some things, such as the maximum force on the string, tend to happen only after the arrow has left the bow. A related requirement for the stopping criterion is that it must allow the simulation progress to be estimated, so that it can be shown to the user.

The simplest solution would be to simulate until a fixed time is reached. This makes the progress estimation very easy, but it also has some drawbacks. It would require users to specify the simulation time, which can only be found by trial and error and might have to be adjusted for different bows. The current approach is therefore to simulate a time interval of $[0, \alpha T]$ where T is the (as yet unknown) time for the arrow to reach brace height and α is an arbitrary positive factor. The default value is $\alpha = 1.5$. The simulation progress at time t can now be formally written as

$$\text{progress} = \frac{t}{\alpha T} \quad (6.17)$$

i.e. the ratio of simulated time to the total amount of time. But only formally, because T is unknown as long as $t < T$, i.e. as long as the arrow has not yet actually reached brace height. For this first part of the simulation, T is estimated by using the current time t and position u of the arrow and extrapolating to the time when the arrow is projected to reach brace height.

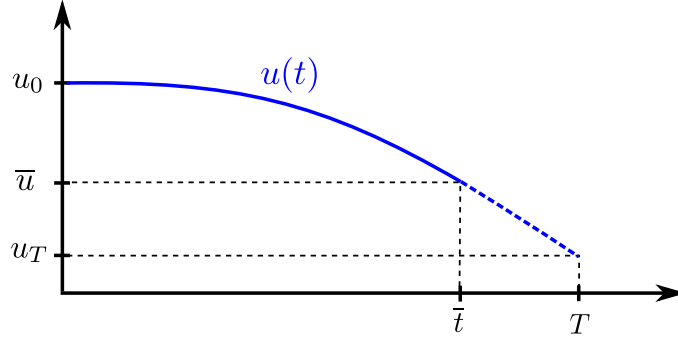


Figure 6.3: Arrow position $u(t)$ during the shot

The progression of the arrow position, denoted by $u(t)$, typically looks somewhat like the curve in Figure 6.3. It starts at time $t = 0$ at full draw u_0 with zero velocity, then increasingly accelerates and reaches brace height u_T at the time $t = T$ by definition. If the acceleration of the arrow is assumed to depend approximately linearly on the distance to brace height (linear force-draw curve), then $u(t)$ will resemble the cosine quarter-wave

$$u(t) \approx (u_0 - u_T) \cos(Ct) + u_T \quad (6.18)$$

where the constant C is yet to be determined. This is of course only a very crude approximation, but it is good enough for estimating the simulation progress. If we know the current time t and position u , we can rearrange equation (6.18) to get a current value for the constant C ,

$$C = \frac{1}{t} \arccos\left(\frac{u - u_T}{u_0 - u_T}\right). \quad (6.19)$$

With C known, the time T can be extrapolated from (6.18) using the condition $u(T) = u_T$,

$$T = \frac{\pi}{2C} = \frac{\pi}{2} t \cdot \arccos\left(\frac{u - u_T}{u_0 - u_T}\right)^{-1}. \quad (6.20)$$

Inserting this estimate for T into (6.17) results in the final expression for the estimated progress,

$$\text{progress} = \frac{2}{\alpha} \arccos\left(\frac{u - u_T}{u_0 - u_T}\right). \quad (6.21)$$

This estimate is used until the arrow has reached brace height. Then the actual value for T is known and used in (6.17) for the rest of the simulation. A numerical example of

the simulation progress over time is shown in Figure 6.4. The resulting curve is very close to a straight line here, which means that the progress tracks the simulation time closely, which is exactly what we want.

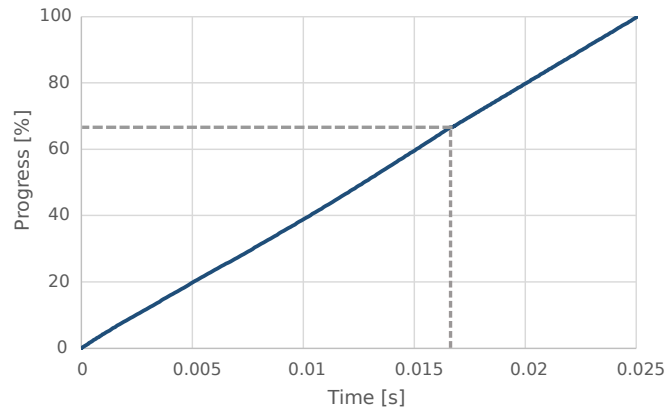


Figure 6.4: Example of the simulation progress over time. Marked in gray is the point where the brace height has been reached.

6.4 Stopping Event Detection and Location

As we saw, the dynamic simulation in VirtualBow is divided into two sequential phases. The first phase starts at full draw with the release of the string and ends when the arrow has reached the point where it separates from the string and starts flying freely. After this phase, the simulation model is modified by removing the arrow, which is modeled as a point mass, from the string. In the second phase the motion of the bow without the arrow is then simulated for an additional amount of time in order to capture what happens to the bow after the arrow has left. This time period is controlled by the “time span factor” in the simulation settings of the software. The two phases of the dynamic simulation place different requirements on the dynamic solver when it comes to stopping the simulation.

The second phase is the easier one in this regard. Here the time integration is to be stopped at a given time $t = t_{\text{end}}$. The only minor complication is that t_{end} does not necessarily line up with the actual time points t_i of the solution and might therefore not be reached exactly. This is, however, easily solved by truncating the last timestep accordingly.

The first phase is more complicated since it does not end at a known time. Instead, it ends when a certain event in the solution has occurred, namely the separation of the arrow. This event is associated with the current accelerations, since the arrow leaves the string as soon as it experiences a negative acceleration that is high enough to overcome the clamping force that holds it on the string. This event condition can be formulated as

$$e(\mathbf{a}) = \mathbf{a}[\text{arrow}] + \frac{F_{\text{clamp}}}{m_{\text{arrow}}} = 0 \quad (6.22)$$

with $e(\mathbf{a})$ being an event function that takes in the current state of the system (in our case only the accelerations are needed) and returns zero when the condition for the event is met. The first task is to detect the occurrence of this event while stepping through the numerical solution process. We assume that an event has occurred between times t_i and t_{i+1} if the event function changed sign,

$$\text{sgn} \{e(\mathbf{a}_i)\} \neq \text{sgn} \{e(\mathbf{a}_{i+1})\}. \quad (6.23)$$

Technically speaking, any odd number of events could have occurred during that timestep, not only one. Conversely, intervals with no sign change could technically contain any even number of events that go undetected with this approach. None of that is practically relevant for our bow simulation, though, since the timesteps are usually small enough to make multiple events very unlikely. The detection can be narrowed down further by looking only for positive or negative sign changes. In the case of arrow separation (6.22) for example, we are looking for a negative sign change (from positive to negative).

When an event that should stop the simulation is detected between t_i and t_{i+1} , we know that the last timestep was actually too large. If we were being sloppy, we could just stop the simulation at t_{i+1} , perform the arrow separation and accept the imperfection that the separation event actually occurred slightly earlier. This is exactly what VirtualBow did when still using an explicit solution method for the dynamic analysis. The much smaller timesteps of explicit methods, compared with implicit ones, made this possible without being too inaccurate. But by looking back at Figure 6.1 we can see that this approach would lead to large errors when using the Newmark method, since the timesteps can be far too large.

We therefore have to take the long route and do something more accurate. The overshooting time point t_{i+1} is therefore discarded and we restart at t_i , this time trying to find a new timestep such that the event occurs exactly at the end of the interval. This is done by treating the timestep as an additional unknown in the Newmark iterations. The equilibrium equation (6.8) for finding the accelerations at the end of the time interval therefore becomes

$$\mathbf{R}(\mathbf{a}_{i+1}, \Delta t) = \mathbf{0}. \quad (6.24)$$

Introducing the additional unknown Δt requires us to add an equation as well. This additional equation or constraint is given by the requirement that the event takes place at t_{i+1} , which means that the event function (6.22) must be zero there,

$$e(\mathbf{a}_{i+1}) = 0. \quad (6.25)$$

So the final task is to solve equation (6.24) with the scalar constraint (6.25), which yields both the accelerations $\mathbf{a}_{i+1}$ and the correct timestep Δt . If this problem seems familiar, it's because it is mathematically equivalent to the problem of displacement control in static analysis, where the static equilibrium equations are solved under the constraint that one of the displacements must match a prescribed value. We can therefore apply the same numerical method here, namely the constrained Newton-Raphson method, which was outlined in Section 5.2.

The partial derivative of the residual forces with respect to the step size, which is required for this, is obtained as

$$\begin{aligned} \frac{\partial \mathbf{R}}{\partial \Delta t} = \frac{\partial \mathbf{Q}_{i+1}}{\partial \mathbf{u}_{i+1}} \frac{\partial \mathbf{u}_{i+1}}{\partial \Delta t} + \frac{\partial \mathbf{Q}_{i+1}}{\partial \mathbf{v}_{i+1}} \frac{\partial \mathbf{v}_{i+1}}{\partial \Delta t} = \mathbf{K}_{i+1} (\mathbf{v}_i + 2\Delta t ((1/2 - \beta) \mathbf{a}_i + \beta \mathbf{a}_{i+1})) \\ + \mathbf{D}_{i+1} ((1 - \gamma) \mathbf{a}_i + \gamma \mathbf{a}_{i+1}) \end{aligned} \quad (6.26)$$

and the partial derivatives of the event function are

$$\frac{\partial e}{\partial \mathbf{a}_{i+1}} = \mathbf{e}_{\text{arrow}}, \quad \frac{\partial e}{\partial \Delta t} = 0 \quad (6.27)$$

where $\mathbf{e}_{\text{arrow}}$ is the unit vector in the direction that represents the arrow's motion.

Chapter 7

Modal Analysis

The goal of a modal analysis is to obtain the natural frequencies and mode shapes of a linear system. Natural frequencies are used in VirtualBow for determining the damping that is applied to the bow limb (see Section 4.2.6) and also in some of the verification tests as a quick way to check the dynamic properties of a system. Since modal analysis is inherently tied to the dynamics of linear systems, we have to linearize our nonlinear system around a certain setpoint before performing one. The results are only meaningful for small displacements relative to that setpoint.

7.1 Generalized Eigenvalue Problem

The starting point is the linearization of the equations of motion (3.4) for the case of free vibration,

$$M\Delta\ddot{\mathbf{u}} + D\Delta\dot{\mathbf{u}} + K\Delta\mathbf{u} = \mathbf{0} \quad (7.1)$$

with the displacement vector $\Delta\mathbf{u}$ being measured relative to the setpoint at which the system has been linearized. Since this is a linear, homogeneous differential equation, we can assume its solutions to take the form $\mathbf{u}(t) = \hat{\mathbf{u}} e^{\lambda t}$ with $\hat{\mathbf{u}} \in \mathbb{R}^n$ and $\lambda \in \mathbb{C}$. Substituting this into the equation gives us the condition

$$(\lambda^2 M + \lambda D + K) \hat{\mathbf{u}} = \mathbf{0}. \quad (7.2)$$

This is a *quadratic eigenvalue problem* [7], whose solutions are the $2n$ eigenvalues λ_i and eigenvectors $\hat{\mathbf{u}}_i$. In order to compute a solution, it can be helpful to transform (7.2) into the more common linear generalized eigenvalue problem

$$A\mathbf{v} = \lambda B\mathbf{v} \quad (7.3)$$

using the following auxiliary matrices (see approach II from [7])

$$\mathbf{A} = \begin{bmatrix} \mathbf{0} & \mathbf{K} \\ \mathbf{K} & \mathbf{D} \end{bmatrix}, \quad \mathbf{B} = \begin{bmatrix} \mathbf{K} & \mathbf{0} \\ \mathbf{0} & -\mathbf{M} \end{bmatrix}, \quad \mathbf{v} = \begin{bmatrix} \mathbf{u} \\ \dot{\mathbf{u}} \end{bmatrix}. \quad (7.4)$$

This can now be solved with standard methods, which we don't have to implement ourselves. Note that $\mathbf{A}$ and $\mathbf{B}$ are symmetric but not positive definite. The symmetry can be exploited when looking for an efficient algorithm.

7.2 Evaluating Modal Properties

For an underdamped system, which is the case if $4\mathbf{K} - \mathbf{D}^2$ is positive definite, the eigenvalues come in complex conjugate pairs of the form [21]

$$\lambda_{i,i+1} = -\zeta_i \omega_i \pm j \omega_i \sqrt{1 - \zeta_i^2} \quad (7.5)$$

where ω_i is the i -th undamped natural frequency and ζ_i is the i -th *modal damping ratio*. When given the eigenvalue $\lambda_i = a_i + jb_i$ with $a_i = \text{Re}(\lambda_i)$ and $b_i = \text{Im}(\lambda_i)$, these can be calculated as

$$\omega_i = \sqrt{a_i^2 + b_i^2}, \quad \zeta_i = -\frac{a_i}{\sqrt{a_i^2 + b_i^2}}. \quad (7.6)$$

Appendix A

Limb Geometry

A.1 Profile Curve

The profile curve in VirtualBow is composed of one or more segments, where each segment can have a different type, like line, arc, spiral, etc. What follows is a geometrical description of each segment type.

A.1.1 Spiral Segment

This spiral segment is a *Clothoid*, also called an *Euler Spiral*, which is a curve whose curvature varies linearly over its length. It can be used as a smooth transition between two segments of different curvature. For a curve segment of length l with a starting curvature of κ_0 and an end curvature of κ_1 the linear curvature distribution is

$$\kappa(s) = \kappa_0 + \frac{s}{l} (\kappa_1 - \kappa_0). \quad (\text{A.1})$$

Integrating this curvature over the arc length leads to the tangent angle φ as

$$\begin{aligned} \varphi(s) &= \varphi_0 + \int_0^s \kappa(\tau) d\tau = \varphi_0 + \frac{s^2}{2l} (\kappa_1 - \kappa_0) \\ &= as^2 + bs + c, \end{aligned} \quad (\text{A.2})$$

which we write in terms of the quadratic coefficients a , b and c . The positions $x(s)$ and $y(s)$ are then given by the integrals

$$x(s) = x_0 + \int_0^s \cos(\varphi(\tau)) d\tau, \quad (\text{A.3})$$

$$y(s) = y_0 + \int_0^s \sin(\varphi(\tau)) d\tau. \quad (\text{A.4})$$

These have different solutions depending on the quadratic coefficients. For $a > 0$ we get

$$x(s) = x_0 + \sqrt{\frac{\pi}{2a}} \left(\cos \left(\frac{b^2}{4a} - c \right) (C_s - C_0) + \sin \left(\frac{b^2}{4a} - c \right) (S_s - S_0) \right) \quad (\text{A.5})$$

$$y(s) = y_0 + \sqrt{\frac{\pi}{2a}} \left(\sin \left(\frac{b^2}{4a} - c \right) (C_0 - C_s) + \cos \left(\frac{b^2}{4a} - c \right) (S_s - S_0) \right) \quad (\text{A.6})$$

with the intermediate expressions

$$S_s = S \left(\frac{b + 2as}{\sqrt{2\pi a}} \right) \quad S_0 = S \left(\frac{b}{\sqrt{2\pi a}} \right) \quad (\text{A.7})$$

$$C_s = C \left(\frac{b + 2as}{\sqrt{2\pi a}} \right) \quad C_0 = C \left(\frac{b}{\sqrt{2\pi a}} \right) \quad (\text{A.8})$$

defined by the *Fresnel integrals* $S(x)$ and $C(x)$. For $a < 0$ we get something very similar, but with a few switched signs,

$$x(s) = x_0 - \sqrt{-\frac{\pi}{2a}} \left(\cos \left(c - \frac{b^2}{4a} \right) (C_s - C_0) + \sin \left(c - \frac{b^2}{4a} \right) (S_s - S_0) \right) \quad (\text{A.9})$$

$$y(s) = y_0 + \sqrt{-\frac{\pi}{2a}} \left(\sin \left(c - \frac{b^2}{4a} \right) (C_0 - C_s) + \cos \left(c - \frac{b^2}{4a} \right) (S_s - S_0) \right) \quad (\text{A.10})$$

$$S_s = S \left(\frac{b + 2as}{\sqrt{-2\pi a}} \right) \quad S_0 = S \left(\frac{b}{\sqrt{-2\pi a}} \right) \quad (\text{A.11})$$

$$C_s = C \left(\frac{b + 2as}{\sqrt{-2\pi a}} \right) \quad C_0 = C \left(\frac{b}{\sqrt{-2\pi a}} \right) \quad (\text{A.12})$$

A.1.2 Arc Segment

An arc segment, which is a segment with a constant nonzero curvature, follows from the same derivation as the clothoid segment, but for the special case $a = 0$ and $b \neq 0$. This results in the curve

$$x(s) = x_0 + \frac{1}{b} \left(\sin(bs + c) - \sin(c) \right) \quad (\text{A.13})$$

$$y(s) = y_0 + \frac{1}{b} \left(-\cos(bs + c) + \cos(c) \right) \quad (\text{A.14})$$

A.1.3 Line Segment

Similarly, a line segment with zero curvature follows as the special case $a = 0$ and $b = 0$. The resulting curve is

$$x(s) = x_0 + s \cdot \cos(c), \quad (\text{A.15})$$

$$y(s) = y_0 + s \cdot \sin(c). \quad (\text{A.16})$$

A.1.4 Spline Segment

The spline segment interpolates a number of user-defined control points. It does this by constructing two cubic splines $x(t)$ and $y(t)$, one for each curve component. The original spline parameter t does not correspond to the arc length of the curve, though. In order to get an arc length parameterization, the arc length function $s(t)$ is approximated numerically and inverted as $t(s)$. This inverse arc length function is then used to parameterize the spline as $x(s) = x(t(s))$ and $y(s) = y(t(s))$.

More about splines in the following sections on width and thickness.

A.2 Width and Thickness

The width and thickness of the limb and its layers are described by monotonic cubic splines that interpolate user-supplied control points.

A.2.1 Cubic C^2 Spline

Consider n points $(x_0, y_0) \dots (x_{n-1}, y_{n-1})$ with increasing values of x . We define the differences $\Delta x_i = x_{i+1} - x_i$ and $\Delta y_i = y_{i+1} - y_i$ as well as the slope $\delta_i = \Delta y_i / \Delta x_i$ of the secant line between two successive points.

For each interval $x \in [x_i, x_{i+1}]$ the data is interpolated by the cubic polynomial

$$f_i(x) = h_{00}(t) y_i + h_{10}(t) \Delta x_i m_i + h_{01}(t) y_{i+1} + h_{11}(t) \Delta x_i m_{i+1} \quad (\text{A.17})$$

$$f'_i(x) = \frac{h'_{00}(t)}{\Delta x_i} y_i + h'_{10}(t) m_i + \frac{h'_{01}(t)}{\Delta x_i} y_{i+1} + h'_{11}(t) m_{i+1} \quad (\text{A.18})$$

$$f''_i(x) = \frac{h''_{00}(t)}{\Delta x_i^2} y_i + \frac{h''_{10}(t)}{\Delta x_i} m_i + \frac{h''_{01}(t)}{\Delta x_i^2} y_{i+1} + \frac{h''_{11}(t)}{\Delta x_i} m_{i+1} \quad (\text{A.19})$$

where y_i , y_{i+1} and m_i , m_{i+1} are the values and slopes at the interval boundaries, respectively, and $h_{00}(t) \dots h_{11}(t)$ are the Hermite basis functions, defined as

$$\begin{aligned} h_{00}(t) &= 2t^3 - 3t^2 + 1 & h'_{00}(t) &= 6t^2 - 6t & h''_{00}(t) &= 12t - 6 \\ h_{10}(t) &= t^3 - 2t^2 + t & h'_{10}(t) &= 3t^2 - 4t + 1 & h''_{10}(t) &= 6t - 4 \\ h_{01}(t) &= -2t^3 + 3t^2 & h'_{01}(t) &= -6t^2 + 6t & h''_{01}(t) &= -12t + 6 \\ h_{11}(t) &= t^3 - t^2 & h'_{11}(t) &= 3t^2 - 2t & h''_{11}(t) &= 6t - 2 \end{aligned} \quad (\text{A.20})$$

with $t = (x - x_i)/(x_{i+1} - x_i)$. By definition, the resulting spline already interpolates the given function values and is C^1 continuous since adjacent segments share the same value and slope at the interval boundaries. To determine the n unknown slopes m_i we require the second derivatives at the connection points to match as well, making the curve C^2 continuous. This gives us $n - 2$ continuity conditions, leaving two more conditions to make the problem uniquely determined. Those additional conditions are obtained from boundary considerations, e.g. by specifying the spline's first and/or second derivatives at the left and right endpoints.

Continuity conditions $f''_{i-1}(x_i) = f''_i(x_i)$ for $i = 1 \dots n - 2$:

$$\begin{aligned} \frac{h''_{00}(1)}{\Delta x_{i-1}^2} y_{i-1} + \frac{h''_{10}(1)}{\Delta x_{i-1}} m_{i-1} + \frac{h''_{01}(1)}{\Delta x_{i-1}^2} y_i + \frac{h''_{11}(1)}{\Delta x_{i-1}} m_i &= \frac{h''_{00}(0)}{\Delta x_i^2} y_i + \frac{h''_{10}(0)}{\Delta x_i} m_i + \frac{h''_{01}(0)}{\Delta x_i^2} y_{i+1} + \frac{h''_{11}(0)}{\Delta x_i} m_{i+1} \\ \frac{-2}{\Delta x_i} m_{i+1} + \left(\frac{-4}{\Delta x_i} - \frac{4}{\Delta x_{i-1}} \right) m_i - \frac{2}{\Delta x_{i-1}} m_{i-1} &= \frac{6 y_{i-1} - 6 y_i}{\Delta x_{i-1}^2} - \frac{-6 y_i + 6 y_{i+1}}{\Delta x_i^2} \\ \frac{1}{\Delta x_i} m_{i+1} + 2 \left(\frac{1}{\Delta x_i} + \frac{1}{\Delta x_{i-1}} \right) m_i + \frac{1}{\Delta x_{i-1}} m_{i-1} &= 3 \left(\frac{\Delta y_i}{\Delta x_i^2} + \frac{\Delta y_{i-1}}{\Delta x_{i-1}^2} \right) \\ \Delta x_{i-1} m_{i+1} + 2 (\Delta x_i + \Delta x_{i-1}) m_i + \Delta x_i m_{i-1} &= 3 (\delta_i \Delta x_{i-1} + \delta_{i-1} \Delta x_i) \end{aligned} \quad (\text{A.21})$$

Boundary condition $f''_0(0) = y''_0$:

$$\begin{aligned} \frac{h''_{00}(0)}{\Delta x_0^2} y_0 + \frac{h''_{10}(0)}{\Delta x_0} m_0 + \frac{h''_{01}(0)}{\Delta x_0^2} y_1 + \frac{h''_{11}(0)}{\Delta x_0} m_1 &= y''_0 \\ \frac{-6}{\Delta x_0^2} y_0 + \frac{-4}{\Delta x_0} m_0 + \frac{6}{\Delta x_0^2} y_1 + \frac{-2}{\Delta x_0} m_1 &= y''_0 \\ 4 m_0 + 2 m_1 &= 6 \delta_0 - \Delta x_0 y''_0 \end{aligned} \quad (\text{A.22})$$

Boundary condition $f''_{n-2}(1) = y''_{n-1}$:

$$\begin{aligned} \frac{h''_{00}(1)}{\Delta x_{n-2}^2} y_{n-2} + \frac{h''_{10}(1)}{\Delta x_{n-2}} m_{n-2} + \frac{h''_{01}(1)}{\Delta x_{n-2}^2} y_{n-1} + \frac{h''_{11}(1)}{\Delta x_{n-2}} m_{n-1} &= y''_{n-1} \\ \frac{6}{\Delta x_{n-2}^2} y_{n-2} + \frac{2}{\Delta x_{n-2}} m_{n-2} + \frac{-6}{\Delta x_{n-2}^2} y_{n-1} + \frac{4}{\Delta x_{n-2}} m_{n-1} &= y''_{n-1} \\ 2 m_{n-2} + 4 m_{n-1} &= 6 \delta_{n-2} + \Delta x_{n-2} y''_{n-1} \end{aligned} \quad (\text{A.23})$$

Boundary condition $f'_0(0) = y'_0$:

$$m_0 = y'_0 \quad (\text{A.24})$$

Boundary condition $f'_{n-1}(1) = y'_{n-1}$:

$$m_{n-1} = y'_{n-1} \quad (\text{A.25})$$

Combining the continuity and boundary conditions gives us the system of equations

$$\begin{bmatrix} a_0 & b_0 & & & \\ & \ddots & \ddots & & \\ & & \Delta x_{i-1} & 2(\Delta x_i + \Delta x_{i-1}) & \Delta x_i \\ & & & \ddots & \ddots \\ & & & & a_{n-1} & b_{n-1} \end{bmatrix} \begin{bmatrix} m_0 \\ \vdots \\ m_i \\ \vdots \\ m_{n-1} \end{bmatrix} = \begin{bmatrix} c_0 \\ \vdots \\ 3(\delta_i \Delta x_{i-1} + \delta_{i-1} \Delta x_i) \\ \vdots \\ c_{n-1} \end{bmatrix} \quad (\text{A.26})$$

where a , b and c depend on the choice of boundary conditions. Since the coefficient matrix has a tridiagonal structure, it can be solved very efficiently by using the *Thomas algorithm*.

A.2.2 Monotonicity

Sometimes it is desirable for an interpolating curve to preserve monotonicity of the input data, i.e. not to introduce any local minima or maxima between the given data points. In VirtualBow this is used for the width and layer heights, to ensure that those quantities are always positive if the user entered positive values.

One popular method for monotonic cubic interpolation is the *Fritsch-Carlson method* [10], which operates on a cubic Hermite spline like the one we defined above and adjusts the slopes according to some simple rules in order to make the spline monotonic within each interval. The only difference is that Fritsch and Carlson use finite differences to determine the initial slopes for their spline, whereas we use the condition of C^2 continuity.

Once the initial slopes have been computed, the first necessary condition for monotonicity in an interval is

$$\text{sgn}(m_i) = \text{sgn}(m_{i+1}) = \text{sgn}(\delta_i) \quad (\text{A.27})$$

which means that the slopes m_i and m_{i+1} must have the same direction as the slope δ_i of the secant. Any slopes that violate this condition are set to zero. With the slopes modified in this way, define $\alpha_i = m_i/\delta_i$ and $\beta_i = m_{i+1}/\delta_i$. A sufficient condition for the segment to be monotone is

$$\alpha_i^2 + \beta_i^2 \leq 9 \quad (\text{A.28})$$

i.e. the curve must be restricted to a circle of radius 3 in the α - β plane. If this condition is violated in any segment, define the factor

$$\tau_i = \frac{3}{\sqrt{\alpha_i^2 + \beta_i^2}} \quad (\text{A.29})$$

and rescale the slopes as

$$m_i = \tau_i \alpha_i \delta_i, \quad m_{i+1} = \tau_i \beta_i \delta_i \quad (\text{A.30})$$

After these modifications the spline will be monotonic within each interval, but may no longer be C^2 continuous. Also, the chosen boundary conditions may no longer be fulfilled.

Appendix B

Element Details

B.1 Derivatives for the Beam Element

Tangent stiffness and damping matrices:

$$\begin{aligned}\frac{\partial Q}{\partial u_i} &= \frac{\partial \mathbf{J}^\top}{\partial u_i} \left(\mathbf{K}_l \mathbf{u}_l + \mathbf{D}_l \dot{\mathbf{u}}_l \right) + \mathbf{J}^\top \left(\mathbf{K}_l \frac{\partial \mathbf{u}_l}{\partial u_i} + \mathbf{D}_l \frac{\partial \mathbf{J}}{\partial u_i} \dot{\mathbf{u}} \right) \\ &= \frac{\partial \mathbf{J}^\top}{\partial u_i} \left(\mathbf{K}_l \mathbf{u}_l + \mathbf{D}_l \dot{\mathbf{u}}_l \right) + \mathbf{J}^\top \mathbf{D}_l \frac{\partial \mathbf{J}}{\partial u_i} \dot{\mathbf{u}} + \mathbf{J}^\top \mathbf{K}_l \mathbf{J}_i\end{aligned}\quad (\text{B.1})$$

$$\begin{aligned}\frac{\partial Q}{\partial \mathbf{u}} &= \left[\dots, \frac{\partial Q}{\partial u_i}, \dots \right]_{6 \times 6} \\ &= \left[\frac{\partial \mathbf{J}^\top}{\partial u_i} \left(\mathbf{K}_l \mathbf{u}_l + \mathbf{D}_l \dot{\mathbf{u}}_l \right) \right]_{6 \times 6} + \mathbf{J}^\top \mathbf{D}_l \left[\frac{\partial \mathbf{J}}{\partial u_i} \dot{\mathbf{u}} \right]_{3 \times 6} + \mathbf{J}^\top \mathbf{K}_l \mathbf{J}\end{aligned}\quad (\text{B.2})$$

$$\frac{\partial Q}{\partial \dot{\mathbf{u}}} = \mathbf{J}^\top \mathbf{D}_l \mathbf{J} \quad (\text{B.3})$$

Further derivatives of the Jacobian:

$$\frac{\partial \mathbf{J}}{\partial x_0} = \begin{bmatrix} -c_0 & -c_2 & 0 & c_0 & c_2 & 0 \\ -c_5 & c_3 & 0 & c_5 & -c_3 & 0 \\ -c_5 & c_3 & 0 & c_5 & -c_3 & 0 \end{bmatrix} \quad \frac{\partial \mathbf{J}}{\partial x_1} = -\frac{\partial \mathbf{J}}{\partial x_0} \quad (\text{B.4})$$

$$\frac{\partial \mathbf{J}}{\partial y_0} = \begin{bmatrix} -c_2 & -c_1 & 0 & c_2 & c_1 & 0 \\ -c_4 & c_5 & 0 & c_4 & -c_5 & 0 \\ -c_4 & c_5 & 0 & c_4 & -c_5 & 0 \end{bmatrix} \quad \frac{\partial \mathbf{J}}{\partial y_1} = -\frac{\partial \mathbf{J}}{\partial y_0} \quad (\text{B.5})$$

$$\frac{\partial \mathbf{J}}{\partial \varphi_0} = \mathbf{0} \quad \frac{\partial \mathbf{J}}{\partial \varphi_1} = \mathbf{0} \quad (\text{B.6})$$

The following abbreviations have been used here:

$$c_0 = \frac{\Delta x^2}{l^3} - \frac{1}{l} \quad c_1 = \frac{\Delta y^2}{l^3} - \frac{1}{l} \quad c_2 = \frac{\Delta y \Delta y}{l^3} \quad (\text{B.7})$$

$$c_3 = 2 \frac{\Delta x^2}{l^4} - \frac{1}{l^2} \quad c_4 = 2 \frac{\Delta y^2}{l^4} - \frac{1}{l^2} \quad c_5 = 2 \frac{\Delta y \Delta y}{l^4} \quad (\text{B.8})$$

B.2 Beam Stiffness Matrix by FEM

As an alternative to the analytical solution by integration, the stiffness matrix of an arbitrarily shaped beam segment can be computed by approximating it with a number of straight beam elements of constant cross-section. First of all, divide the beam segment into n elements with index $i = 0 \dots n - 1$. The length l_i and orientation angle α_i of each element are then given by

$$l_i = |r_{i+1} - r_i|, \quad (\text{B.9})$$

$$\alpha_i = \arctan2\left(\frac{y_{i+1} - y_i}{x_{i+1} - x_i}\right). \quad (\text{B.10})$$

The stiffness matrix of a straight Timoshenko beam element with a constant section stiffness $\mathbf{C} = \text{diag}(EA, EI, GA_s)$ with longitudinal stiffness EA , bending stiffness EI and shear stiffness GA_s is well known in structural mechanics, see for example [11], and reads

$$\bar{\mathbf{K}}_i = \begin{bmatrix} \frac{EA_i}{l_i} & 0 & 0 & -\frac{EA_i}{l_i} & 0 & 0 \\ 0 & \frac{12EI_i}{l_i^3(1+\Phi)} & \frac{6EI_i}{l_i^2(1+\Phi)} & 0 & -\frac{12EI_i}{l_i^3(1+\Phi)} & \frac{6EI_i}{l_i^2(1+\Phi)} \\ 0 & \frac{6EI_i}{l_i^2(1+\Phi)} & \frac{EI_i(4+\Phi)}{l_i(1+\Phi)} & 0 & -\frac{6EI_i}{l_i^2(1+\Phi)} & \frac{EI_i(2-\Phi)}{l_i(1+\Phi)} \\ -\frac{EA_i}{l_i} & 0 & 0 & \frac{EA_i}{l_i} & 0 & 0 \\ 0 & -\frac{12EI_i}{l_i^3(1+\Phi)} & -\frac{6EI_i}{l_i^2(1+\Phi)} & 0 & \frac{12EI_i}{l_i^3(1+\Phi)} & -\frac{6EI_i}{l_i^2(1+\Phi)} \\ 0 & \frac{6EI_i}{l_i^2(1+\Phi)} & \frac{EI_i(2-\Phi)}{l_i(1+\Phi)} & 0 & -\frac{6EI_i}{l_i^2(1+\Phi)} & \frac{EI_i(4+\Phi)}{l_i(1+\Phi)} \end{bmatrix} \quad (\text{B.11})$$

with the shear parameter $\Phi = \frac{12EI}{GA_s l^2}$. These local stiffness matrices are transformed by the transformation matrix

$$\mathbf{T}_i = \begin{bmatrix} \cos(\alpha_i) & \sin(\alpha_i) & 0 & 0 & 0 & 0 \\ -\sin(\alpha_i) & \cos(\alpha_i) & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & \cos(\alpha_i) & \sin(\alpha_i) & 0 \\ 0 & 0 & 0 & -\sin(\alpha_i) & \cos(\alpha_i) & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix} \quad (\text{B.12})$$

in order to account for the orientation angle α of each element. This leads to the element stiffness matrices

$$\mathbf{K}_i = \mathbf{T}_i^\top \overline{\mathbf{K}}_i \mathbf{T} = \begin{bmatrix} \mathbf{K}_{00}^{(i)} & \mathbf{K}_{01}^{(i)} \\ \mathbf{K}_{10}^{(i)} & \mathbf{K}_{11}^{(i)} \end{bmatrix} \quad (\text{B.13})$$

which we partitioned into four equal blocks. Due to the interconnection of the elements, where the end node of one element is the starting node of the next one, the full stiffness matrix of the assembly is given by the following matrix of overlapping blocks,

$$\mathbf{K}_{\text{full}} = \begin{bmatrix} \ddots & & & & & \\ & \ddots & & & & \\ & & \mathbf{K}_{11}^{(i-1)} + \mathbf{K}_{00}^{(i)} & \mathbf{K}_{01}^{(i)} & & \\ & & \mathbf{K}_{10}^{(i)} & \mathbf{K}_{11}^{(i)} + \mathbf{K}_{00}^{(i+1)} & \mathbf{K}_{01}^{(i+1)} & \\ & & & \mathbf{K}_{10}^{(i+1)} & \mathbf{K}_{11}^{(i+1)} + \mathbf{K}_{00}^{(i+2)} & \\ & & & & \ddots & \ddots \end{bmatrix}. \quad (\text{B.14})$$

This matrix has the dimension $3(n+1) \times 3(n+1)$ and relates the forces and displacements of all nodes to each other. We are, however, only interested in the 6×6 stiffness matrix with respect to the first and last node of the assembly. This can be achieved by something called a *static reduction* or *Guyan reduction*.

First of all, the full stiffness relation is partitioned as follows,

$$\begin{bmatrix} \mathbf{F}_1 \\ \mathbf{0} \\ \mathbf{F}_3 \end{bmatrix} = \underbrace{\begin{bmatrix} \mathbf{K}_{11} & \mathbf{K}_{12} & \mathbf{K}_{13} \\ \mathbf{K}_{12}^\top & \mathbf{K}_{22} & \mathbf{K}_{23} \\ \mathbf{K}_{13}^\top & \mathbf{K}_{23}^\top & \mathbf{K}_{33} \end{bmatrix}}_{\mathbf{K}_{\text{full}}} \begin{bmatrix} \mathbf{u}_1 \\ \mathbf{u}_2 \\ \mathbf{u}_3 \end{bmatrix} \quad (\text{B.15})$$

where $\mathbf{u}_1 \in \mathbb{R}^3$ are the displacements of the first node, $\mathbf{u}_3 \in \mathbb{R}^3$ are the displacements of the last node and $\mathbf{u}_2 \in \mathbb{R}^{3n-1}$ are the displacements of all the intermediate nodes. Only the first and last nodes have a corresponding applied force $\mathbf{F}_1$ and $\mathbf{F}_3$ while the intermediate nodes are free of external forces.

From the second block equation we can calculate the equilibrium displacements $\mathbf{u}_2$ of the intermediate nodes in terms of the other displacements,

$$\begin{aligned} 0 &= \mathbf{K}_{12}^\top \mathbf{u}_1 + \mathbf{K}_{22} \mathbf{u}_2 + \mathbf{K}_{23} \mathbf{u}_3 \\ \Rightarrow \mathbf{u}_2 &= -\mathbf{K}_{22}^{-1} (\mathbf{K}_{12}^\top \mathbf{u}_1 + \mathbf{K}_{23} \mathbf{u}_3). \end{aligned} \quad (\text{B.16})$$

Using this, the intermediate displacements can be eliminated from the first and third block

equations,

$$\begin{aligned}
\mathbf{F}_1 &= \mathbf{K}_{11}\mathbf{u}_1 + \mathbf{K}_{11}\mathbf{u}_2 + \mathbf{K}_{11}\mathbf{u}_3 \\
&= \mathbf{K}_{11}\mathbf{u}_1 - \mathbf{K}_{11}\mathbf{K}_{22}^{-1}(\mathbf{K}_{11}^\top\mathbf{u}_1 + \mathbf{K}_{11}\mathbf{u}_3) + \mathbf{K}_{11}\mathbf{u}_3 \\
&= (\mathbf{K}_{11} - \mathbf{K}_{11}\mathbf{K}_{22}^{-1}\mathbf{K}_{11}^\top)\mathbf{u}_1 + (\mathbf{K}_{11} - \mathbf{K}_{11}\mathbf{K}_{22}^{-1}\mathbf{K}_{11})\mathbf{u}_3, \tag{B.17}
\end{aligned}$$

$$\begin{aligned}
\mathbf{F}_3 &= \mathbf{K}_{11}^\top\mathbf{u}_1 + \mathbf{K}_{11}^\top\mathbf{u}_2 + \mathbf{K}_{33}\mathbf{u}_3 \\
&= \mathbf{K}_{11}^\top\mathbf{u}_1 - \mathbf{K}_{11}^\top\mathbf{K}_{22}^{-1}(\mathbf{K}_{11}^\top\mathbf{u}_1 + \mathbf{K}_{11}\mathbf{u}_3) + \mathbf{K}_{33}\mathbf{u}_3 \\
&= (\mathbf{K}_{11}^\top - \mathbf{K}_{11}^\top\mathbf{K}_{22}^{-1}\mathbf{K}_{11}^\top)\mathbf{u}_1 + (\mathbf{K}_{33} - \mathbf{K}_{11}^\top\mathbf{K}_{22}^{-1}\mathbf{K}_{11})\mathbf{u}_3. \tag{B.18}
\end{aligned}$$

Finally, rearranging the forces $\mathbf{F}_1$ and $\mathbf{F}_3$ and the corresponding displacements back into the matrix form

$$\begin{bmatrix} \mathbf{F}_1 \\ \mathbf{F}_3 \end{bmatrix} = \underbrace{\begin{bmatrix} \mathbf{K}_{11} - \mathbf{K}_{12}\mathbf{K}_{22}^{-1}\mathbf{K}_{12}^\top & \mathbf{K}_{13} - \mathbf{K}_{12}\mathbf{K}_{22}^{-1}\mathbf{K}_{23} \\ \mathbf{K}_{13}^\top - \mathbf{K}_{23}^\top\mathbf{K}_{22}^{-1}\mathbf{K}_{12}^\top & \mathbf{K}_{33} - \mathbf{K}_{23}^\top\mathbf{K}_{22}^{-1}\mathbf{K}_{23} \end{bmatrix}}_{\mathbf{K}_{\text{red}}} \begin{bmatrix} \mathbf{u}_1 \\ \mathbf{u}_3 \end{bmatrix} \tag{B.19}$$

reveals the reduced stiffness matrix $\mathbf{K}_{\text{red}} \in \mathbb{R}^{6 \times 6}$ as the final result. Some quick benchmarks have shown that this method is less efficient than the numerical integration if high precision is required (i.e. large numbers of elements). That's why it is only used in VirtualBow's unit tests as a cross-check for the integration-based method.

B.3 Derivatives for the String Element

To complete the string element, the derivatives $\partial l / \partial \mathbf{u}$ and $\partial^2 l / \partial \mathbf{u}^2$ of the length with respect to the displacement vector are required. Since the total length l is a sum of partial lengths l_{ij} between the contact points, we can compute the derivatives of the partial lengths with respect to their adjacent nodes and assemble the full first derivative as

$$\frac{\partial l}{\partial \mathbf{u}} = \sum_k \frac{\partial l_{k-1,k}}{\partial \mathbf{u}} = \begin{bmatrix} \frac{\partial l_{01}}{\partial x_0} \\ \frac{\partial l_{01}}{\partial y_0} \\ \frac{\partial l_{01}}{\partial \varphi_0} \\ \frac{\partial l_{01}}{\partial x_1} \\ \frac{\partial l_{01}}{\partial y_1} \\ \frac{\partial l_{01}}{\partial \varphi_1} \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ \vdots \end{bmatrix} + \begin{bmatrix} 0 \\ 0 \\ 0 \\ \frac{\partial l_{12}}{\partial x_1} \\ \frac{\partial l_{12}}{\partial y_1} \\ \frac{\partial l_{12}}{\partial \varphi_1} \\ \frac{\partial l_{12}}{\partial x_2} \\ \frac{\partial l_{12}}{\partial y_2} \\ \frac{\partial l_{12}}{\partial \varphi_2} \\ 0 \\ 0 \\ 0 \\ \vdots \end{bmatrix} + \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ \frac{\partial l_{23}}{\partial x_2} \\ \frac{\partial l_{23}}{\partial y_2} \\ \frac{\partial l_{23}}{\partial \varphi_2} \\ \frac{\partial l_{23}}{\partial x_3} \\ \frac{\partial l_{23}}{\partial y_3} \\ \frac{\partial l_{23}}{\partial \varphi_3} \\ \vdots \end{bmatrix} + \dots \quad (\text{B.20})$$

The same applies to the second derivative $\partial^2 l / \partial \mathbf{u}^2$, which results in a block-diagonal structure of overlapping 6×6 blocks.

The first derivatives of l_{ij} with respect to the positions and rotations of nodes i and j are:

$$\frac{\partial l_{ij}}{\partial x_i} = \frac{x_i - x_j}{l_{i,j}}, \quad \frac{\partial l_{ij}}{\partial y_i} = \frac{y_i - y_j}{l_{i,j}}, \quad \frac{\partial l_{ij}}{\partial \varphi_i} = -d_i \left(\frac{\partial l_{ij}}{\partial x_i} \cos(\varphi_i) + \frac{\partial l_{ij}}{\partial y_i} \sin(\varphi_i) \right) \quad (\text{B.21})$$

$$\frac{\partial l_{ij}}{\partial x_j} = -\frac{\partial l_{i,j}}{\partial x_i}, \quad \frac{\partial l_{ij}}{\partial y_j} = -\frac{\partial l_{i,j}}{\partial y_i}, \quad \frac{\partial l_{ij}}{\partial \varphi_j} = -d_j \left(\frac{\partial l_{ij}}{\partial x_j} \cos(\varphi_j) + \frac{\partial l_{ij}}{\partial y_j} \sin(\varphi_j) \right) \quad (\text{B.22})$$

The second derivatives with respect to the positions x_i, x_j, y_i, y_j are (omitting symmetric ones since $\partial/\partial a\partial b = \partial/\partial b\partial a$):

$$\frac{\partial l_{ij}}{\partial x_i \partial x_i} = \frac{(y_j - y_i)^2}{l_{ij}^3} \quad (\text{B.23})$$

$$\frac{\partial l_{ij}}{\partial x_i \partial y_i} = -\frac{(x_j - x_i)(y_j - y_i)}{l_{ij}^3} \quad (\text{B.24})$$

$$\frac{\partial l_{ij}}{\partial x_i \partial x_j} = -\frac{\partial l_{ij}}{\partial x_i \partial x_i} \quad (\text{B.25})$$

$$\frac{\partial l_{ij}}{\partial x_i \partial y_j} = -\frac{\partial l_{ij}}{\partial x_i \partial y_i} \quad (\text{B.26})$$

$$\frac{\partial l_{ij}}{\partial y_i \partial y_i} = \frac{(x_j - x_i)^2}{l_{ij}^3} \quad (\text{B.27})$$

$$\frac{\partial l_{ij}}{\partial y_i \partial x_j} = -\frac{\partial l_{ij}}{\partial x_i \partial y_i} \quad (\text{B.28})$$

$$\frac{\partial l_{ij}}{\partial y_i \partial y_j} = -\frac{\partial l_{ij}}{\partial y_i \partial y_i} \quad (\text{B.29})$$

$$\frac{\partial l_{ij}}{\partial x_j \partial x_j} = \frac{\partial l_{ij}}{\partial x_i \partial x_i} \quad (\text{B.30})$$

$$\frac{\partial l_{ij}}{\partial x_j \partial y_j} = \frac{\partial l_{ij}}{\partial x_i \partial y_i} \quad (\text{B.31})$$

$$\frac{\partial l_{ij}}{\partial y_j \partial y_j} = \frac{\partial l_{ij}}{\partial y_i \partial y_i} \quad (\text{B.32})$$

The second derivatives involving the angles φ_i, φ_j are:

$$\frac{\partial^2 l_{ij}}{\partial x_i \partial \varphi_j} = -d_i \left(\frac{\partial^2 l_{ij}}{\partial x_i \partial x_i} \cos(\varphi_i) + \frac{\partial^2 l_{ij}}{\partial x_i \partial y_i} \sin(\varphi_i) \right) \quad (\text{B.33})$$

$$\frac{\partial^2 l_{ij}}{\partial y_i \partial \varphi_i} = -d_i \left(\frac{\partial^2 l_{ij}}{\partial y_i \partial y_i} \sin(\varphi_i) + \frac{\partial^2 l_{ij}}{\partial x_i \partial y_i} \cos(\varphi_i) \right) \quad (\text{B.34})$$

$$\frac{\partial^2 l_{ij}}{\partial \varphi_i \partial \varphi_i} = -d_i \left(\left(\frac{\partial^2 l_{ij}}{\partial x_i \partial \varphi_i} + \frac{\partial l}{\partial \bar{y}_i} \right) \cos(\varphi_i) + \left(\frac{\partial^2 l_{ij}}{\partial y_i \partial \varphi_i} - \frac{\partial l}{\partial \bar{x}_i} \right) \sin(\varphi_i) \right) \quad (\text{B.35})$$

$$\frac{\partial^2 l_{ij}}{\partial x_j \partial \varphi_j} = -d_j \left(\frac{\partial^2 l_{ij}}{\partial x_j \partial x_j} \cos(\varphi_j) + \frac{\partial^2 l_{ij}}{\partial x_j \partial y_j} \sin(\varphi_j) \right) \quad (\text{B.36})$$

$$\frac{\partial^2 l_{ij}}{\partial y_j \partial \varphi_j} = -d_j \left(\frac{\partial^2 l_{ij}}{\partial y_j \partial y_j} \sin(\varphi_j) + \frac{\partial^2 l_{ij}}{\partial x_j \partial y_j} \cos(\varphi_j) \right) \quad (\text{B.37})$$

$$\frac{\partial^2 l_{ij}}{\partial \varphi_j \partial \varphi_j} = -d_j \left(\left(\frac{\partial^2 l_{ij}}{\partial x_j \partial \varphi_j} + \frac{\partial l}{\partial \bar{y}_j} \right) \cos(\varphi_j) + \left(\frac{\partial^2 l_{ij}}{\partial y_j \partial \varphi_j} - \frac{\partial l}{\partial \bar{x}_j} \right) \sin(\varphi_j) \right) \quad (\text{B.38})$$

$$\frac{\partial^2 l_{ij}}{\partial x_i \partial \varphi_j} = -d_j \left(\frac{\partial^2 l_{ij}}{\partial x_i \partial x_j} \cos(\varphi_j) + \frac{\partial^2 l_{ij}}{\partial x_i \partial y_j} \sin(\varphi_j) \right) \quad (\text{B.39})$$

$$\frac{\partial^2 l_{ij}}{\partial y_i \partial \varphi_j} = -d_j \left(\frac{\partial^2 l_{ij}}{\partial y_i \partial y_j} \cos(\varphi_j) + \frac{\partial^2 l_{ij}}{\partial y_i \partial x_j} \sin(\varphi_j) \right) \quad (\text{B.40})$$

$$\frac{\partial^2 l_{ij}}{\partial \varphi_i \partial \varphi_j} = -d_i \left(\frac{\partial^2 l_{ij}}{\partial x_i \partial \varphi_j} \cos(\varphi_i) + \frac{\partial^2 l_{ij}}{\partial y_i \partial \varphi_j} \sin(\varphi_i) \right) \quad (\text{B.41})$$

$$\frac{\partial^2 l_{ij}}{\partial x_j \partial \varphi_i} = -d_i \left(\frac{\partial^2 l_{ij}}{\partial x_i \partial x_j} \cos(\varphi_i) + \frac{\partial^2 l_{ij}}{\partial x_i \partial y_j} \sin(\varphi_i) \right) \quad (\text{B.42})$$

$$\frac{\partial^2 l_{ij}}{\partial y_j \partial \varphi_i} = -d_i \left(\frac{\partial^2 l_{ij}}{\partial x_i \partial y_j} \cos(\varphi_i) + \frac{\partial^2 l_{ij}}{\partial y_i \partial y_j} \sin(\varphi_i) \right) \quad (\text{B.43})$$

B.4 Vibrations of a Viscoelastic Bar

Consider the bar shown in Figure B.1. It is fixed on the left hand side and free on the right, has a length of L and can undergo longitudinal motion as described by the continuous function $u(x, t)$. We want to calculate the undamped natural frequency ω_0 and associated damping ratio ζ for each eigenmode of the system.

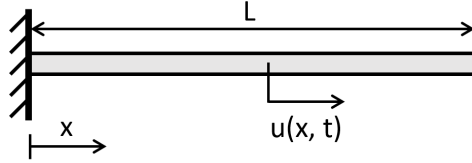


Figure B.1: Kinematics and boundary conditions of the bar

We are going to follow the usual derivation for longitudinal vibrations of bars that can be found in many mechanics textbooks, like [14], except that we introduce a viscoelastic material into the mix. The bar is assumed to behave according to the Kelvin-Voigt model, which expresses the normal stress σ in the bar as

$$\sigma = E \varepsilon + \eta \dot{\varepsilon} \quad (\text{B.44})$$

where $\varepsilon = \frac{\partial u}{\partial x}$ is the longitudinal strain, E the elastic modulus and η the viscosity of the material. The acceleration at every point in the bar is given by

$$\rho \frac{\partial^2 u}{\partial t^2} = \frac{\partial \sigma}{\partial x} \quad (\text{B.45})$$

where ρ is the bar's density. Combining the equations (B.44) and (B.45) and multiplying by the cross-section area A results in the equation of motion

$$\rho A \frac{\partial^2 u}{\partial t^2} = EA \frac{\partial^2 u}{\partial x^2} + \eta A \frac{\partial^3 u}{\partial x^2 \partial t}. \quad (\text{B.46})$$

This partial differential equation can be solved by separation of variables. The solution is assumed to be of the form $u(x, t) = X(x)T(t)$, a product of two functions depending only on x and t , respectively. Inserting this into the equation of motion (B.46) and separating terms depending on x from terms depending on t leads to

$$\begin{aligned} \rho A X(x) \ddot{T}(t) &= EA X''(x) T(t) + \eta A X''(x) \dot{T}(t) \\ \frac{X''(x)}{X(x)} &= \frac{\ddot{T}(t)}{\frac{EA}{\rho A} T(t) + \frac{\eta A}{\rho A} \dot{T}(t)} =: -\frac{\rho A}{EA} \omega_0^2 \end{aligned} \quad (\text{B.47})$$

Apply the usual argument: As one side of the equation depends on position and the other on time, they have to be constant in order to be equal. We choose that constant to be $-\frac{\rho A}{EA} \omega_0^2$ for reasons that will become clear later and obtain the two ordinary differential equations

$$\ddot{T}(t) + \frac{\eta A}{EA} \omega_0^2 \dot{T}(t) + \omega_0^2 T(t) = 0, \quad (\text{B.48})$$

$$X''(x) + \frac{\rho A}{EA} \omega_0^2 X(x) = 0. \quad (\text{B.49})$$

The time equation (B.48) describes a damped harmonic oscillator with undamped natural frequency ω_0 and damping ratio

$$\zeta = \frac{1}{2} \frac{\eta A}{EA} \omega_0 \quad (\text{B.50})$$

as determined by the coefficients of the equation. The last remaining task is to determine the natural frequencies ω_0 that solve the second differential equation (B.49) while also satisfying the boundary conditions of the bar. The general solution of (B.49) is

$$X(x) = C \cdot \sin \left(\sqrt{\frac{\rho A}{EA}} \omega_0 x - \varphi \right). \quad (\text{B.51})$$

Boundary condition on the fixed end:

$$\begin{aligned} u(0, t) = 0 : \quad & X(0) T(t) = 0 \\ & C \cdot \sin(-\varphi) = 0 \\ & \varphi = 0 \end{aligned} \quad (\text{B.52})$$

The case $T(t) = 0$ was ignored here because it is of no interest. The boundary condition for the free end is

$$\begin{aligned} \sigma(L, t) = 0 : \quad & E \varepsilon(L, t) + \eta \dot{\varepsilon}(L, t) = 0 \\ & EA \frac{\partial u}{\partial x}(L, t) + \eta A \frac{\partial^2 u}{\partial x \partial t}(L, t) = 0 \\ & EA X'(L) T(t) + \eta A X'(L) \dot{T}(t) = 0 \\ & (EA T(t) + \eta A \dot{T}(t)) X'(L) = 0 \\ & \ddot{T}(t) X'(L) = 0 \end{aligned}$$

And therefore the solution for the k -th natural frequency is

$$\begin{aligned}
 X'(L) &= C \cdot \sqrt{\frac{\rho A}{EA}} \omega_0 \cos \left(\sqrt{\frac{\rho A}{EA}} \omega_0 L \right) = 0, \\
 \omega_0 &= \frac{\pi(2k-1)}{2L} \cdot \sqrt{\frac{EA}{\rho A}}, \quad k \in \mathbb{Z}.
 \end{aligned} \tag{B.53}$$

And finally, with equations (B.50) and (B.53) the damping ratio follows as

$$\zeta = \frac{\pi(2k-1)}{4L} \cdot \frac{\eta A}{\sqrt{\rho A \cdot EA}}, \quad k \in \mathbb{Z}. \tag{B.54}$$

Appendix C

Unfinished Work

This chapter contains unfinished ideas and improvements that have already been partly documented while exploring them, but haven't made it into VirtualBow yet for various reasons.

C.1 Static Step Size Control

With the displacement controlled Newton-Raphson method we can solve for static equilibrium at any given displacement. We often want to do this repeatedly for a whole range of target displacements. The static simulation of the bow, for example, produces a series of equilibrium states from brace height to full draw, from which things like the force-draw curve can be extracted. Even if the intermediate states were of no interest, sometimes the target displacement is too far from the initial displacement for the Newton-Raphson iterations to converge, unless the displacement is applied in multiple smaller steps.

The easiest way to move through a range of displacements is to use a fixed number of equal steps. This works fine as long as there is good convergence at each step. However, in some parts of this equilibrium path the step size might be too large, leading to either no convergence or many iterations at each step. In this case we would like to decrease the step size temporarily. Conversely, if convergence gets better, we would like to increase the displacement step again for efficiency. This adaptation is called *step size control*.

The method intended for VirtualBow first calculates a lower and upper bound on the displacement step as

$$\Delta u_{\min} = \frac{u_{\text{target}} - u_{\text{start}}}{n_{\max}}, \quad (\text{C.1})$$

$$\Delta u_{\max} = \frac{u_{\text{target}} - u_{\text{start}}}{n_{\min}}, \quad (\text{C.2})$$

where $n_{\min}$ and $n_{\max}$ are parameters that determine the minimum resolution of the output and the maximum resolution, beyond which the procedure is aborted because the steps become too small. Another required parameter is the desired number of Newton iterations `iter_target` at each displacement step. We start the step size control with the maximum step of $\Delta u = \Delta u_{\max}$. After performing the Newton-Raphson iterations we adjust the step size depending on whether the iteration was successful or not:

Success: If the iterations converged, the step size is adjusted based on the number of iterations `iter_actual` that were needed and the desired number of iterations `iter_target`:

$$\Delta u := \text{clamp} \left\{ \Delta u \sqrt{\frac{\text{iter_target}}{\text{iter_actual}}}, \Delta u_{\min}, \Delta u_{\max} \right\} \quad (\text{C.3})$$

In other words, the step size is decreased if more iterations than desired were needed and increased if fewer were needed. Also, the step size is never decreased below $\Delta u_{\min}$ or increased above $\Delta u_{\max}$.

Failure: In the case of iteration failure, the number of iterations can't be used to adjust the step size. But we know that the step size has to be reduced in order to increase the chance of a successful iteration. That's why it is multiplied by a reduction factor $0 < r < 1$,

$$\Delta u := r \cdot \Delta u. \quad (\text{C.4})$$

The simulation is aborted if there was a failure and the step size cannot be decreased any further (because the lower bound $\Delta u_{\min}$ was already reached).

C.2 Backtracking Line Search

One last modification to the Newton-Raphson method, which could be important for the robustness of the static bow simulation, is a line search. As we saw previously, the Newton-Raphson method repeatedly computes solution increments $\Delta \mathbf{x}_i$ based on a linearization of the objective function around the current approximation $\mathbf{x}_i$ of the solution. But can we be sure that applying $\Delta \mathbf{x}_i$ actually improves the solution? We can interpret $\Delta \mathbf{x}_i$ as the direction in which the error decreases most rapidly at the current point of linearization. But when moving away from the point of linearization, the actual nonlinear objective function might very well change in ways that make the error go up again. So the step $\Delta \mathbf{x}_i$ can actually be too large, which potentially leads to slow convergence or even failure to converge on a solution at all. One example where such badly behaved objective functions can arise are systems with contact events, such as the contact between limb and string in our bow model.

The idea behind a line search is to keep the increments $\Delta \mathbf{x}_i$ produced by the Newton-Raphson iterations, but instead of applying them blindly we treat them as a direction in which to advance our solution. How much we actually travel in that direction is determined by a separate step, the so-called *line search*. First of all, the update step of the Newton-Raphson method is modified to

$$\mathbf{x}_{i+1} = \mathbf{x}_i + \alpha \Delta \mathbf{x}_i \quad (\text{C.5})$$

with the scalar step size $\alpha > 0$ that is to be determined by the line search. We define the error in the given search direction by a scalar *merit function* $g(\alpha)$, often chosen as the squared norm of the residual function. In our case we also include the scaling matrix $\mathbf{S}$ used by the convergence criterion,

$$g(\alpha) = \|\mathbf{S}^{-1} \mathbf{f}(\mathbf{x}_i + \alpha \Delta \mathbf{x}_i)\|^2. \quad (\text{C.6})$$

The conceptually simplest kind of line search is the exact line search, which consists of finding the parameter α_{min} that actually minimizes the merit function and therefore the residual error in the given search direction,

$$\alpha_{min} = \underset{\alpha > 0}{\operatorname{argmin}} g(\alpha). \quad (\text{C.7})$$

But this is easier to write down than to compute, and doing so would be fairly costly. Usually it is more advantageous to do a much cruder search and settle for a value of α that just decreases the error instead of exactly minimizing it. The saved computational effort can then be better invested in a new Newton iteration with a new search direction. Many different schemes have been established for inexact line searches. They can be divided into multiple categories. Zero-order methods only evaluate the merit function for

different values of α , but not its derivative. First-order methods also evaluate the first derivative

$$g'(\alpha) = \left(\frac{\partial g}{\partial \mathbf{f}} \right)^\top \left(\frac{\partial \mathbf{f}}{\partial \mathbf{x}} \cdot \frac{\partial \mathbf{x}}{\partial \alpha} \right) = 2 (\mathbf{S}^{-2} \mathbf{f}(\mathbf{x}_i + \alpha \Delta \mathbf{x}_i))^\top \mathbf{J}(\mathbf{x}_i + \alpha \Delta \mathbf{x}_i) \Delta \mathbf{x} \quad (\text{C.8})$$

of the merit function, assuming that $g(\alpha)$ is differentiable. Looking for a root of the first derivative of the merit function by bisection or other root-finding algorithms would be an example of a first-order method. Curve-fitting methods additionally try to approximate the merit function by some sort of curve, such as a polynomial, in order to achieve faster convergence. For VirtualBow, a zero-order method called *Armijo Backtracking* was chosen, since zero-order methods do not assume or require any sort of nice behavior from the merit function.

The core of the scheme is trying to find a value for α that fulfills the *Armijo condition*, also called the *condition of sufficient decrease*, which is written as

$$g(\alpha) \leq g(0) + c\alpha g'(0). \quad (\text{C.9})$$

Stated in words, the value of the merit function at step length α must be less than or equal to the linear projection according to $g(0)$ and the slope $g'(0)$, reduced by the Armijo constant $c \in (0, 1)$. Usually c is chosen between 0.1 and 0.0001. Note that the coefficients on the right hand side of this expression are constant during each line search, which is consistent with the method being zero-order.

As for the actual line searching, the method starts with the full step size $\alpha_0 = 1$ and checks if this fulfills the Armijo condition. If it already does, there is no line search to be done. We just take the full step and continue with the next Newton-Raphson iteration with barely any overhead from our line search method. Only if the initial step doesn't fulfill the condition do we start to backtrack by iteratively reducing the step by $\alpha_{k+1} = \tau \cdot \alpha_k$ with the backtracking factor $\tau \in (0, 1)$.

Constrained Newton-Raphson

To combine the constrained Newton-Raphson method with backtracking, we have to look at the complete system of equations, including the constraint and the load factor, as if it were a single equation. Augmented system of equations, denoted by an overline:

$$\overline{\mathbf{x}} = \begin{bmatrix} \mathbf{x} \\ \lambda \end{bmatrix}, \quad \overline{\mathbf{f}}(\overline{\mathbf{x}}) = \begin{bmatrix} \mathbf{f}(\mathbf{x}, \lambda) \\ c(\mathbf{x}, \lambda) \end{bmatrix} \quad (\text{C.10})$$

Jacobian and Newton-Raphson step:

$$\frac{\partial \bar{\mathbf{f}}}{\partial \bar{\mathbf{x}}} = \begin{bmatrix} \frac{\partial \mathbf{f}}{\partial \mathbf{x}} & \frac{\partial \mathbf{f}}{\partial \lambda} \\ \frac{\partial c}{\partial \mathbf{x}} & \frac{\partial c}{\partial \lambda} \end{bmatrix} \quad (\text{C.11})$$

Could leave it at that and use existing implementation, but efficiency etc. Instead, write Newton update step explicitly as

$$\underbrace{\begin{bmatrix} \frac{\partial \mathbf{f}}{\partial \mathbf{x}} & \frac{\partial \mathbf{f}}{\partial \lambda} \\ \frac{\partial c}{\partial \mathbf{x}} & \frac{\partial c}{\partial \lambda} \end{bmatrix}}_{\frac{\partial \bar{\mathbf{f}}}{\partial \bar{\mathbf{x}}}} \underbrace{\begin{bmatrix} \Delta \mathbf{x} \\ \Delta \lambda \end{bmatrix}}_{\Delta \bar{\mathbf{x}}} = - \underbrace{\begin{bmatrix} \mathbf{f}(\mathbf{x}, \lambda) \\ c(\mathbf{x}, \lambda) \end{bmatrix}}_{\bar{\mathbf{f}}(\bar{\mathbf{x}})} \quad (\text{C.12})$$

$$\frac{\partial \mathbf{f}}{\partial \mathbf{x}} \Delta \mathbf{x} + \frac{\partial \mathbf{f}}{\partial \lambda} \Delta \lambda = -\mathbf{f}(\mathbf{x}, \lambda) \quad (\text{C.13})$$

$$\frac{\partial c}{\partial \mathbf{x}} \Delta \mathbf{x} + \frac{\partial c}{\partial \lambda} \Delta \lambda = -c(\mathbf{x}, \lambda) \quad (\text{C.14})$$

Rest of derivation and concrete constraints from above. Convergence criterion:

$$\|\bar{\mathbf{S}}^{-1} \bar{\mathbf{f}}\|^2 = \|\mathbf{S}_f^{-1} \mathbf{f}\|^2 + |S_c^{-1} c|^2 < 1 \quad (\text{C.15})$$

Slope term of the Armijo condition:

$$\begin{aligned} g'(0) &= 2 \left(\bar{\mathbf{S}}^{-2} \bar{\mathbf{f}}^\top \right)^\top \bar{\mathbf{J}} \Delta \bar{\mathbf{x}} \\ &= 2 \left(\begin{bmatrix} \mathbf{S}_f^{-2} & \\ & S_c^{-2} \end{bmatrix} \begin{bmatrix} \mathbf{f} \\ c \end{bmatrix} \right)^\top \begin{bmatrix} \frac{\partial \mathbf{f}}{\partial \mathbf{x}} & \frac{\partial \mathbf{f}}{\partial \lambda} \\ \frac{\partial c}{\partial \mathbf{x}} & \frac{\partial c}{\partial \lambda} \end{bmatrix} \begin{bmatrix} \Delta \mathbf{x} \\ \Delta \lambda \end{bmatrix} \\ &= 2 (\mathbf{S}_f^{-2} \mathbf{f})^\top \left(\frac{\partial \mathbf{f}}{\partial \mathbf{x}} \Delta \mathbf{x} + \frac{\partial \mathbf{f}}{\partial \lambda} \Delta \lambda \right) + 2 S_c^{-2} c \left(\frac{\partial c}{\partial \mathbf{x}} \Delta \mathbf{x} + \frac{\partial c}{\partial \lambda} \Delta \lambda \right) \end{aligned}$$

Numerical Experiment

The figure shows the norm of the residual error for the static simulation of a cantilever beam with VirtualBow's FEM code. No line search has been used. The error doesn't decrease in each iteration; it zig-zags around until it eventually converges. Seems like something a line search would fix? The problem was that it never seemed to work when a line search was introduced that tried to force a monotonic decrease of the error in each iteration. So maybe the problem inherently needs the freedom to increase the error temporarily in order to eventually converge? Or the line search was not implemented correctly? In any case, some more thought needs to be put into this before line searching can make its way into VirtualBow.

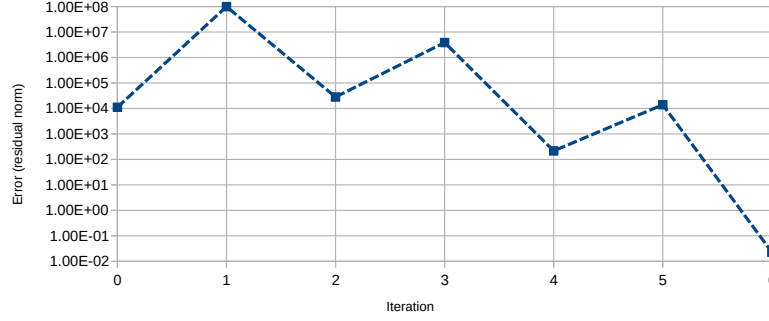


Figure C.1: Evolution of the error during Newton iterations without line search (cantilever beam example)

C.3 Generalized- α Method

The Newmark method is the basis for various other methods that improve on some of its numerical properties. One of those is the Generalized- α method [5], which itself is a generalization of various other Newmark-derived schemes. Its key improvement over the classical Newmark method is configurable numerical dissipation, i.e. damping of spurious oscillations in the high-frequency range that are not properly resolved. A good description and analysis of the method for the nonlinear case can be found in [8].

The Newmark approximations (6.6), (6.5) are left unchanged, but the balance equation (6.7) is modified to

$$M\mathbf{a}_{i+1-\alpha_m} + \mathbf{Q}_{i+1-\alpha_f} = \mathbf{P}_{i+1-\alpha_f}, \quad (\text{C.16})$$

so the equation of motion is not evaluated at the interval end t_{i+1} but at two different points $t_{i+1-\alpha_m}$ and $t_{i+1-\alpha_f}$ within the interval. This can be configured with the additional parameters α_m and α_f . The intermediate displacements, velocities and accelerations are interpolated linearly as

$$\mathbf{u}_{i+1-\alpha_f} = (1 - \alpha_f) \mathbf{u}_{i+1} + \alpha_f \mathbf{u}_i \quad (\text{C.17})$$

$$\mathbf{v}_{i+1-\alpha_f} = (1 - \alpha_f) \mathbf{v}_{i+1} + \alpha_f \mathbf{v}_i \quad (\text{C.18})$$

$$\mathbf{a}_{i+1-\alpha_m} = (1 - \alpha_m) \mathbf{a}_{i+1} + \alpha_m \mathbf{a}_i \quad (\text{C.19})$$

Several alternatives for evaluating the forces are described in [8]. The most simple one, referred to by the authors as the *generalized trapezoidal rule*, is to use linear interpolation as well:

$$\mathbf{Q}_{i+1-\alpha_f} = (1 - \alpha_f) \mathbf{Q}_{i+1} + \alpha_f \mathbf{Q}_i, \quad (\text{C.20})$$

$$\mathbf{P}_{i+1-\alpha_f} = (1 - \alpha_f) \mathbf{P}_{i+1} + \alpha_f \mathbf{P}_i. \quad (\text{C.21})$$

With this, the system of equations to solve at each timestep, including its Jacobian matrix, becomes

$$\mathbf{R}(\mathbf{a}_{i+1}) = \mathbf{M}\mathbf{a}_{i+1-\alpha_m} + \mathbf{Q}_{i+1-\alpha_f} - \mathbf{P}_{i+1-\alpha_f}, \quad (\text{C.22})$$

$$\frac{\partial \mathbf{R}}{\partial \mathbf{a}_{i+1}} = (1 - \alpha_m)\mathbf{M} + (1 - \alpha_f)(\gamma\Delta t\mathbf{D}_{i+1} + \beta\Delta t^2\mathbf{K}_{i+1}). \quad (\text{C.23})$$

The numerical damping of high-frequency modes is controlled by specifying the spectral radius $\rho_\infty \in [0, 1]$ at infinity. A value of 1 corresponds to no numerical dissipation, while a value of 0 corresponds to the so-called *asymptotic annihilation* of the high-frequency response. The other constants are chosen in terms of the spectral radius as

$$\alpha_m = \frac{2\rho_\infty - 1}{\rho_\infty + 1}, \quad \alpha_f = \frac{\rho_\infty}{\rho_\infty + 1}, \quad \beta = \frac{1}{4}(1 - \alpha_m + \alpha_f)^2, \quad \gamma = \frac{1}{2} - \alpha_m + \alpha_f. \quad (\text{C.24})$$

These relations guarantee second-order accuracy as well as unconditional stability in the linear case.

References

- [1] Karsten Ahnert and Mario Mulansky. Odeint – solving ordinary differential equations in c++. *AIP Conference Proceedings*, 1389(1):1586–1589, 2011.
- [2] K.J. Bathe. *Finite Element Procedures*. Prentice Hall, 2006.
- [3] Pål G. Bergan and Egil Mollestad. An automatic time-stepping algorithm for dynamic problems. *Computer Methods in Applied Mechanics and Engineering*, 49(3):299–318, 1985.
- [4] Marcello Berzeri, Marcello Campanelli, and Ahmed A. Shabana. Definition of the elastic forces in the finite-element absolute nodal coordinate formulation and the floating frame of reference formulation. *Multibody System Dynamics*, 5(1):21–54, 02 2001.
- [5] J. Chung and G. M. Hulbert. A Time Integration Algorithm for Structural Dynamics With Improved Numerical Dissipation: The Generalized- α Method. *Journal of Applied Mechanics*, 60(2):371–375, 06 1993.
- [6] M.A. Crisfield. *Non-linear Finite Element Analysis of Solids and Structures*, volume 1. John Wiley & Sons, 1991.
- [7] David M. Day and Timothy F. Walsh. Quadratic eigenvalue problems. Technical report, Sandia National Laboratories, 2007.
- [8] S. Erlicher, L. Bonaventura, and O. S. Bursi. The analysis of the generalized- α method for non-linear dynamic problems. *Computational Mechanics*, 28(2):83–104, 03 2002.
- [9] Carlos A. Felippa. *Lumped and Consistent Mass Matrices*. University of Colorado at Boulder.
- [10] F. N. Fritsch and R. E. Carlson. Monotone piecewise cubic interpolation. *SIAM Journal on Numerical Analysis*, 17(2):238–246, 1980.
- [11] GFAS — help.geostru.eu. https://help.geostru.eu/gfas/en/index.html?elemento_trave-pilastro_a_due_.htm. [Online; accessed 15-August-2026].

- [12] Guillermo Giraldo. How to compute the coefficients for rayleigh damping? <https://www.simscale.com/knowledge-base/rayleigh-damping-coefficients>, 2021. [Online; accessed 24-September-2024].
- [13] Dietmar Gross, Werner Hauger, Jörg Schröder, and Wolfgang A. Wall. *Technische Mechanik 2: - Elastostatik (Springer-Lehrbuch)*. Springer, Berlin, 11 edition, 2011.
- [14] Dietmar Gross, Werner Hauger, and Peter Wriggers. *Technische Mechanik 4 - Hydromechanik, Elemente der Höheren Mechanik, Numerische Methoden*. Springer, Berlin, 11 edition, 2011.
- [15] C. N. Hickman. The dynamics of a bow and arrow. *Journal of Applied Physics*, 8(6):404–409, 1937.
- [16] B. W. Kooi. On the mechanics of the bow and arrow. *Journal of Engineering Mathematics*, 15(2):119–145, 04 1981.
- [17] B. W. Kooi. On the mechanics of the modern working-recurve bow. *Computational Mechanics*, 8(5):291–304, 09 1991.
- [18] B. W. Kooi and J. A. Sparenberg. On the static deformation of a bow. *Journal of Engineering Mathematics*, 14(1):27–45, 01 1980.
- [19] B. W. Kooi and J. A. Sparenberg. On the mechanics of the arrow: Archer’s paradox. *Journal of Engineering Mathematics*, 31(2):285–303, 05 1997.
- [20] D. Kuhl and G. Meschke. Finite elemente methoden i & ii. Lehrstuhl für Statik und Dynamik, Ruhr-Universität Bochum, 2002.
- [21] G. Lallement and Daniel Inman. A tutorial on complex eigenvalues. *Proceedings of SPIE - The International Society for Optical Engineering*, 01 1995.
- [22] Thanh-Nam Le, Jean-Marc Battini, and Mohammed Hjjaj. Efficient formulation for dynamics of corotational 2d beams. *Computational Mechanics*, 48(2):153–161, 08 2011.
- [23] W. C. Marlow. Bow and arrow dynamics. *American Journal of Physics*, 49(4):320–333, 1981.
- [24] E. Marotta and P. Salvini. Analytical stiffness matrix for curved metal wires. *Procedia Structural Integrity*, 8:43–55, 2018. AIAS2017 - 46th Conference on Stress Analysis and Mechanical Engineering Design, 6-9 September 2017, Pisa, Italy.
- [25] Diogo Folador Rossi, Walnório Graça Ferreira, Webe João Mansur, and Adenilcia Fernanda Grobério Calenzani. A review of automatic time-stepping strategies on numerical time integration for structural dynamics analysis. *Engineering Structures*, 80:118–136, 2014.

- [26] Wikipedia. Castigliano's method — Wikipedia, the free encyclopedia. https://en.wikipedia.org/wiki/Castigliano's_method, 2026. [Online; accessed 24-July-2026].
- [27] Wikipedia. Graham scan — Wikipedia, the free encyclopedia. https://en.wikipedia.org/wiki/Graham_scan, 2026. [Online; accessed 05-August-2026].
- [28] Wikipedia. Sandwich theory — Wikipedia, the free encyclopedia. https://en.wikipedia.org/wiki/Sandwich_theory, 2026. [Online; accessed 16-August-2026].
- [29] Louie L. Yaw. 2d corotational beam formulation. Technical report, Walla Walla University, 11 2009.
- [30] Dan Zenkert. *An Introduction to Sandwich Structures*. Engineering Materials Advisory Services, 1995.